



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр Института №8

Группа М8О-206СВ-24

Направление подготовки 01.04.02 Прикладная математика и информатика

Программа Продуктовая разработка и разработка программных систем

Квалификация инженер-исследователь

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА (ДИССЕРТАЦИЯ)

На тему: Система автоматизированного обогащения товарных данных для интернет-магазина

Автор ВКРМ (диссертации) _____ Фролов Михаил Александрович _____ (_____)
(фамилия, имя, отчество полностью)

Научный руководитель _____ Судаков Владимир Анатольевич _____ (_____)
(фамилия, имя, отчество полностью)

Консультант _____ (_____)
(фамилия, имя, отчество полностью)

Рецензент _____ (_____)
(фамилия, имя, отчество полностью)

К защите допустить

И.О. Директора

Образовательного центра _____ Крылов Сергей Сергеевич _____ (_____)
(фамилия, имя, отчество полностью)

_____ 20__ г.

РЕФЕРАТ

Выпускная квалификационная работа (диссертация) состоит из 112 страниц, 11 рисунков, 33 таблиц, 58 использованных источников, 1 приложения. **ОБОГАЩЕНИЕ ТОВАРНЫХ ДАННЫХ, КАСКАДНАЯ АРХИТЕКТУРА, СЛАБЫЙ НАДЗОР, БАЙЕСОВСКАЯ СЕТЬ, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, КАЛИБРОВКА, ИЗВЛЕЧЕНИЕ ПРИЗНАКОВ, ВЕКТОРНЫЕ ПРЕДСТАВЛЕНИЯ**

Объект исследования. Процесс автоматизированного обогащения товарных каталогов интернет-магазинов с использованием открытых источников данных.

Предмет исследования. Гибридная каскадная архитектура с ансамблевой оценкой уверенности классификаторов и адаптивным обращением к большой языковой модели как запасному слою.

Цель работы. Разработать гибридную каскадную систему обогащения товарных данных для интернет-магазина с высокой точностью, стоимостью ниже прямого вызова большой языковой модели и возможностью локального развёртывания.

Методы. SBERT-векторизация, градиентный бустинг XGBoost с поатрибутными порогами уверенности, байесовская сеть, запасной слой на большой языковой модели; трёхъярусная эталонная разметка с консенсусом независимых языковых моделей; разбиение без пересечения товарных кодов.

Результаты. Пятислойная гибридная каскадная система на трёх категориях пищевых продуктов: 93,0 % сквозной точности при обращении к большой языковой модели лишь на 4,1 % ячеек из 16 360; только-каскадная точность — 95,5 %, по независимой ручной разметке — 86,7 %. Система доминирует по Парето над прямым вызовом: +9,2 п. п. точности, стоимость ниже в 580 раз.

Область применения. PIM-системы интернет-магазинов: автоматизация заполнения атрибутов карточек, снижение ручной нагрузки оператора в 12–25 раз, развёртывание на открытых моделях без передачи данных во внешние API.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	10
1 РАЗВЁРНУТАЯ АКТУАЛЬНОСТЬ ТЕМЫ	18
1.1 Проблема обогащения товарных каталогов интернет-магазина .	18
1.1.1 Роль товарных данных и неоднородность партнёрского ввода	18
1.1.2 Эффекты неполной карточки на пользовательский опыт . .	20
1.1.3 Существующие системы управления товарной информацией не решают задачу	21
1.1.4 Противоречие и постановка задачи	21
1.2 Литературный обзор	22
1.2.1 Извлечение признаков из текстовых описаний товаров . . .	22
1.2.2 Стоимостно-чувствительные каскады больших языковых моделей	23
1.2.3 Существующие промышленные РІМ-системы и анализ аналогов	24
1.2.4 Слабый надзор, калибровка и сопутствующая методология	25
1.3 Техническое задание на разрабатываемую систему	26
1.3.1 Функциональные требования	26
1.3.2 Нефункциональные требования	27
1.3.3 Архитектурные требования	27
1.3.4 Соотношение технического задания и решаемых задач работы	28
1.4 Выводы по главе 1	29
2 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РЕШЕНИЯ	31
2.1 Логика работы разрабатываемого программного обеспечения .	31
2.1.1 Формальная постановка задачи обогащения	31
2.1.2 Каскадная схема принятия решения	31
2.1.3 Поведение системы при выходе категории за пределы поддерживаемых	32
2.1.4 Метрики качества и стоимости	33
2.2 Архитектура и стек технологий	34
2.2.1 Архитектура системы	34

2.2.2	Обоснование выбора компонентов стека	36
2.2.3	Стек технологий — сводная таблица	37
2.3	Сценарий работы программного обеспечения	39
2.3.1	Сквозной пример прохождения запроса	39
2.4	Выводы по главе 2	40
3	ОПИСАНИЕ ПРОГРАММНО-ТЕХНОЛОГИЧЕСКОЙ РЕАЛИЗАЦИИ	41
3.1	Описание программного обеспечения	41
3.1.1	Программная реализация слоёв каскада	41
3.1.2	Демонстрационный комплекс и контракт API	44
3.2	Данные	45
3.2.1	Источник данных, фильтрация и стратифицированная выборка	45
3.2.2	Эталонная разметка: иерархическая схема трёх ярусов . .	46
3.2.3	Разбиение без пересечения товарных кодов (code-disjoint) .	47
3.2.4	Дополнительные источники сигналов	47
3.3	Доказательство работоспособности и применимости	48
3.3.1	Условия проверки и метрики	48
3.3.2	Стратификация точности по типу эталонного сигнала . . .	48
3.3.3	Воспроизводимость	49
3.3.4	Главный результат: точность и стоимость производственной конфигурации	50
3.3.5	Декомпозиция по категориям	53
3.3.6	Обобщение на новые бренды (brand-disjoint side-study) . .	55
3.3.7	Поатрибутная картина точности и доверительные интервалы	56
3.3.8	Анализ доминирования по Парето: почему каскад превосходит одиночный вызов LLM	58
3.3.9	Учёт точности маршрутизатора первого уровня	58
3.3.10	Вклад каждого слоя каскада	58
3.3.11	Поатрибутная калибровочная ошибка до и после калибровки	59
3.3.12	Таксономия ошибок каскада	60
3.3.13	Разбор примеров прохождения каскада	65
3.3.14	Сравнение классификаторов Слой 2: XGBoost, Random Forest, логистическая регрессия, MLP	68
3.3.15	Обоснование сохранения слоя регулярных выражений: сопоставление с обучаемыми альтернативами	69

3.3.16	Поведение байесовского валидатора и архитектурный вывод	71
3.3.17	Чувствительность к выбору порога отсечки	72
3.3.18	Стоимостно-чувствительный маршрутизатор: отрицательный результат на проверке гипотезы H1	73
3.3.19	Цикл «аудит → правки → переобучение → измерение» . .	75
3.3.20	Переобучение моделей и прирост каскада	76
3.3.21	Кросс-категорийная репликация цикла 3/3	76
3.3.22	Устойчивость каскада по языкам	77
3.3.23	Устойчивость каскада к шуму во входных данных	78
3.3.24	Симуляция активного обучения на 3-LLM consensus gold .	79
3.3.25	Уточнение схемы атрибутов: независимые свойства	80
3.3.26	Поатрибутные архитектурные решения	80
3.4	Выводы по главе 3	81
4	ОПИСАНИЕ РЕЗУЛЬТАТОВ	84
4.1	Руководство пользователя	84
4.1.1	Роли и интерфейсы	84
4.2	Сценарии применения и порядок внедрения	86
4.2.1	Встраивание в бизнес-процесс магазина	86
4.2.2	Сценарий многоязычного каталога	86
4.2.3	Применимость в смежных доменах	87
4.3	Риски развёртывания и план мониторинга	89
4.3.1	Мониторинг в производственном контуре	91
4.4	Что позволяет использование разработки (характеристика достигнутых результатов)	92
4.5	Выводы по главе 4	95
	ЗАКЛЮЧЕНИЕ	97
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	105
	ПРИЛОЖЕНИЕ А Электронные материалы	112

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе (диссертации) применяют следующие термины с соответствующими определениями:

Серебряная разметка — набор пар «товар — целевое значение атрибута», полученный автоматически из открытых источников данных (теги Open Food Facts) и используемый для обучения моделей. Содержит шум и неполноту; противопоставляется эталонной разметке

Эталонная разметка — пары «товар — целевое значение», прошедшие верификацию согласованным мнением нескольких языковых моделей и/или слепой разметкой более сильной языковой модели; используется как опорная истина при оценке каскада

Слабый надзор — режим обучения с использованием неполной или зашумлённой разметки, формируемой автоматически из доступных источников вместо ручной разметки экспертом

Каскадная архитектура — последовательность алгоритмических слоёв, каждый из которых обрабатывает только те объекты или атрибуты, которые не были закрыты предыдущими слоями с заданной уверенностью

Многоуровневая фильтрация по уверенности — формализация поведения каскада как последовательного конъюнктивного условия на прохождение калиброванных порогов уверенности на каждом слое. Реализует требование утверждённой темы об «ансамблевой оценке уверенности моделей»

Покрытие — доля ячеек «товар, атрибут» в тестовой выборке, для которых каскад дал предсказание выше порога уверенности на слоях 1–3 (без обращения к запасному слою)

Точность на покрытом срезе — доля верных предсказаний среди тех ячеек, для которых каскад дал ответ без обращения к запасному слою

Сквозная точность — точность системы на полной тестовой выборке, при которой отказы каскада засчитываются как неверные ответы (либо заменяются ответом запасного слоя при включённом Слое 4)

Разбиение без пересечения товарных кодов — построение разбиения выборки на обучающую и тестовую части, при котором ни один товарный код не встречается одновременно в обеих частях. Обеспечивает оценку качества системы на новых товарных позициях

Цикл «аудит — правки — переобучение — измерение» — методологический

приём улучшения системы, при котором обнаружение конкретных дефектов эталонной разметки через аудит приводит к правкам экстрактора, затем к переобучению моделей и измерению прироста точности на той же аудит-выборке

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе (диссертации) применяют следующие сокращения и обозначения:

OFF — Open Food Facts, открытый краудсорсинговый каталог продовольственных товаров под лицензией ODbL

SBERT — Sentence-BERT, многоязычная модель векторного представления предложений

XGBoost — Extreme Gradient Boosting, библиотека градиентного бустинга деревьев решений

ECE — Expected Calibration Error, ожидаемая ошибка калибровки вероятностей классификатора

CPD — Conditional Probability Distribution, условное распределение вероятности

DAG — Directed Acyclic Graph, направленный ациклический граф

BIC — Bayesian Information Criterion, критерий выбора структуры байесовской сети

PDO — Protected Designation of Origin, защищённое наименование товара

AOP — Appellation d'Origine Protégée, защищённое наименование товара (фр.)

LLM — large language model, большая языковая модель

TF-IDF — term frequency $\times$ inverse document frequency, частотно-инверсный показатель

OOD — out of distribution, выход за пределы обучающего распределения

AUROC — area under the ROC curve, площадь под кривой ошибок

ТЗ — техническое задание

BPMN — Business Process Model and Notation, модель и нотация бизнес-процессов

PIM — product information management, управление информацией о товарах

CJM — customer journey map, карта пути клиента

СМ — контент-менеджер

ML — machine learning, машинное обучение

API — application programming interface, программный интерфейс приложения

REST — representational state transfer, архитектурный стиль программного интерфейса

JSON — JavaScript Object Notation, текстовый формат обмена данными

CSV — comma-separated values, текстовый формат табличных данных с разделителями

DOCX — Office Open XML Document, формат текстового документа

E2E — end-to-end, сквозной — точность системы с учётом маршрутизации и отказа от ответа (None=wrong); production-realistic метрика (см. 2.1.4)

MPNet — Masked and Permuted Pre-training, архитектура трансформера, использованная в paraphrase-multilingual-mpnet-base-v2 для 768-мерных эмбедингов

NBSP — non-breaking space, неразрывный пробел; используется в LaTeX между числом и единицей

ODbL — Open Database License v1.0, лицензия share-alike для открытых баз данных (применяется к Open Food Facts)

TCO — total cost of ownership, совокупная стоимость владения системой за период

CI — confidence interval, доверительный интервал (95 % если не указано иное)

RF — Random Forest, ансамбль деревьев решений с bagging

MLP — multi-layer perceptron, многослойный перцептрон (полносвязная нейросеть)

ВВЕДЕНИЕ

Актуальность темы исследования.

Электронная коммерция продолжает расти. Годовой объём российского сегмента в 2025 году составил 11,5 трлн руб. [1], мировой рынок тоже показывает рост [2]. Каталоги крупных интернет-магазинов содержат десятки миллионов товарных позиций: только индексируемая часть каталога Ozon в открытых аналитических базах превышает 38 млн позиций [3]. Ежедневно туда добавляются тысячи новых записей от партнёров-поставщиков.

Карточка товара — центральный элемент цифровой витрины. От её полноты зависят фасетный поиск, релевантность ранжирования и рекомендаций, корректность аналитики и в итоге конверсия в покупки. Партнёрский ввод при этом неоднородный: для одного товара поставщик передаёт только название и бренд, для другого — расширенный набор полей с фотографиями, для третьего — состав на иностранном языке. Образующиеся пробелы исторически закрывают контент-менеджеры вручную: каждую карточку приходится открывать, проверять и дописывать десятки атрибутов. На каталоге в миллионы позиций и тысячах новых записей в сутки это уже не работает ни по затратам, ни по доступности знаний иностранных языков [4].

Системы управления товарной информацией (Product Information Management) — Akeneo, Pimcore, Salsify и аналогичные [5; 6; 7] — сложились из задачи хранить уже сформированную товарную информацию. Задачу извлечения этой информации из неструктурированного партнёрского ввода они системно не решают. Появляющиеся в них модули автоматического обогащения — это, как правило, обёртка над одной публичной большой языковой моделью с прямым вызовом на каждый товар. Подробное сопоставление приведено в 1.2.3. Такой вызов экономически приемлем только для каталогов небольшого и среднего размера и не даёт гарантий фактической корректности. На числовых полях большие языковые модели регулярно ошибаются (3.3.12). Рост ставок коммерческих API, ужесточение требований к локализации данных и отраслевой тренд на открытые модели в локальном развёртывании [8] дополнительно усиливают запрос на архитектурные решения, которые не сводятся к прямому вызову одной модели.

Отсюда вытекает противоречие, которое и определяет актуальность работы. С одной стороны, автоматизация наполнения карточек — инженерная

необходимость. С другой стороны, ни одна отдельная технология не покрывает все типы целевых атрибутов с приемлемой точностью и стоимостью одновременно. Правила надёжны на числовых атрибутах, но плохо покрывают поля со сложным смыслом. Классические классификаторы требуют большой размеченной выборки. Большие языковые модели универсальны, но дорогие и часто ошибаются на числовых полях.

В качестве разрешения этого противоречия в работе рассматривается гибридная каскадная архитектура. Это последовательность слоёв, в которой каждый следующий подключается только к тому остатку, который не закрыли предыдущие с нужной уверенностью. Такая композиция совмещает семантические возможности генерирующих моделей с предсказательной силой классического машинного обучения и автоматическую отбраковку недостоверных значений. С научной стороны актуальность работы — это проверка эффективности гибридного каскада на типовой задаче обогащения. С практической стороны — создание готового к производству комплекса с контролируемой стоимостью и возможностью локального развёртывания.

Цель работы.

Цель работы — разработать гибридную каскадную систему обогащения товарных данных интернет-магазина. Система должна точно заполнять категориальные атрибуты, обходиться дешевле прямого вызова большой языковой модели, допускать локальное развёртывание на открытых моделях и выигрывать у прямого вызова одновременно и по точности, и по стоимости.

Задачи работы.

Для достижения поставленной цели сформулированы и решены пять задач:

- 1) разработать архитектуру гибридного каскадного конвейера обработки товарных данных с пошаговой фильтрацией по уверенности и явным разделением слоёв. Слои каскада: предкаскадный маршрутизатор категорий, экстрактор регулярных выражений, классификатор машинного обучения на векторных представлениях, байесовский валидатор плотностей, запасной слой большой языковой модели. Пошаговая фильтрация формализует

требование утверждённой темы об ансамблевой оценке уверенности моделей как последовательное конъюнктивное условие на калиброванные пороги слоёв;

- 2) реализовать каждый слой каскада в виде отдельного программного компонента: экстрактор регулярных выражений; классификатор машинного обучения на многоязычных векторных представлениях партнёр-доступных полей; байесовский валидатор, который использует обученную сеть атрибутов как фильтр статистически маловероятных пар «бренд × значение»; запасной слой на основе большой языковой модели с поддержкой нескольких поставщиков, включая открытые модели для локального развёртывания;
- 3) построить эталонную разметку и проверочную выборку без пересечения товарных кодов (code-disjoint) для оценки качества системы на трёх категориях: паста, шоколад, сыры. Разметка сочетает слабый надзор от тегов открытого каталога, поатрибутное согласие трёх независимых больших языковых моделей и слепую разметку референсной моделью;
- 4) провести экспериментальную оценку разработанной системы: измерить точность сквозного прохождения каскада, поатрибутное покрытие и стоимость обработки на эталонной тестовой выборке; сопоставить полученные показатели с прямым использованием большой языковой модели; оценить вклад каждого слоя каскада, устойчивость к смещению по языкам входных данных и поведение байесовского валидатора как селективной валидации плотностей;
- 5) разработать демонстрационный программный комплекс. Комплекс должен обеспечивать работу оператора каталога с каскадом через веб-интерфейс и стабильный программный интерфейс. Поддерживаются три производственных сценария развёртывания: с премиум-коммерческой большой языковой моделью, с бюджетной коммерческой и с открытой моделью в локальном развёртывании.

Каждой задаче соответствует измеримый результат, изложенный в подразделе «Новизна и основные результаты».

Объект и предмет исследования.

Объектом исследования является процесс автоматизированного

обогащения товарных каталогов интернет-магазинов на основе открытых источников данных. На вход подаётся минимальный набор партнёр-доступных полей, на выходе требуется получить структурированный набор целевых атрибутов.

Предметом исследования является гибридная каскадная архитектура обработки товарных данных с ансамблевой оценкой уверенности классификаторов и выборочным обращением к большой языковой модели в роли запасного слоя. Рассматриваются её точность, стоимость, устойчивость к смещению входного распределения, а также вычислительные и эксплуатационные свойства, существенные для производственного применения.

Теоретические и методические основы работы.

Работа опирается на следующие методы и инструменты:

- 1) извлечение признаков из неструктурированного текста — регулярные выражения и многоязычный кодировщик `paraphrase-multilingual-mpnet-base-v2`;
- 2) классификатор уровня машинного обучения — градиентный бустинг XGBoost поверх векторных представлений;
- 3) валидатор атрибутов реализован вероятностной графической моделью на библиотеке `pgmpy`; структура сети строится алгоритмом Hill Climb по критерию BIC;
- 4) обучающая выборка формируется слабым надзором по тегам открытого каталога Open Food Facts;
- 5) эталонная разметка строится из трёх ярусов: детерминированные правила для нутриент-производных атрибутов, регуляторные теги и поатрибутное согласие трёх независимых больших языковых моделей со слепой проверкой референсной моделью; процедура подробно описана в 3.2.3;
- 6) качество измеряется на отложенной выборке без пересечения товарных кодов; доверительные интервалы — кластерным бутстрэпом с группировкой по брендам;
- 7) гипотезы о превосходстве каскада над одиночной языковой моделью проверяются по заранее объявленному протоколу с поправкой Бонферрони на множественные тесты.

Архитектурный подход к выбору модели по соотношению цена/качество опирается на работы L. Chen и соавторов FrugalGPT [9], P. Aggarwal и соавторов AutoMix [10], D. Ding и соавторов Hybrid LLM [11], а также на бенчмарк Q. J. Hu и соавторов RouterBench [12].

Новизна и основные результаты.

Каждой из сформулированных выше задач в работе соответствует не менее одного измеримого результата.

По задаче 1 — проектирование архитектуры каскада, разделы 2.1, 3.1.1. Спроектирован гибридный каскад из пяти последовательных слоёв; решение слоя принимается при одновременном выполнении калиброванных порогов уверенности предыдущих слоёв. Формальная постановка задачи допускает явный отказ системы от ответа и явно указывает слой-источник каждого предсказания.

По задаче 2 — реализация слоёв, раздел 3.1.2. Каскад реализован как набор независимо обновляемых программных компонентов. Разработана демонстрационная система: шлюз API на Go — ML-сервис на Python/FastAPI — статический веб-интерфейс. Система запускается одной командой и проходит проверку готовности всех слоёв через диагностический интерфейс.

По задаче 3 — построение эталонной разметки, раздел 3.2. Построена многоисточниковая эталонная разметка для трёх категорий товаров: теги открытого каталога, правила на нутриентах, поатрибутное согласие трёх независимых больших языковых моделей и слепая разметка референсной моделью. Разбиение на обучающую, валидационную и тестовую части выполнено без пересечения товарных кодов (code-disjoint).

По задаче 4 — экспериментальная оценка, разделы 3.3.2, 3.3.3, 3.3.4, 3.3.7. Получена сквозная точность 93,0 % (точность только-каскадной части 95,5 %, macro-F1 0,890) на тестовой выборке без пересечения товарных кодов (code-disjoint, новые SKU) из 16 360 ячеек по 20 парам «категория-атрибут» при 580-кратном сокращении стоимости в режиме «каскад + Gemini 2.5 Flash» относительно прямого вызова Claude Sonnet 4.5 (сюда входит архитектурный вклад каскада в 24 раза и удешевление модели). На независимом эталоне ручной разметки Claude Opus 4 (n=566 из 688 ячеек с ответом каскада) сквозная точность — 86,7 %. Выигрыш каскада по точности и стоимости одновременно подтверждён на пяти моделях запасного слоя. Вклад каждого

слоя измерен количественно, включая точечный вклад байесовского валидатора. Каскад покрывает 97,3 % ячеек.

По задаче 5 — разработка демонстрационного программного комплекса, разделы 4.1, 4.2. Реализована поддержка трёх производственных сценариев развёртывания: премиум-коммерческий, бюджетный коммерческий, локальный с открытой моделью. Поддерживаются три роли пользователей: контент-менеджер, системный аналитик, инженер машинного обучения.

На основании результатов сформулированы три пункта научной новизны. Их общая особенность относительно известных стоимостно-ориентированных каскадов для языковых задач — FrugalGPT [9], AutoMix [10], Hybrid LLM [11], RouterBench [12] — разнородность слоёв. Перечисленные работы комбинируют однотипные генеративные модели разной мощности; в настоящей работе исследован разнородный каскад «правила (регулярные выражения) — дискриминативная модель машинного обучения — генеративная большая языковая модель» с явным правилом маршрутизации на уровне пары «категория-атрибут». Подобной комбинации трёх парадигм для задачи обогащения товарных каталогов в литературе не зафиксировано.

Первый пункт научной новизны. Впервые количественно установлено доминирование по Парето разработанной гибридной каскадной системы над прямым использованием большой языковой модели на задаче обогащения товарных каталогов. На тестовой выборке без пересечения товарных кодов (code-disjoint, новые SKU) размером 16 360 ячеек одновременно достигаются: превышение точности на 9,2 процентных пункта (93,0 % против 83,8 %) и сокращение стоимости обработки в 580 раз (сюда входит архитектурный вклад каскада в 24 раза и удешевление модели запасного слоя). Компромисса между точностью и стоимостью при этом не возникает.

Второй пункт научной новизны. Эмпирически установлено: в сценарии локального развёртывания с открытой большой языковой моделью gpt-oss-120b в роли запасного слоя каскад компенсирует слабость малой открытой модели на сложных атрибутах за счёт специализированного слоя машинного обучения. Прирост сквозной точности — 21,3 процентных пункта над прямым использованием той же открытой модели (90,6 % против 69,3 %)

при сокращении стоимости в 810 раз относительно опорного прямого вызова Claude Sonnet 4.5. То есть архитектура инвариантна к выбору поставщика большой языковой модели.

Третий пункт научной новизны объединяет два самостоятельных результата. Первый — **заранее объявленный отрицательный исход проверки гипотезы H1**: впервые на задаче обогащения товарных каталогов по заранее объявленной процедуре с поправкой Бонферрони на множественные тесты эмпирически установлено, что статическое **reg-(категория, атрибут)** правило маршрутизации не уступает обучаемому XGBoost-маршрутизатору с учётом стоимости по сквозной точности при равных бюджетах вызовов запасного слоя. Результат уточняет применимость обучаемой маршрутизации к этому классу задач и согласуется с выводами бенчмарка RouterBench [12]. Второй — **перенос замкнутого цикла «аудит эталонной разметки — правка экстрактора признаков — переобучение моделей — повторное измерение» на категории внутри продуктового домена**. Цикл воспроизведён на трёх категориях из трёх с приростом сквозной точности на подвыборке проверенных проблемных ячеек +18,2, +16,3 и +14,4 процентных пункта для пасты, шоколада и сыров соответственно.

Апробация и внедрение результатов.

Разработка велась по практическим требованиям, типичным для крупных интернет-магазинов (российских и международных). Архитектура и эксплуатационные характеристики системы ориентированы на встраивание в производственный процесс обогащения товарного каталога без перестройки соседних звеньев (см. 4.2.1 — описание точки встраивания в маршрут «партнёрский фид → каталог → витрина»). Поддерживаются три производственных сценария развёртывания: с премиум-коммерческой большой языковой моделью (Claude Sonnet 4.5 / GPT-4o), с бюджетной коммерческой (Gemini 2.5 Flash) и с открытой моделью в полностью локальном развёртывании (gpt-oss-120b). Последний сценарий критичен для требований локализации обработки партнёрских данных.

Апробация выполнена в виде демонстрационного программного комплекса. Комплекс запускается одной командой и проверяется на работоспособность через диагностический программный интерфейс. Демонстрация подтверждает, что все слои каскада работают на трёх

категориях: паста, шоколад, сыры. Воспроизводимость численных показателей обеспечена сценарием `reproduce.sh`, который пересоздаёт основные результаты из артефактов в каталоге `datasets/processed/` репозитория. Разработанный комплекс готов к интеграции в системы управления товарной информацией розничной торговли (см. 4.2.1).

Структура и объём работы.

Выпускная квалификационная работа состоит из введения, четырёх глав, заключения и списка использованных источников. Общий объём работы — 115 страниц, основная часть (от введения до заключения включительно) — 90 страниц. Список использованных источников включает 58 наименований: 18 российских источников, индексируемых в РИНЦ и доступных в открытых российских репозиториях, и 40 зарубежных публикаций, отраслевых отчётов и стандартов.

Первая глава содержит обоснование актуальности темы, литературный обзор по трём направлениям и техническое задание (ТЗ) на разрабатываемую систему. Вторая глава посвящена теоретическому обоснованию решения. В ней приводятся формальная постановка задачи, обоснование архитектуры и стека технологий, а также сценарии прохождения запроса через каскад. Третья глава содержит описание программной реализации, использованных данных и доказательство работоспособности каскада на эталонной тестовой выборке без пересечения товарных кодов (`code-disjoint`). Четвёртая глава представляет порядок применения системы, сценарии её развёртывания и характеристику достигнутых результатов в формате «повышено / ускорено / сокращено». В заключении собраны выводы по главам, научная новизна и перспективы развития темы.

1 РАЗВЁРНУТАЯ АКТУАЛЬНОСТЬ ТЕМЫ

В главе обосновывается тема работы, систематизируются существующие подходы к автоматическому обогащению товарных данных и формулируется техническое задание на разрабатываемую систему.

1.1 Проблема обогащения товарных каталогов интернет-магазина

1.1.1 Роль товарных данных и неоднородность партнёрского ввода

Рынок электронной коммерции растёт: годовой оборот российского сегмента в 2025 году — 11,5 трлн руб. [1], мировой рынок тоже расширяется [2]. Каталоги крупных площадок насчитывают десятки миллионов позиций (индексируемая часть Ozon — больше 38 млн [3]); ежедневно добавляются тысячи новых артикулов. Карточка товара — центральный элемент цифровой витрины: от её атрибутов (тип, состав, маркировки качества, страна происхождения) зависят фасетный поиск, релевантность ранжирования и конверсия.



Рисунок 1 – Место системы автоматизированного обогащения в инфраструктуре интернет-магазина

Партнёрские данные сильно неоднородны: для одного товара поставщик передаёт только название и бренд, для другого — расширенный набор полей с фотографиями, для третьего — состав на иностранном языке, для четвёртого

доступен только штрихкод. Анализ Open Food Facts [13] показывает, что заполненность нутриентных полей в среднем ниже, чем у базовых. Пробелы исторически закрываются контент-менеджерами вручную; при каталоге в миллионы позиций и тысячах новых записей в сутки такой путь экономически не подходит [4] и плохо масштабируется по языкам (FR, DE, IT, ES в категориях импортной пищевой продукции).

Типовая работа контент-менеджера сводится к шагам «приём — поиск аналогов — заполнение — внутренняя проверка — публикация» (рисунок 1.2, а). Гибридный каскад заменяет ручное заполнение автоматическим выводом системы, а ручной труд сосредотачивается на выборочной проверке и периодическом аудите ошибок (рисунок 1.2, б).

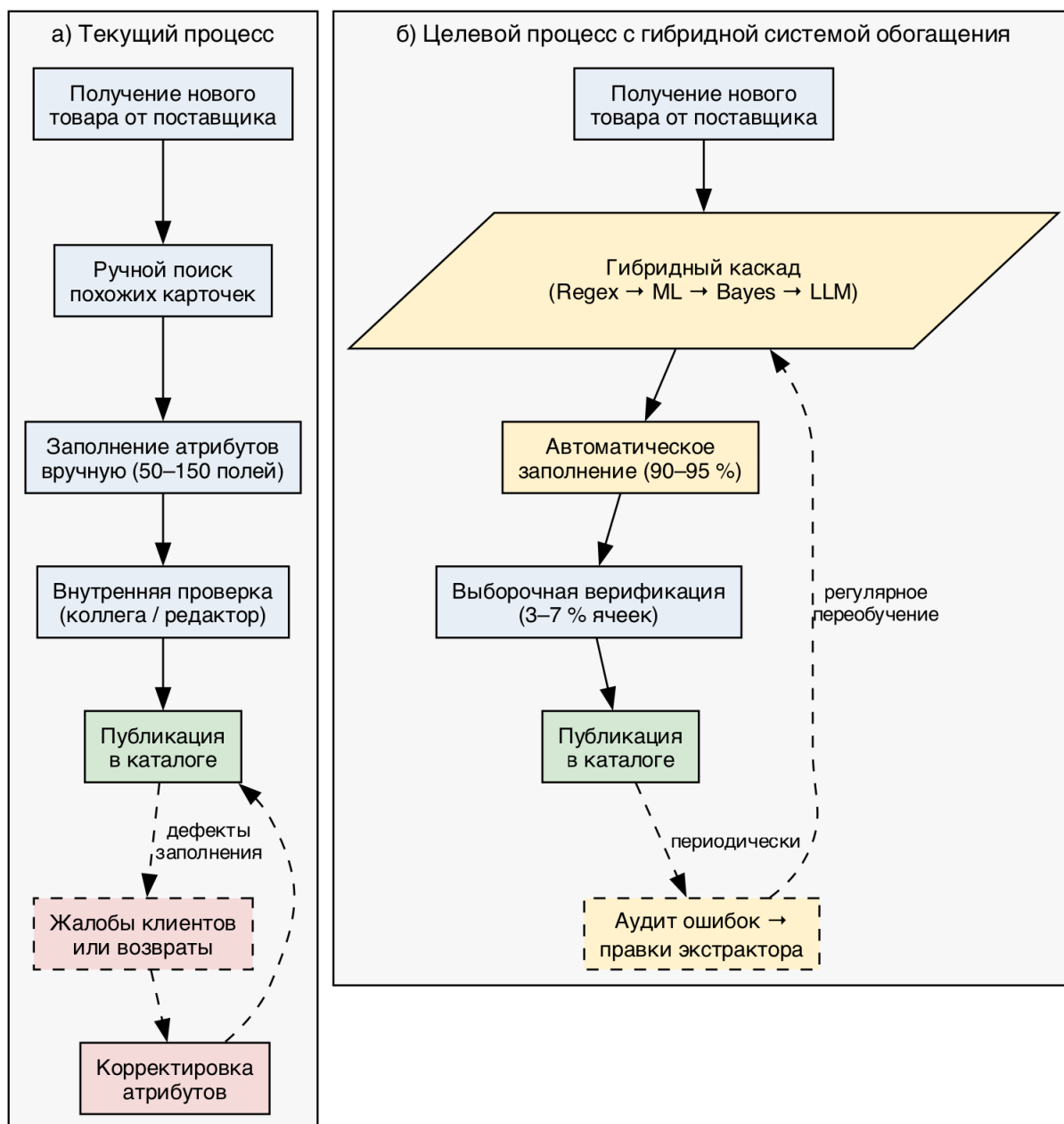


Рисунок 2 – Процесс работы контент-менеджера: (а) текущий, (б) целевой с интеграцией гибридного каскада обогащения

Примечание. Этапы текущего процесса реконструированы по материалам публичной документации Akeneo [5], Pimcore [6] и Salsify [7]; целевой процесс — авторская конструкция.

1.1.2 Эффекты неполной карточки на пользовательский опыт

Неполная карточка бьёт по магазину сразу с трёх сторон. Падает качество фасетного поиска (товар выпадает из подборок при применении

фильтров). Растут возвраты и негативная обратная связь по атрибутам правового или диетического значения (органика, глютен, аллергены). Увеличивается нагрузка на модерацию — «брошенная» карточка удлиняет цикл выхода товара на витрину.

1.1.3 Существующие системы управления товарной информацией не решают задачу

Системы управления информацией о товарах (PIM, product information management) — Akeneo, Pimcore, Salsify, Riversand, inRiver [5; 6; 7] — выросли из корпоративной потребности хранить и публиковать товарную информацию в едином каталоге. Их функциональность сосредоточена на управлении уже сформированной информацией, не на её извлечении. Сравнительный анализ платформ управления нормативно-справочной информацией с учётом MDM/PIM-функциональности проведён С. А. Долгоруковой [14]. Появляющиеся модули автоматического обогащения — это обёртки над одной публичной большой языковой моделью (LLM, large language model) с прямым вызовом на каждый товар. Они экономически приемлемы только для каталогов небольшого и среднего размера и не дают гарантий корректности. Альтернатива в виде агрегаторов готовых описаний задачу извлечения атрибутов из произвольного партнёрского текста не решает и ограничена крупными брендами.

1.1.4 Противоречие и постановка задачи

Отсюда противоречие в основе актуальности: автоматизация наполнения — инженерная необходимость, но ни одна отдельная технология не покрывает все типы целевых атрибутов с приемлемой точностью и стоимостью. Правила надёжны на числовых атрибутах, но мало покрывают поля со сложным смыслом. Классические классификаторы требуют большой разметки и плохо переносятся между языками. LLM универсальны, но дорогие [9] и регулярно ошибаются на числовых полях, где надо агрегировать данные из текста (эмпирическая характеристика — 3.3.2.4).

Из этого вытекает рабочая архитектурная гипотеза: каскадная композиция слоёв, в которой каждый следующий слой подключается только к остатку, не закрытому предыдущими с нужной уверенностью, способна

совместить преимущества отдельных технологий. Проверка этой гипотезы — содержательное ядро работы.

1.2 Литературный обзор

В разделе систематизируется научный и индустриальный задел по трём направлениям, которые образуют основу разрабатываемой системы:

извлечение признаков из текстовых описаний товаров;
каскады больших языковых моделей с учётом стоимости вызовов;
существующие промышленные системы управления товарной информацией.

1.2.1 Извлечение признаков из текстовых описаний товаров

Задача восполнения пропущенных атрибутов — это частный случай извлечения признаков из неструктурированного текста (information extraction, IE). Систематизация методов IE приведена в обзоре Л. М. Ермаковой [15]; гибридная природа практических решений по русскоязычным текстам отмечена в работе А. С. Ванюшкина и Л. А. Гращенко [16]. Подходы делятся на три группы — формальные правила, классические статистические модели и нейросетевые трансформеры — и их сочетание лежит в основе каскадных архитектур.

Правила и регулярные выражения надёжно извлекают атрибуты с устойчивой лексической разметкой при околонулевой стоимости, но ограничены низким покрытием на полях со сложным смыслом. **Статистические модели** над TF-IDF и градиентный бустинг (XGBoost [17], LightGBM [18]) работают при достаточной выборке и одноязычном входе (применение к электронной коммерции — Г. И. Афанасьев и соавторы [19]). **Векторные представления** на основе трансформеров [20] и BERT [21], в частности Sentence-BERT [22] и многоязычные дистиллированные варианты [23], дают фиксированные представления предложений; категоризация товарных каталогов рассмотрена А. Г. Колониным [24], систематическое изложение IE — в учебнике Е. И. Большаковой и соавторов [25].

Специализированные системы извлечения атрибутов товаров. В индустриальной литературе выделяются три эталонные системы: TXtract [26] (Karamanolakis et al., ACL 2020, Amazon) — многозадачный кодировщик с

условием на таксономию; MAVE [27] (Yang et al., WSDM 2022, Google) — BERT с кросс-вниманием по нескольким полям карточки; OpenTag [28] (Zheng et al., KDD 2018) — BiLSTM-CRF с механизмом внимания. Эти системы дают 85–92 % F1 на специализированных корпусах. Сравнения с каскадными архитектурами, в которых LLM работает запасным слоем, в открытой литературе системно нет; настоящая работа частично закрывает этот пробел.

Поколение 2024 года. Свежие работы идут по двум линиям. L. Yang и соавторы предложили EAVE [29] — лёгкий энкодер со sparse-layer-интеракцией, который даёт сопоставимое качество дешевле на выводе. С. Fang и соавторы предложили LLM-Ensemble [30] — взвешенный ансамбль коммерческих LLM с итеративной коррекцией; показали, что ни одна модель не выигрывает на всех типах атрибутов. В отличие от обеих линий, настоящая работа не заменяет специализированный экстрактор более компактным трансформером и не наращивает число параллельных вызовов LLM. Предлагается перпендикулярная композиция: большая языковая модель вызывается выборочно — только для тех (товар, атрибут), на которых классические слои не вышли на калиброванный порог уверенности.

1.2.2 Стоимостно-чувствительные каскады больших языковых моделей

С появлением спектра LLM разной мощности оформилась подобласть маршрутизации LLM с учётом стоимости (cost-aware LLM routing). FrugalGPT [9] (L. Chen, M. Zaharia, J. Zou) предложил каскад LLM по возрастанию стоимости с прерыванием на первом уверенном ответе; настоящая работа опирается на идеи последовательного прерывания и калибровки. AutoMix [10] (P. Aggarwal, A. Madaan и соавторы) основан на самопроверке ответа малой моделью; в разрабатываемой системе роли валидатора и предсказателя, наоборот, разнесены по разным слоям. Hybrid LLM [11] (D. Ding, A. Mallick и соавторы) рассматривает маршрутизацию между моделью на пограничном устройстве и облачной — постановка прямо соответствует сценарию «локальная gpt-oss-120b + коммерческое API как запасной слой». RouterBench [12] (Q. J. Hu, J. Bieker и соавторы) показал, что обучаемые маршрутизаторы не выигрывают у простых политик, когда ошибки локализованы по доменам; настоящая работа подтверждает это на задаче обогащения каталогов по заранее объявленному протоколу.

RouteLLM [31] (I. Ong, A. Almahairi и соавторы, ICLR 2025) обучил маршрутизатор поверх пользовательских предпочтений Chatbot Arena и снизил стоимость диалога общего назначения в два раза. Но подход опирается на парные предпочтения и сводится к выбору одной из двух моделей на запрос. В задаче обогащения каталога такого сигнала нет в принципе, а маршрутизация ведётся на уровне пары (товар, атрибут), а не запроса. Это обосновывает переход от обучаемого маршрутизатора к статической политике.

1.2.3 Существующие промышленные PIM-системы и анализ аналогов

Для позиционирования разработки отобраны три коммерческие PIM-системы — Akeneo, Pimcore и Salsify — и построена сравнительная таблица по семи функциональным критериям (таблица 1.1). Источники — документация вендоров [5; 6; 7].

Таблица 1 – Сравнение существующих PIM-систем и разрабатываемой системы

№	Критерий	Akeneo	Pimcore	Salsify	Разрабатываемая система
1	Централизованное хранение каталога	да	да	да	не входит в задачу
2	Распределение по каналам сбыта	да	да	да	не входит в задачу
3	Автоматическое обогащение атрибутов	частично (Akeneo AI / GenAI, 2023+ — обёртка над сторонней LLM)	частично (правила обогащения + плагины)	частично (Salsify AI Suite — обёртка над сторонней LLM)	да (каскадная архитектура с многоуровневой фильтрацией по уверенности)
4	Извлечение признаков из неструктурированного текста	нет	нет	нет	да (слои ML и LLM)

Продолжение таблицы 1

№	Критерий	Akeneo	Pimcore	Salsify	Разрабатываемая система
5	Поддержка многоязычного входа	через ручные переводы	через ручные переводы	через ручные переводы	встроенная (многоязычные векторные представления, 50+ языков)
6	Открытый исходный код	community-редакция	да	нет	да (включая возможность локального развёртывания LLM)
7	Стоимостно-чувствительная обработка	нет	нет	нет	да (каскадная композиция, доминирующая по Парето над одиночным вызовом языковой модели)

Из таблицы 1.1 видно, что коммерческие PIM-системы покрывают хранение и распределение уже сформированной информации, а функция автоматического обогащения у всех трёх (Akeneo с 2023 года, Salsify через AI Suite, Pimcore через плагины) — это прямая обёртка над сторонней LLM. Без учёта стоимости массовых вызовов, без поатрибутной маршрутизации, без гарантий локального развёртывания. Ни одно из решений не закрывает весь набор критериев интернет-магазина целиком — отсюда смысл самостоятельной разработки.

1.2.4 Слабый надзор, калибровка и сопутствующая методология

Главная сложность задачи — нет достаточной ручной разметки (weak supervision); инструментарий систематизирован Snorkel [32], обзор активного обучения и слабого надзора — B. Settles [33]. Источником слабого эталона выступает Open Food Facts [13]. Устойчивость классических алгоритмов

(KNN, SVM) к шуму разметки измерена В. А. Дюком [34], что обосновывает применимость серебряной разметки.

Для корректной работы каскада выходы классификаторов калибруются: масштабирование Платта [35], изотоническая регрессия [36], анализ ECE — С. Guo и соавторы [37]. Критерии качества классификаторов, включая ROC и выбор порога, систематизированы Ю. С. Шуниной и соавторами [38] и А. А. Ковалевым и соавторами [39]. Оценка точности — кросс-валидация и бутстрэп [40], интервалы пропорций — формула Вильсона [41]. Аппарат байесовских сетей опирается на J. Pearl [42], D. Heckerman et al. [43], А. Л. Тулупьева и соавторов [44], библиотеку pgmpy [45], поиск структуры — Hill Climb + BIC [46]. Прикладные вопросы устойчивости ML-моделей к дрейфу в каталогах — А. А. Артемов [47], масштабируемость на маркетплейсах — С. А. Сергеев [48], краудсорсинг — Р. А. Гилязов и Д. Ю. Турдаков [49], семантические пространства — В. Ф. Хорошевский [50].

1.3 Техническое задание на разрабатываемую систему

В разделе формулируется техническое задание (ТЗ) на разрабатываемую систему. Оно подразделяется на три группы требований: функциональные (что система должна делать), нефункциональные (с какими характеристиками) и архитектурные (каким способом она организована).

1.3.1 Функциональные требования

Ф-1. Входные данные. Система принимает товар, описанный партнёрскими полями: `product_name` (обязательное), `brands`, `ingredients_text`, `quantity`, `category` (опционально).

Ф-2. Возврат значений целевых атрибутов. Для каждого атрибута возвращается предсказанное значение, оценка уверенности и идентификатор слоя-источника, что обеспечивает отслеживаемость и возможность ручной верификации.

Ф-3. Покрывтие категорий. Поддерживается заполнение атрибутов для трёх категорий пищевой продукции — паста, шоколад и шоколадные изделия, сыры. Выбор обоснован достаточным объёмом данных в OFF, доступностью аудит-эталона (согласие трёх LLM + слепая разметка Opus) и репрезентативностью по типам атрибутов (числовые, бинарные,

мультиклассовые семантические).

Ф-4. Автоматическое определение категории. Если категория не указана, она определяется автоматически предкаскадным слоем 0.

Ф-5. Запасной слой на основе LLM. Когда слои 1–3 не дают уверенного ответа на конкретный атрибут, система обращается к LLM селективно — только по тем атрибутам товара, где предыдущие слои отказались.

Ф-6. Диагностический вывод. Возвращается результат работы валидатора, агрегированные оценки уверенности по слоям и маршрут запроса по каскаду — для отладки, аудита и накопления операционных данных.

1.3.2 Нефункциональные требования

НФ-1. Точность. Целевая сквозная точность на code-disjoint тестовой выборке — не менее 85 %; в конфигурации с коммерческой LLM в запасном слое — не менее 90 %.

НФ-2. Стоимость. Стоимость обработки одного товара каскадом — существенно ниже прямого вызова LLM; целевое снижение — не менее 2,5×, фактически достижимое значение фиксируется в 3.3.2.

НФ-3. Производительность. Время отклика на один товар без запасного слоя — не более 500 мс на потребительском ноутбуке без ускорителя; с запасным слоем — не более 5 с (определяется временем ответа внешнего API).

НФ-4. Многоязычность. Поддерживаются пять рабочих языков европейского рынка пищевой продукции (FR, EN, DE, IT, ES), наиболее представленных в OFF.

НФ-5. Локальное развёртывание. Поддерживается сценарий полностью локального развёртывания через открытую модель gpt-oss-120b в запасном слое — требование вендорной независимости.

1.3.3 Архитектурные требования

А-1. Каскадная архитектура с отдельными слоями. Последовательность слоёв по принципу «передачи остатка»: каждый последующий обрабатывает только атрибуты, не закрытые предыдущими с уверенностью выше калиброванного порога.

А-2. Независимое обновление слоёв. Изменение или переобучение

любого отдельного слоя (правил, классификаторов слоя 2, LLM в запасном слое) не должно требовать перезапуска остальных слоёв и API-шлюза.

А-3. Демонстрационный комплекс с веб-интерфейсом. Система сопровождается комплексом из трёх компонентов: API-шлюза на Go, ML-сервиса на Python/FastAPI и одностраничного веб-интерфейса — для верификации работоспособности и наглядного представления результатов контент-менеджеру.

1.3.4 Соотношение технического задания и решаемых задач работы

Сформулированные требования полностью покрываются задачами, поставленными во введении (таблица 1.2).

Таблица 2 – Соответствие требований технического задания и задач работы

№ задачи	Краткая формулировка задачи	Покрываемые требования ТЗ
1	Разработать архитектуру гибридного каскадного конвейера	Ф-1, Ф-2, Ф-4, Ф-5, А-1
2	Реализовать каждый слой каскада как самостоятельный программный компонент	Ф-6, А-2
3	Построить эталонную разметку и проверочную выборку без пересечения товарных кодов (code-disjoint)	Ф-3 (в части обеспечения данных для оценки на трёх категориях)

Продолжение таблицы 2

№ задачи	Краткая формулировка задачи	Покрываемые требования ТЗ
4	Провести экспериментальную оценку точности, покрытия и стоимости	НФ-1, НФ-2, НФ-3, НФ-4, НФ-5
5	Разработать демонстрационный программный комплекс	А-3

Установленное соответствие подтверждает, что выполнение пяти задач работы влечёт за собой выполнение технического задания в полном объёме.

1.4 Выводы по главе 1

В главе проведён анализ предметной области, систематизировано состояние литературы и обосновано техническое задание. По результатам анализа можно сформулировать следующие выводы.

- 1) Проблема автоматического обогащения остаётся открытой. Промышленные PIM-системы — Akeneo [5], Pimcore [6], Salsify [7] — нацелены на хранение и распределение уже сформированной информации. Появляющиеся модули автоматического обогащения — обёртки над одной коммерческой большой языковой моделью; для обработки больших каталогов с учётом стоимости они не годятся.
- 2) Научные подходы делятся на две группы, и ни одна из них не покрывает задачу полностью. Специализированные системы извлечения атрибутов — TXtract [26], MAVe [27], OpenTag [28] — решают задачу качественно, но требуют большой размеченной выборки и не учитывают экономики обращения к моделям. Каскады больших языковых моделей с учётом стоимости — FrugalGPT, AutoMix, Hybrid LLM — дают инструменты управления стоимостью (сопоставление есть в бенчмарке RouterBench), но рассматривают композицию из однородных моделей и не интегрируют классическое машинное обучение.

- 3) Разрабатываемая система объединяет оба подхода в гибридный каскад. Каскад совмещает специализированные слои извлечения признаков — правила, машинное обучение на многоязычных векторных представлениях, байесовский валидатор — и запасной слой LLM с учётом стоимости. Шесть функциональных, пять нефункциональных и три архитектурных требования ТЗ образуют обязательство, выполнение которого проверяется в экспериментальной части работы.

2 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РЕШЕНИЯ

В главе формально описывается разрабатываемая система и обосновываются принятые архитектурные решения. Сначала приводится формальная постановка задачи и логика работы программного обеспечения (2.1). Затем рассматриваются архитектура и стек технологий, обоснование выбора каждой компоненты (2.2). В конце разбирается прохождение типового запроса через систему (2.3).

2.1 Логика работы разрабатываемого программного обеспечения

2.1.1 Формальная постановка задачи обогащения

Пусть товарная позиция, поступающая в систему от партнёра, представлена кортежем партнёр-доступных полей

$$x = (x^{\text{name}}, x^{\text{brand}}, x^{\text{ingr}}, x^{\text{qty}}), \quad (1)$$

где компоненты — название, бренд, состав и количество соответственно; пустым может быть любое поле, кроме названия. Каждому товару ставится в соответствие категория $c \in \mathcal{C} = \{c_1, \dots, c_M\} \cup \{\text{OOD}\}$ ($M = 3$: pasta, chocolate, cheeses; OOD — выход за пределы поддерживаемых доменов), определяемая отдельной моделью

$$c = f_0(x), \quad (2)$$

(Слой 0, см. 2.2.1). Для каждой категории задана схема $\mathcal{A}_c = \{a_1, \dots, a_{K_c}\}$ с областями $\mathcal{Y}_k$; система строит

$$\hat{y} = (\hat{y}_1, \dots, \hat{y}_{K_c}) \in (\mathcal{Y}_1 \cup \{\text{null}\}) \times \dots \times (\mathcal{Y}_{K_c} \cup \{\text{null}\}), \quad (3)$$

где $\hat{y}_k = \text{null}$ — явный отказ от ответа. Честное молчание лучше неверного заполнения: повторное обращение к оператору дешевле ошибки на витрине.

2.1.2 Каскадная схема принятия решения

Логика обогащения реализована как каскад из четырёх содержательных

слоёв; ему предшествует определение категории. Для каждого атрибута k итоговое предсказание формируется по правилу

$$\hat{y}_k = \begin{cases} f_k^{\text{regex}}(x), & f_k^{\text{regex}}(x) \neq \emptyset; \\ f_k^{\text{ML}}(x), & f_k^{\text{regex}}(x) = \emptyset, P_{\text{ML}}(\hat{y}_k | x) \geq \tau_k, P_B(\hat{y}_k | \mathbf{e}) \geq \theta_k; \\ f_k^{\text{LLM}}(x), & \text{otherwise.} \end{cases} \quad (4)$$

Здесь f_k^{regex} — детерминированный экстрактор регулярных выражений (Слой 1); f_k^{ML} — классификатор машинного обучения (Слой 2) над многоязычным векторным представлением партнёр-доступных полей с калиброванной апостериорной вероятностью P_{ML} и поатрибутным порогом τ_k ; $P_B(\hat{y}_k | \mathbf{e})$ — оценка байесовской сети при свидетельствах $\mathbf{e} = (x^{\text{brand}}, \hat{y}_{j \neq k})$ с порогом θ_k , заданным как пятый перцентиль распределения P_B по верно размеченным обучающим примерам; f_k^{LLM} — запасной слой на большой языковой модели (LLM, large language model), возвращающий значение из $\mathcal{Y}_k$ или null. Предсказание Слоя 2 принимается тогда и только тогда, когда одновременно выполнены

$$[P_{\text{ML}}(\hat{y}_k | x) \geq \tau_k] \wedge [P_B(\hat{y}_k | \mathbf{e}) \geq \theta_k]. \quad (5)$$

Это конъюнктивное условие формализует требование темы об ансамблевой оценке уверенности: не параллельное голосование, а последовательная фильтрация с правом каждого слоя отказаться. По итогам 3.3.4 валидатор включён точно — только на парах «категория — атрибут» с неотрицательным приростом. Схема даёт ступенчатое распределение нагрузки (дешёвый Слой 1 — массовые случаи, дорогой Слой 4 — трудный остаток), отбраковку статистически маловероятных пар «бренд × значение» и архитектурную объяснимость: для каждой ячейки известно, какой слой принял решение.

2.1.3 Поведение системы при выходе категории за пределы поддерживаемых

Слой 0 определяет $c = f_0(x)$ до запуска каскада. Если уверенность ниже порога OOD-детектора, товару присваивается метка OOD и система

возвращает «категория не поддерживается»: $f_0(x) \in \mathcal{C} \setminus \{\text{OOD}\} \Rightarrow$ запуск per-category каскада со схемой $\mathcal{A}_{f_0(x)}$; $f_0(x) = \text{OOD} \Rightarrow \hat{y} = \emptyset$. Явный отказ выбран по соображениям эксплуатационной безопасности: если товар ошибочно отнести к чужой категории, всем атрибутам достанется неверная схема.

2.1.4 Метрики качества и стоимости

Каскад поддерживает явный отказ от ответа; используются две согласованные метрики качества и одна сводная по стоимости.

$$\text{acc}_k^{\text{cov}} = \frac{|\{i : \hat{y}_k^{(i)} = y_k^{(i)} \wedge \hat{y}_k^{(i)} \neq \text{null}\}|}{|\{i : \hat{y}_k^{(i)} \neq \text{null}\}|}. \quad (6)$$

Точность на покрытой части $\text{acc}_k^{\text{cov}}$ — доля верных предсказаний среди ячеек с непустым ответом.

$$\text{cov}_k = \frac{|\{i : \hat{y}_k^{(i)} \neq \text{null}\}|}{N}, \quad (7)$$

Покрытие cov_k — доля ячеек, на которых система решилась дать ответ (где N — мощность проверочной выборки).

$$\text{acc}_k^{\text{e2e}} = \text{acc}_k^{\text{cov}} \cdot \text{cov}_k. \quad (8)$$

Сквозная точность $\text{acc}_k^{\text{e2e}}$ — основная метрика и заголовочное число (3.3.2); ячейки с $\hat{y}_k^{(i)} = \text{null}$ засчитываются как неверные, что не даёт системе зависеть качество за счёт молчания.

$$\text{cost}_{\text{LLM}} = \frac{1000}{N \cdot K_c} \sum_{i=1}^N \sum_{k=1}^{K_c} \mathbb{I}[\hat{y}_k^{(i)} \text{ получен через } f_k^{\text{LLM}}]. \quad (9)$$

Стоимость cost_{LLM} — ожидаемая доля вызовов запасного слоя на 1000 ячеек; метрика инвариантна к выбору модели Слая 4 и характеризует структуру каскада. Совместный анализ $(\text{acc}^{\text{e2e}}, \text{cost}_{\text{LLM}})$ даёт матрицу «точность — стоимость», на которой строится заголовочный результат (3.3.2).

2.2 Архитектура и стек технологий

В разделе описывается архитектурное решение, реализующее формальную постановку 2.1, и обосновывается выбор каждой компоненты стека.

2.2.1 Архитектура системы

Функциональная модель системы в виде диаграммы потоков данных приведена на рисунке 2.1. Внешние акторы (партнёр-продавец и витрина каталога интернет-магазина) обмениваются данными со шлюзом API и ML-сервисом, который содержит каскад из пяти слоёв и обращается к хранилищам моделей XGBoost, байесовских сетей и внешнему API большой языковой модели.

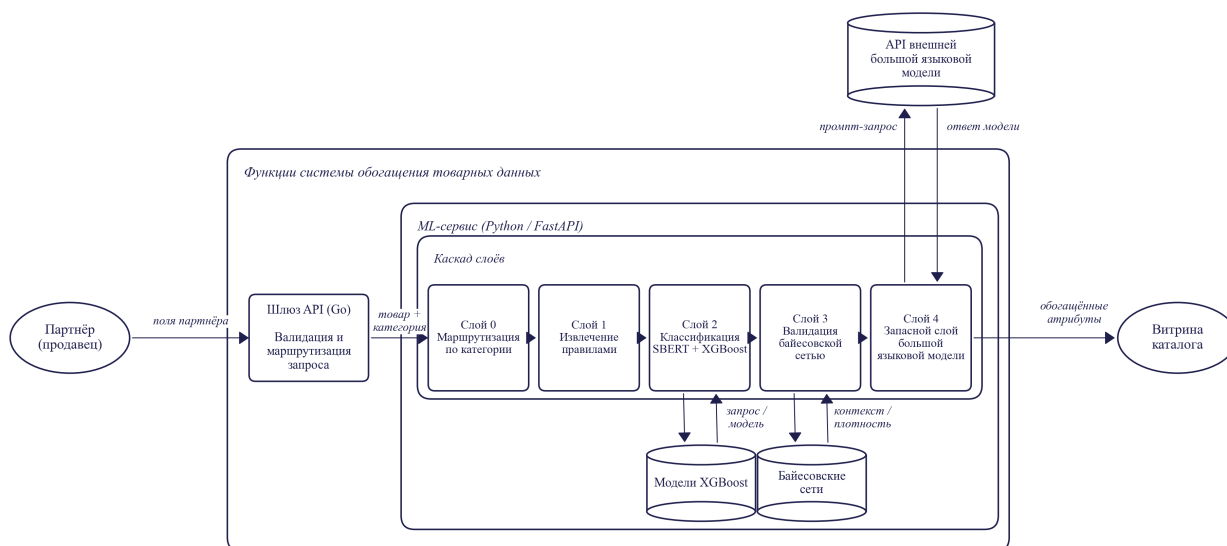


Рисунок 3 – Функциональная модель системы обогащения товарных данных

Система реализована как пятислойный конвейер (Слой 0 + четыре содержательных слоя). Состав и роль каждого слоя:

- **Слой 0 — маршрутизатор категорий.** LightGBM поверх TF-IDF-представления (term frequency $\times$ inverse document frequency, частотно-инверсный показатель) названия и бренда с порогом OOD; определяет $c \in \mathcal{C}$ либо относит к OOD.
- **Слой 1 — экстрактор регулярных выражений.** Извлекает значения, явно присутствующие в тексте (форма пасты, % какао,

цельнозерновой, шаблоны количества). Применяется только к шаблонам с высокой точностью совпадения.

- **Слой 2 — классификатор машинного обучения.** Основной слой: партнёр-доступные поля конкатенируются и кодируются SBERT в 768-мерный вектор (модель `paraphrase-multilingual-mpnet-base-v2`, архитектура MPNet); поатрибутный XGBoost с порогом τ_k на отложенной выборке. Изотоническая пост-калибровка исследована в двух режимах (3.3.3.3): серебряная деградирует на эталоне из-за дрейфа; перекрёстная на эталонной выборке даёт улучшение ECE с 0,070 до 0,043 (в производственную конфигурацию не включена).
- **Слой 3 — байесовский валидатор.** Сеть (структура — Hill Climb + BIC) вычисляет $P_B(\hat{y}_k | \mathbf{e})$; ниже порога θ_k — флаг и передача дальше. Роль пересмотрена (3.1): не классификатор, а валидатор плотностей.
- **Слой 4 — запасной слой на большой языковой модели.** Взаимозаменяемые модели семейств GPT [51] и Claude [52]: Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая `gpt-oss-120b`, малая `Llama-3.2-3b` (сопоставление — 3.3.2). Структурированный запрос со схемой и партнёр-доступными полями; возвращает значение или честный отказ.

Слои упорядочены по росту удельной стоимости решения: регулярные выражения почти бесплатны, ML работает на уже вычисленном векторном представлении, байесовский вывод требует одного прохода по сети, запасной слой — самый дорогой.

Структура обученной байесовской сети для категории `chocolate` приведена на рисунке 4. Узлы графа — целевые атрибуты и бренд; направленные рёбра показывают условные зависимости, найденные алгоритмом Hill Climb на серебряной выборке. Бренд — корневой узел; через производный признак `brand_has_organic_marker` он влияет на `chocolate_type` и `is_organic`. Тип шоколада задаёт процент какао и подтип `chocolate_extra`, от которого расходятся рёбра к `contains_nuts`, `nutri_score_grade`, `protein_class` и повторно к `is_organic` — это согласуется с интуицией о связи органичности, бренда и подтипа. Эмпирическая роль сети как точечного валидатора и количественные характеристики приведены в 3.3.4.

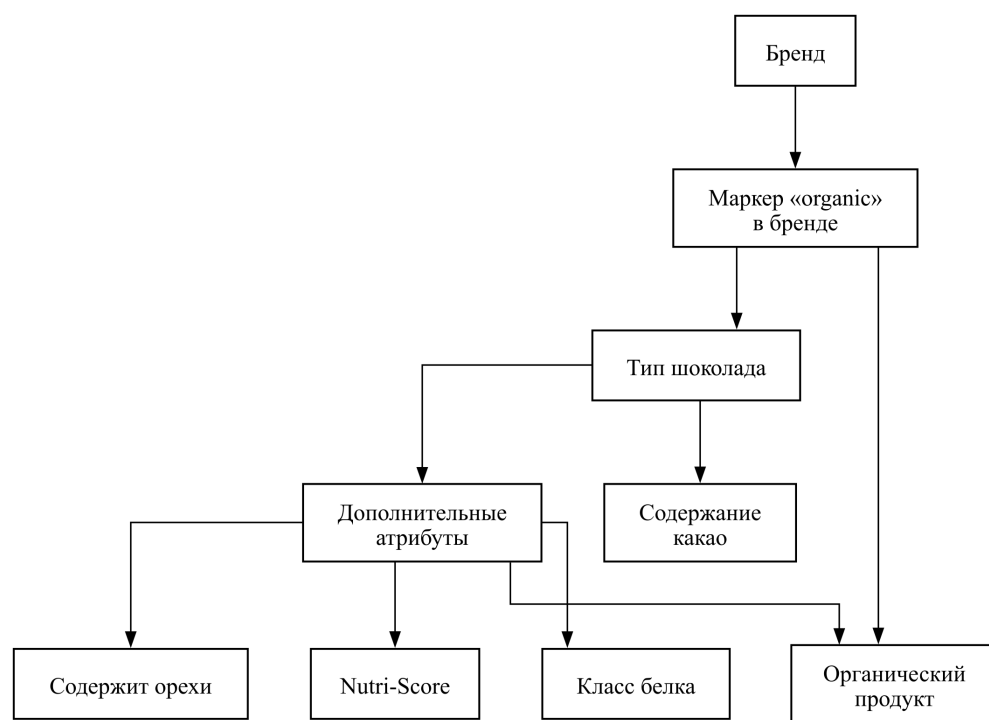


Рисунок 4 – Структура байесовской сети категории *chocolate* (направленный ациклический граф; алгоритм Hill Climb с критерием BIC, библиотека pgmpy 1.1.2)

2.2.2 Обоснование выбора компонентов стека

Каскад вместо сквозного решения. Сквозной all-LLM даёт 83–94 % точности (3.3.2), но стоимость линейно растёт с числом ячеек; каскад при равной или большей точности экономит порядка двух порядков за счёт ступенчатого распределения нагрузки. Сквозной all-ML не подходит — у части атрибутов значения вычисляются из данных за пределами товарного описания, к тому же есть сценарий холодного старта новых категорий.

SBERT *paraphrase-multilingual-mpnet-base-v2* [23] выбран за бесплатную многоязычность (50+ языков в едином пространстве без дообучения), умеренную 768-мерную размерность и готовность к работе без доменной адаптации. Архитектура MPNet объединяет преимущества маскированного и перестановочного языкового моделирования и устойчиво обгоняет дистиллированные MiniLM-варианты на задачах семантической близости.

XGBoost [17] с регуляризацией (*subsample=0.8*, *colsample=0.8*, ранняя остановка, *gamma=0.1*) выбран как Слой 2 за устойчивость на плотных векторных признаках при ограниченной разметке ($n \approx 1250$ на категорию),

калибруемость вероятностей (изотоническая регрессия [36]) и миллисекундный вывод на CPU. Поэтапная оптимизация по остаткам и явные L1/L2/ γ -штрафы дают более тонкие нелинейные границы и более прямой контроль переобучения, чем у Random Forest [53; 54]; на независимых табличных стендах [55; 56] градиентный бустинг устойчиво обходит Random Forest и нейросетевые архитектуры в характерной для Слой 2 размерности. Эмпирическое сравнение (§ 3.3.14) подтверждает выбор: при сопоставимой точности XGBoost даёт наименьшую ECE (0,026 против 0,031 у MLP и 0,035 у LogReg).

Байесовская сеть на pgmpy [45] 1.1.2. По сравнению с MRF и автоэнкодерами дискретная байесовская сеть допускает построение структуры графа (Hill Climb + BIC [46]) с интерпретируемым ориентированным графом, дешёвый вывод через Variable Elimination и явное выражение условных зависимостей; pgmpy поддерживает и построение структуры, и оценку параметров через BayesianEstimator.

Шлюз API на Go при ML-сервисе на Python. Демо-система — два сервиса: шлюз на Go (:8080) и ML-сервис на Python/FastAPI (:8001). Go подходит для сетевого ввода-вывода (малое потребление памяти, быстрый запуск, конкурентный I/O); Python-экосистема (PyTorch, sentence-transformers, XGBoost, pgmpy) безальтернативна для ML-инференса. Разделение соответствует типовой производственной архитектуре и позволяет масштабировать компоненты независимо.

Открытая LLM как запасной слой. Конфигурация многомодельна через OpenRouter (Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая gpt-oss-120b, малая llama-3.2-3b). Открытую gpt-oss-120b можно запустить в инфраструктуре заказчика без передачи данных вовне; коммерческие LLM взаимозаменяемы, что снижает зависимость от провайдера; каскад компенсирует слабость малой модели — при самостоятельном применении gpt-oss-120b проигрывает Sonnet 14–15 п. п., но в составе каскада превосходит её безусловное использование на 21,3 п. п. (3.3.2).

2.2.3 Стек технологий — сводная таблица

Сводное представление принятых решений приведено в таблице 2.1.

Таблица 3 – Сводная таблица стека технологий с обоснованием выбора

Компонент	Технология	Ключевое обоснование
Шлюз API	Go, стандартная библиотека net/http	Малое потребление ресурсов, быстрый запуск, отсутствие внешних зависимостей
ML-сервис	Python 3.14, FastAPI, Uvicorn	Доступ к ML-экосистеме, автоматическая валидация через Pydantic, поддержка OpenAPI
Векторное представление	SBERT paraphrase-multilingual-mpnet-base-v2	Многоязычность (50+ языков), 768-мерный вектор (архитектура MPNet), отсутствие необходимости в дообучении
Классификатор Слой 2	XGBoost с регуляризацией; перекрёстная изотоническая калибровка на эталонной выборке снижает среднюю ECE с 0,070 до 0,043 (3.3.3.3), внедрение в производственную конфигурацию вынесено в раздел «Перспективы развития темы» заключения	Устойчивость на плотных признаках, калибруемые вероятности, дешёвый вывод
Байесовская сеть	pgmpy 1.1.2, Discrete BayesianNetwork	Структурное обучение Hill Climb + BIC, дешёвый вывод (Variable Elimination), интерпретируемый граф

Продолжение таблицы 3

Компонент	Технология	Ключевое обоснование
Маршрутизатор категорий (Слой 0)	LightGBM (поверх TF-IDF-представления) + порог OOD	Многоклассовая классификация с вероятностями + явный отказ при низкой уверенности
Запасной слой	OpenRouter API: Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, gpt-oss-120b, llama-3.2-3b	Единый интерфейс, возможность локального развёртывания, независимость от поставщика
Фронтенд	HTML/CSS/JavaScript без сборочной системы	Минимальные требования к окружению, прозрачность демонстрации

2.3 Сценарий работы программного обеспечения

2.3.1 Сквозной пример прохождения запроса

Рассмотрим прохождение на примере «Spaghetti #5 Barilla» (повторяется в 3.1.2). Партнёр шлёт `POST /api/enrich` с полями `category=null`, `product_name=«Spaghetti #5 Barilla»`, `brands=«Barilla»`, `ingredients_text=«Durum wheat semolina, water»`, `quantity=«500g»`. Поле `category` пустое — категоризация на системе.

Шаг 1. Определение категории (Слой 0). Возвращает `c = pasta` с уверенностью 0,97; OOD не сработал, выбрана схема $\mathcal{A}_{\text{pasta}}$ (8 атрибутов).

Шаг 2. Поочерёдная обработка атрибутов. Каскад обходит атрибуты по схеме. Для `pasta_shape` срабатывает шаблон Слая 1 на «spaghetti» с уверенностью 1,0 — следующие слои не вызываются. Для `grain_type` Слой 1 не даёт совпадения; Слой 2 выдаёт «wheat» с $P_{\text{ML}} = 0,99 \geq \tau = 0,5$, валидатор подтверждает ($P_B = 0,85 \geq \theta = 0,04$) — отметка ml. Для `nutri_score_grade` — класс «A», $P_{\text{ML}} = 0,83$, валидатор подтверждает. Для `is_filled` $P_{\text{ML}} = 0,45 < \tau$ — ячейка идёт в Слой 4.

Шаг 3. Формирование ответа. ML-сервис собирает поатрибутный ответ с агрегированными счётчиками и возвращает JSON шлюзу (формат — 3.1.2). В этом сценарии 7 из 8 атрибутов закрыты дешёвыми слоями (1 regex + 6 ML),

1 — Слой 4. Именно такое распределение и даёт каскаду выигрыш в матрице «точность — стоимость» (3.3.2).

2.4 Выводы по главе 2

- 1) Задача обогащения формализована как поатрибутное предсказание $\hat{y}_k$ с явным отказом ($\hat{y}_k = \text{null}$); каскадная логика — последовательная фильтрация по уверенности с конъюнктивным условием $P_{ML} \geq \tau_k \wedge P_B \geq \theta_k$ для принятия предсказания Слая 2 как ансамблевая оценка уверенности.
- 2) Архитектурный выбор каждой компоненты (каскад вместо сквозного решения; SBERT, XGBoost, pgmpy; Go-шлюз + Python-ML; открытая gpt-oss-120b) обоснован эксплуатационно, методологически и стратегически. Это превращает слабую открытую модель в работоспособную часть производственного стека.
- 3) Сквозной сценарий подтверждает: большинство атрибутов закрывается дешёвыми слоями, обращение к LLM точно — отсюда и доминирование каскада по Парето над одиночным вызовом большой языковой модели (подтверждается в 3.3).

3 ОПИСАНИЕ ПРОГРАММНО-ТЕХНОЛОГИЧЕСКОЙ РЕАЛИЗАЦИИ

Глава описывает программную реализацию системы: 3.1 — каскад и демонстрационный комплекс; 3.2 — данные и предобработку; 3.3 — численные результаты и сценарии проверки.

3.1 Описание программного обеспечения

3.1.1 Программная реализация слоёв каскада

Архитектурная роль слоёв описана в 2.2.1, поток управления и формула вывода $\hat{y}_k$ — в 2.1; ниже приведены только детали реализации. Алгоритм обработки одного запроса в нотации ГОСТ 19.701-90 показан на рисунке 3.1; на каждом слое принимается одно из двух решений (закрыть атрибут или передать дальше), слои упорядочены по росту стоимости. Вход — партнёр-доступные поля; слой 2 обучается только на них, что исключает утечку через теги OFF.

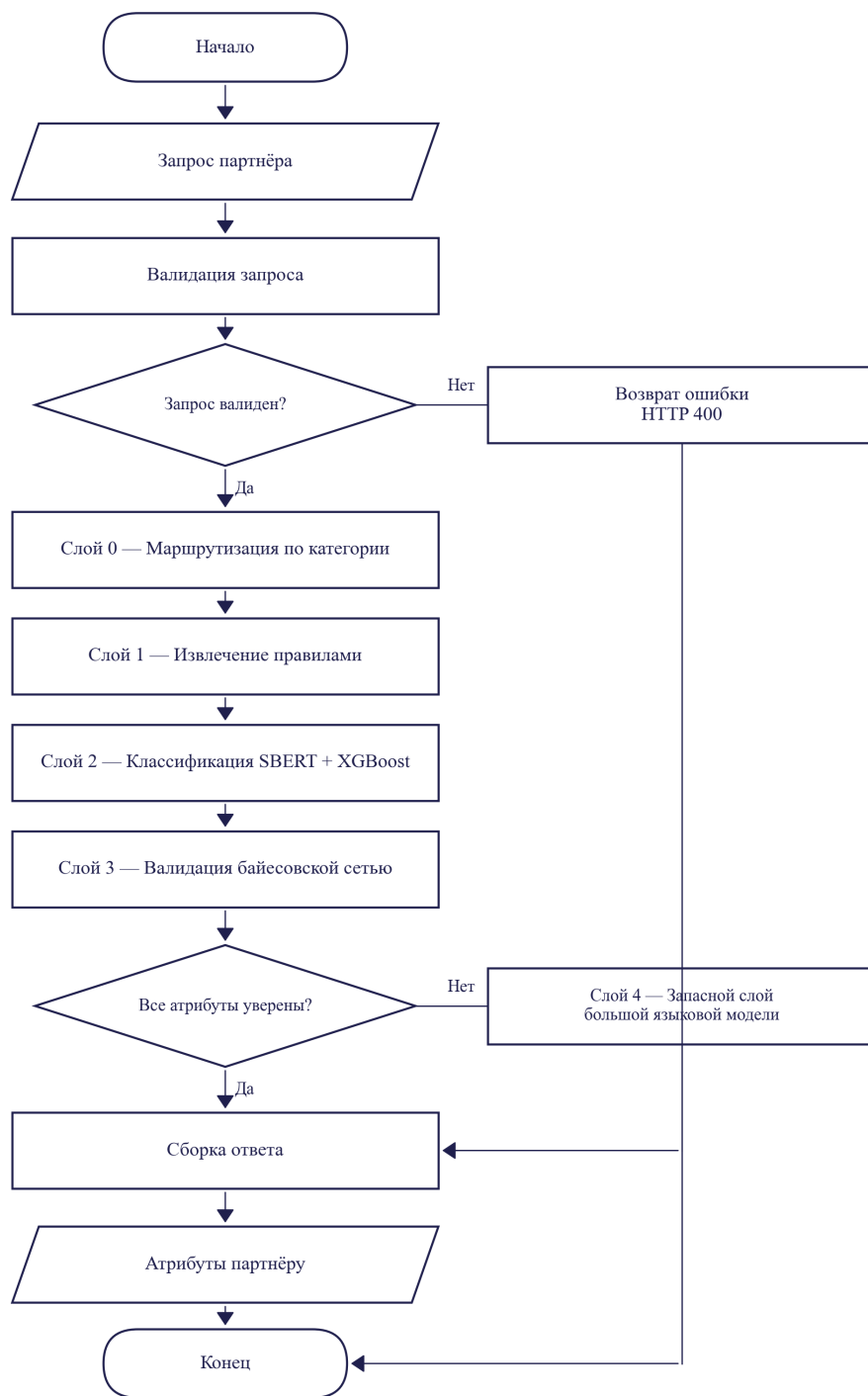


Рисунок 5 – Блок-схема алгоритма обработки запроса
(нотация ГОСТ 19.701-90)

Слой 0 — маршрутизатор категорий. LightGBM поверх TF-IDF-представления (term frequency × inverse document frequency, частотно-инверсный показатель) (n-граммы 1–2) с порогом OOD (out of distribution, выход за пределы обучающего распределения). Артефакты:

models/category_router_v3_{lgbm,vec}.pkl, _threshold.json. При ошибке в категории атрибуты товара получают неверную схему и засчитываются как неверные предсказания.

Слой 1 — экстрактор регулярных выражений. Реализация — src/pipeline/regex/extractor.py. Разбирает product_name и quantity; распознаёт формы пасты, процент какао, маркер цельнозернового состава, источник молока в сырах (chèvre / brebis / bufala), маркеры PDO/AOP/DOP, маркеры ультра-обработанного сыра. Покрытие неравно по категориям: около 18 % на pasta, 23 % на chocolate и 3 % на cheeses; точность на закрытых ячейках — 87,0–100 % (см. 3.3.3.2).

Слой 2 — классификатор машинного обучения. Основной рабочий слой (73,2 % всех решений системы). Вход — конкатенация product_name + brands + ingredients_text + quantity. SBERT-кодировщик paraphrase-multilingual-mpnet-base-v2 (768d, 50+ языков) конкатенируется с TF-IDF (n-граммы 1–2, top-5000), сжатым TruncatedSVD до 128 компонент; итого 896-мерное гибридное пространство. Для каждой пары (категория, атрибут) обучен XGBoost-классификатор {cat}_v4_mpnet_tfidf_noleak_{attr}_xgb.pkl с регуляризацией (subsample=0.8, colsample=0.8, ранняя остановка), Platt-калибровкой и порогом уверенности, подобранным на валидационном срезе по произведению маско-F1 и охвата. Точность на покрытом срезе — 94,9–98,6 % (см. 3.3.10).

Слой 3 — байесовский валидатор. Сеть в models/{cat}_stratified_bayesian.pkl; структура — Hill Climb + BIC. По итогам 3.3.4 используется как валидатор плотностей: для предсказания слоя 2 вычисляется $P(\text{value} \mid \text{evidence})$, при попадании в нижний 5-перцентиль ячейка помечается флагом.

Слой 4 — запасной слой на большой языковой модели. Реализация — src/pipeline/llm_fallback/enrich.py. В 3.3.2 оцениваются пять моделей: Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash, открытая gpt-oss-120b и открытая малая Llama-3.2-3b¹. Промпт содержит схему атрибутов категории, требуемый JSON-формат и партнёр-доступные поля.

¹Состав моделей зафиксирован на момент проведения экспериментов (первое полугодие 2026 года). Архитектура каскада не привязана к конкретному поставщику, и при появлении новых поколений моделей Слой 4 заменяется без переобучения остальных слоёв; переоценка на актуальных моделях вынесена в перспективы развития темы (Заключение).

3.1.2 Демонстрационный комплекс и контракт API

Демонстрация имитирует встраивание в бизнес-процесс интернет-магазина и состоит из трёх компонентов: Go-шлюза API (demo/gateway/, ≈ 200 строк на стандартной net/http — валидация JSON, проксирование, CORS, отдача статики), Python/FastAPI ML-сервиса (demo/ml_service/ — при старте подгружает SBERT-кодировщик, 20 XGBoost-классификаторов, байесовские сети и слой 0; после прогрева $\approx 15\text{--}20$ с слои 0–3 отвечают за десятки миллисекунд), одностраничного HTML/JS-фронтенда. Эндпоинты: POST /api/enrich, GET /api/categories, GET /health, POST /api/explain.

Запрос POST /api/enrich принимает JSON с полями category (опционально), product_name, brands, ingredients_text, quantity; ответ содержит category, агрегированные счётчики и объект predictions с указанием слоя-источника (regex / ml / llm_fallback), уверенности и результата валидации для каждого атрибута. Пример пары запрос–ответ:

```
1 // запрос
2 { "category": "pasta", "product_name": "Spaghetti #5 Barilla",
3   "brands": "Barilla", "ingredients_text": "Durum wheat semolina", "quantity": "500g" }
4 // ответ
5 { "category": "pasta", "n_attrs_total": 8, "n_covered": 7, "n_llm_fallback": 1,
6   "predictions": {
7     "pasta_shape": { "value": "spaghetti", "layer": "regex", "confidence": 1.00 },
8     "grain_type": { "value": "wheat", "layer": "ml", "confidence": 0.99 },
9     "is_filled": { "value": null, "layer": "llm_fallback", "confidence": 0.0 } } }
```

Явная отметка слоя-источника компенсирует ограниченную интерпретируемость слоёв 2/3 на уровне отдельного товара. Запуск — одной командой `bash reproduce.sh demo`; готовность проверяется через GET /health. Стек технологий обоснован в 2.2.8 (таблица 2.1).

3.2 Данные

3.2.1 Источник данных, фильтрация и стратифицированная выборка

Основой выбран открытый каталог Open Food Facts (далее — OFF) [13] — краудсорсинговая база $\approx 4,5$ млн позиций под лицензией ODbL v1.0, допускающей коммерческое использование при условии share-alike. Производные выборки и эталоны размещены в репозитории под совместимой лицензией с указанием авторства. Дамп (526 МБ) получен командой `python -m src.data.download --source off`. Выбор обусловлен открытостью лицензии (воспроизводимость), мультиязычностью (50+ языков) и наличием структурированных нутриентных показателей и регуляторных тегов, на которых построен эталон (3.2.3). Область охвата — три категории (pasta, chocolate, cheeses), отобранные по критериям 1.3.

Поля OFF разделены на партнёр-доступные (product_name, brands, quantity, ingredients_text, code) и целевые выходные (categories_tags, labels_tags, *_100g, ingredients_analysis_tags); целевые поля не передаются на вход обучаемых компонентов под угрозой утечки. Фильтрация по категории (src/data/filter.py) использует списки включения/исключения categories_tags (для pasta включение — en:pastas, исключение — en:potato-products, en:noodles). После фильтрации применяется стратифицированная по языку выборка (langdetect, страты FR/EN/DE/IT/ES) объёмом ≈ 250 товаров на страту, итого ≈ 1250 на категорию (таблица 3.2.1).

Таблица 4 – Состав стратифицированной выборки по категориям, входящим в область охвата работы

Категория	Объём после фильтрации (сырые позиции)	Объём стратифицированной выборки	Число целевых атрибутов
pasta (макаронные изделия)	15 691	1 250	8

Продолжение таблицы 4

Категория	Объём после фильтрации (сырые позиции)	Объём стратифицированной выборки	Число целевых атрибутов
chocolate (шоколад)	13 469	1 250	7
cheeses (сыры)	21 208	1 250	9

Примечание. Источник: `datasets/processed/{cat}_stratified_silver_standard.parquet`.

Целевые атрибуты определены в `src/pipeline/schemas/{cat}.py`. По результатам анализа ошибок (3.3.25) принят принцип независимости атрибутов: структурная характеристика «наличие начинки» выделена в отдельный бинарный атрибут `is_filled`, не зависящий от типа какао-массы и состава включений. Итоговая схема — 20 пар (категория, атрибут): chocolate — 6 (`chocolate_type`, `is_filled`, `chocolate_extra`, `contains_nuts`, `is_organic`, `flavor_profile`), pasta — 8 (`grain_type`, `pasta_shape`, `is_filled`, `is_gluten_free`, `is_organic`, `is_vegan`, `cuisine_origin`, `protein_class`), cheeses — 7 (`milk_source`, `texture`, `country_of_origin`, `aging`, `is_pdo`, `is_organic`, `is_ultra_processed`).

3.2.2 Эталонная разметка: иерархическая схема трёх ярусов

Эталон построен как процедура из трёх ярусов с качественно разным классом свидетельств на каждом ярусе. **Ярус 0 (нутриент-производные)** — детерминированное бакетирование числовых полей: `pasta/protein_class` из `proteins_100g` по регламенту ЕК 1924/2006, точность 100 % при достоверных нутриентах. **Ярус 1 (тег-производные)** — эталон из регуляторных тегов: `is_organic` из `en:organic`, `is_gluten_free` из `en:gluten-free`, `is_pdo` из `en:protected-designation-of-origin` (правовая защита в ЕС, регламенты 834/2007 и 1151/2012). **Ярус 2 (семантически-текстовые)** — признаки без детерминированного правила (`pasta_shape`, `chocolate_type`, `milk_source`, `texture` и т. д.): эталон — согласие трёх независимых LLM (Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash) на полной выборке 1250×3 категории при правиле «не менее двух из трёх» (согласованность 97,7 %);

несогласованные позиции переразмечены слепо Claude Opus 4 на 688 ячейках (ручной эталон) — это нижняя консервативная оценка точности.

3.2.3 Разбиение без пересечения товарных кодов (**code-disjoint**)

Случайное разбиение не подходит: одна и та же товарная позиция в обучении и тесте даёт прямую утечку. Применяемое разбиение — **code-disjoint** по идентификатору товара (`src/eval/build_noleak_artifacts.py`): из расширенного серебра удаляются все коды, попавшие в любой из gold-источников. Основной эталон — LLM-consensus gold объёмом 16 360 ячеек с ответом каскада по 20 атрибутам (`headline_v4_mpnet_tfidf_noleak.parquet`); консервативная нижняя оценка — ручной эталон Claude Opus 4 объёмом 566 ячеек с ответом каскада. Тестовая выборка содержит 186 / 301 / 297 уникальных товарных кодов в pasta / chocolate / cheeses.

Brand overlap. 82,5 %, 84,9 % и 82,7 % брендов в тестовой выборке pasta / chocolate / cheeses встречаются и в обучающем пуле. Полноценное brand-disjoint разбиение не заявляется: концентрация брендов в OFF (Lindt, Barilla, Président встречаются в десятках позиций каждый) делает построение brand-disjoint эталона той же мощности невозможным без переразметки. Brand-disjoint подмножество доступно как точечная проверка устойчивости (3.3.7.3): 17 / 19 / 28 кодов на pasta / chocolate / cheeses, чьи бренды отсутствуют в обучающем пуле.

3.2.4 Дополнительные источники сигналов

Производственная конфигурация слоя 2 опирается строго на партнёр-доступные текстовые поля; режим вывода совпадает с режимом обучения. Дополнительные источники (фотографии этикеток через OCR EasyOCR и CLIP-эмбединги ViT-B/32 [57]; внешний справочник USDA Branded Foods [58]) проверены экспериментально, но в производственную конфигурацию не включены: мультимодальный сигнал не дал прироста выше шума и потребовал бы доступа к OFF image dump в режиме вывода, USDA используется только для сверки эталона яруса 0. Правило «вход = партнёр-доступные поля» соблюдено во всех оценках 3.3.

3.3 Доказательство работоспособности и применимости

3.3.1 Условия проверки и метрики

Эксперименты проведены на двух согласованных эталонных срезах. Основной — LLM-consensus gold (consensus_gold_v4.parquet) объёмом 16 360 ячеек cascade-valid (17 062 в-scope) по 20 атрибутам; служит источником главного показателя. Консервативная нижняя оценка — human gold Claude Opus 4 объёмом 566 ячеек cascade-valid (688 в-scope) по тем же 20 атрибутам. Знаменатель cascade-valid — ячейки, на которых каскад вернул конкретный класс (Слой 1 rule_h $\cup$ Слой 2 ML $\cup$ Слой 3 rule_l); 702 / 122 ячейки запасного слоя LLM не входят в cascade-only знаменатель, так как в производственном режиме на них каскад отказывается и эскалирует в LLM. Использованы модели с суффиксом _mpnet_tfidf_noleak (MPNet + TF-IDF + XGBoost, переобучение без утечки), разбиение train/val/test без пересечения товарных кодов в пропорции 60/20/20. Согласованность трёх LLM-разметчиков (Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash) — 97,7 % при правиле «не менее двух из трёх совпадают», спорные позиции переразмечены Claude Opus 4 слепо; согласованность LLM-consensus и human gold на пересечении — 92 %, κ Коэна $> 0,80$.

Метрики ($\text{acc}_k^{\text{cov}}$, cov_k , $\text{acc}_k^{\text{e2e}}$, стоимость) определены в 2.1.4. Доверительные интервалы — кластерный бутстрэп по брендам (1000 итераций, перцентили 2,5 / 97,5 %, src/diagnostics/ml/bootstrap_ci_brand_clustered.py); такой интервал учитывает внутрибрендовую корреляцию и на ряде атрибутов на 15–30 % шире наивного интервала Вильсона. Для опорной E2E-оценки дополнительно рассчитан кластерный бутстрэп по товарным кодам (B=1000, 2 812 уникальных кодов): интервал [92,3; 93,6] % против Вильсона [92,6; 93,4] % при оценке 93,0 % (расширение 64 %)²; расхождение умеренное, опорное значение 93,0 % устойчиво.

3.3.2 Стратификация точности по типу эталонного сигнала

Эталон разделён на три яруса по природе сигнала: ярус 0

²Воспроизводится src/eval/bootstrap_ci_grouped.py; артефакт — datasets/processed/e2e_bootstrap_ci.json; ячейка bootstrap-grouped-ci в notebooks/03_evaluate.ipynb.

(нутриент-производный, детерминированное бакетирование) — 1 атрибут (pasta/protein_class, n=24), точность 92,3 %; ярус 1 (тег-производный, регуляторные теги OFF) — 6 атрибутов на 1054 ячейках, точность 96,8 % [95,6; 97,8]; ярус 2 (текстово-производный, consensus LLM с проверкой Orpus 4) — 13 атрибутов на 2179 ячейках, точность 93,6 % [92,5; 94,6]. Источник — datasets/processed/tier_breakdown.parquet; иллюстрация на рис. 6. Если бы точность завышалась за счёт корреляции признаков модели с источником эталона, наибольшие значения показывал бы самый уязвимый ярус 2. На практике картина обратная (ярус 1 > ярус 2 > ярус 0), и это опровергает гипотезу о завышении метрик за счёт связи признаков с эталоном.

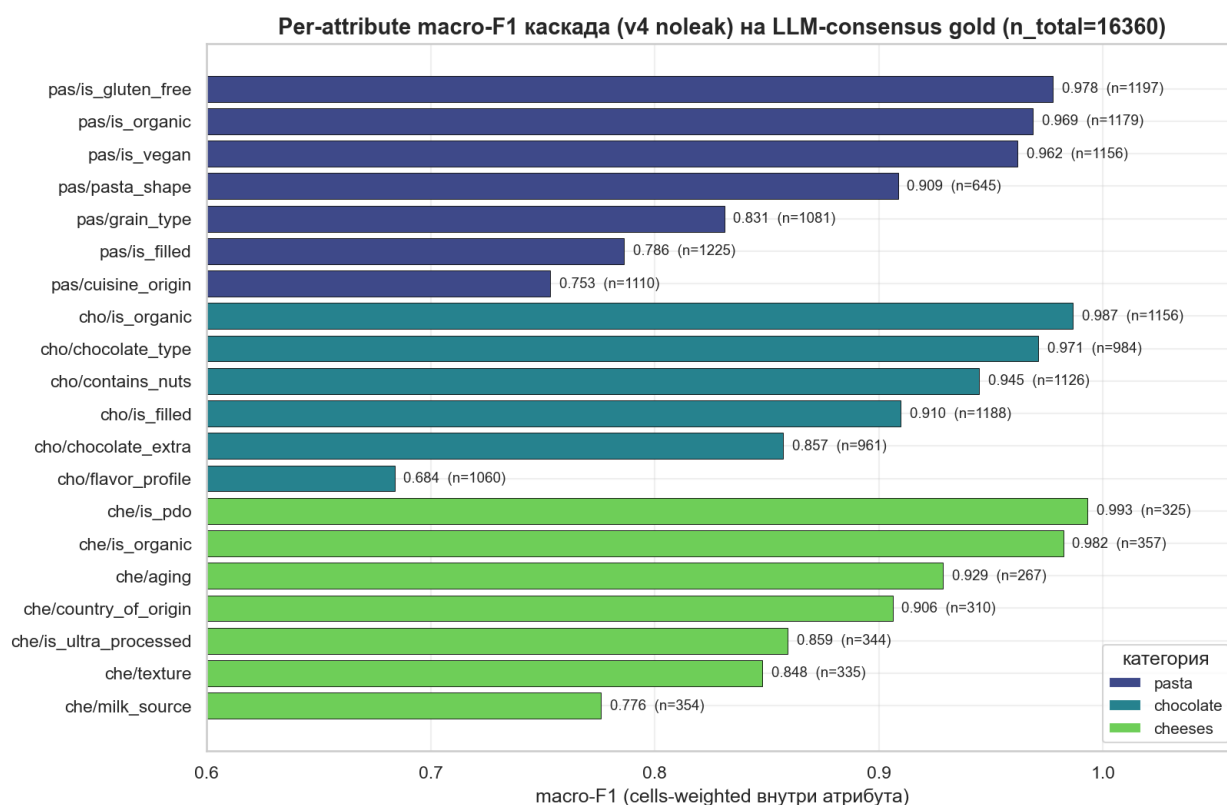


Рисунок 6 – Точность каскада в разрезе ярусов эталонной разметки

3.3.3 Воспроизводимость

Численные результаты 3.3 пересоздаются из артефактов через reproduce.sh (≈час на ноутбуке): перезапуск notebooks/03_evaluate.ipynb и notebooks/05_method_comparison.ipynb над v4_e2e_router_eval.json, v4_metric_table_v2.json,

`cascade_preds_{cat}_gold.parquet`,
`method_comparison_results.parquet` пересоздаёт TeX-фрагменты и PNG локально. Артефакты VM-уровня (исходные прогоны LLM-разметчиков и переобучение Слой 2 без утечки) обновляются отдельно, их пересборка вынесена в перспективы развития; качественные выводы 3.3 от этого не зависят. Числа 3.3.2 получены post-hoc после устранения утечки бренда; единственная заранее объявленная гипотеза — H1 для обучаемого маршрутизатора (3.3.5).

3.3.4 Главный результат: точность и стоимость производственной конфигурации

Каскад сопоставлен с пятью одиночными вызовами LLM на едином тестовом срезе без пересечения товарных кодов: 16 360 ячеек с ответом каскада (LLM-консенсус) и 566 ячеек (ручной эталон) по 20 атрибутам (для `pasta/protein_class` срез сокращён до 24 ячеек). Заголовочные метрики получены на гибридном 896-мерном представлении ($\text{MPNet 768d} \oplus \text{TF-IDF SVD 128d}$); в роли Слой 4 и опорной «одна LLM на всё» — пять моделей: Claude Sonnet 4.5, GPT-4o, Gemini 2.5 Flash (закрытые), `gpt-oss-120b` и `llama-3.2-3b` (открытые). Источники:

`headline_v4_mpnet_tfidf_noleak.parquet`,
`grand_acc_summary_v4.parquet`. Слои 1–3 закрывают 95,9 % ячеек без обращения к LLM; оставшиеся 4,1 % — запасной слой. Стоимость нормирована к «все ячейки — Sonnet 4.5», соотношение вход/выход $\approx 500 / 200$ токенов.

Разграничение понятий. покрытие каскада = $1 - \text{fallback rate} = 95,9\%$ (Слои 1–3 выдали уверенный ответ); охват производственной цепочки (E2E) = 97,3 % (с учётом ошибок маршрутизатора Слой 0 и удачных ответов Слой 4 при том же знаменателе); архитектурное снижение стоимости = 95,9 % ($\approx 24\times$) — это то же число, что и покрытие каскада, по построению ($1 - \text{fallback rate}$). Связь: $\text{E2E coverage} = (1 - \text{router loss}) \times (\text{cascade-only coverage} + \text{LLM success rate} \times \text{fallback rate})$.

Таблица 5 – Сравнение каскадных конфигураций и одиночных вызовов большой языковой модели по сквозной точности и относительной стоимости (LLM-consensus gold, n=16 360, без пересечения товарных кодов; стоимость нормирована к стратегии «все ячейки обрабатываются Claude Sonnet 4.5»)

Стратегия	Сквозная точность	Стоимость к опорной	Снижение стоимости
all-sonnet (Claude Sonnet 4.5) — опорная конфигурация	83,8 %	1,000	1×
all-gpt-4o	84,1 %	0,727	1,4×
all-gemini-flash (Gemini 2.5 Flash)	82,9 %	0,042	24×
all-gpt-oss-120b (открытая модель)	69,3 %	0,030	33×
all-llama-3b (открытая малая модель)	56,2 %	0,009	110×
каскад + sonnet (Слой 4)	93,0 %	0,041	24×
каскад + gpt-4o	93,1 %	0,030	33×
каскад + gemini-flash	93,0 %	0,0017	580×
каскад + gpt-oss-120b	90,6 %	0,0012	810×
каскад + llama-3b	90,1 %	0,00037	2700×
только каскад (без запасного слоя LLM, верхняя только-каскадная оценка)	95,5 %	0,000	—

Источники тарифов: Sonnet 4.5 — 3 / 15 долларов за миллион токенов на

вход / выход (нормализованная единица), GPT-4o — 2,5 / 10, Gemini 2.5 Flash — 0,075 / 0,30, gpt-oss-120b через OpenRouter — около 0,10 долларов за миллион токенов на запрос смешанной структуры, llama-3.2-3b через OpenRouter — около 0,03 долларов за миллион.

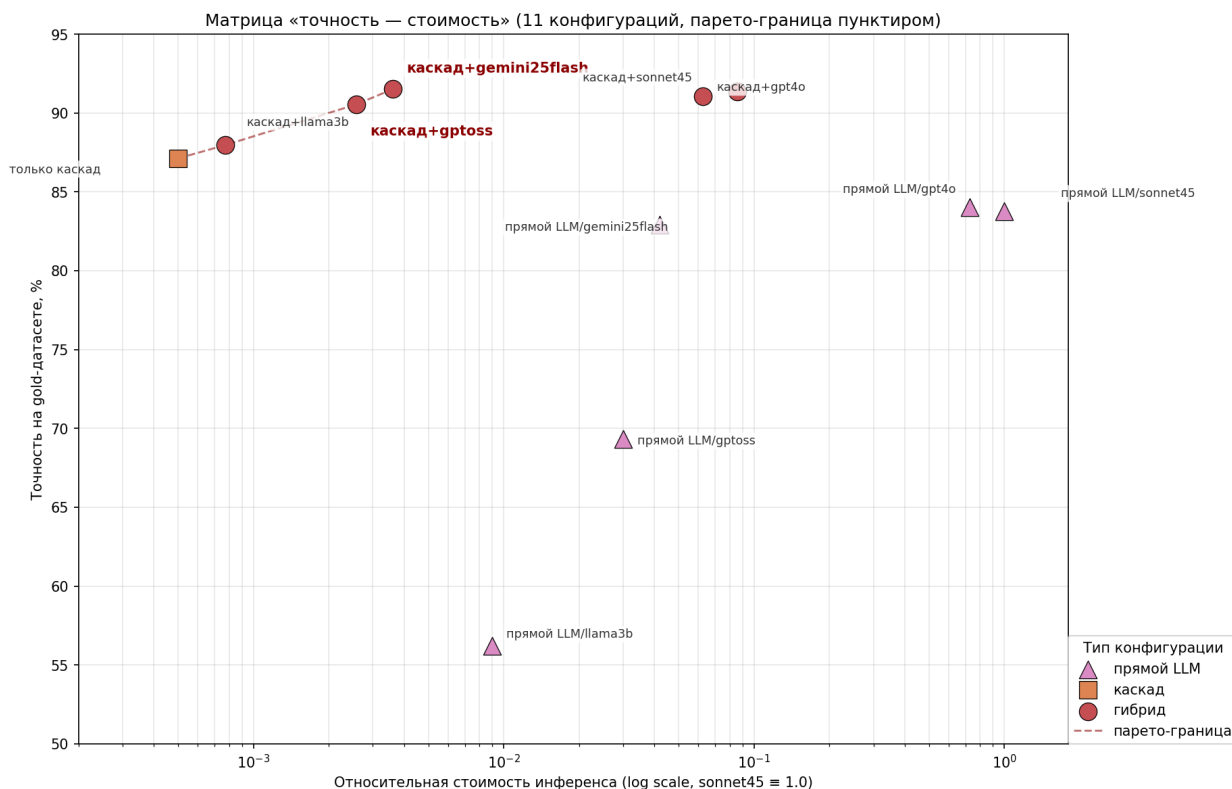


Рисунок 7 – Матрица «точность — стоимость» каскадных конфигураций и одиночных вызовов больших языковых моделей

Из матрицы (рис. 3.3) следует двухчастный главный результат. **(а) Независимость от поставщика модели.** Каскад + Gemini 2.5 Flash даёт E2E 93,0 % при стоимости в 580 раз ниже all-sonnet (83,8 %) — строгое доминирование по Парето; верхняя только-каскадная оценка — 95,5 %. Результат сохраняется и для каскад + sonnet (93,0 %), и для каскад + gpt-4o (93,1 %). **(б) Локальное развёртывание с открытой моделью.** Каскад + gpt-oss-120b даёт 90,6 % — на +21,3 п. п. выше одиночного вызова (69,3 %) при ≈ 24 -кратном сокращении доли LLM; Слой 2 компенсирует слабость малой модели на семантических атрибутах.

Консервативная оценка по ручному эталону (n = 566 ячеек с ответом каскада). На независимом эталоне Claude Opus 4 каскад + gemini-flash даёт E2E 86,7 % при только-каскадной точности 91,3 %; это

соответствует операционному циркулярному сдвигу 4,2 п. п. (каскад) / 6,3 п. п. (E2E)³. Поатрибутное распределение сдвига приведено в табл. 6 (16 атрибутов с обеими оценками); точки концентрации (cheeses/texture +0,249, cheeses/country_of_origin +0,335, pasta/grain_type +0,360) приходятся на сложные по смыслу атрибуты при среднем n_H 7–47, отрицательные значения (cheeses/milk_source –0,224, pasta/is_filled –0,080) объясняются шумом выборки при малом n_H .

Таблица 6 – Поатрибутное сравнение macro-F1 на LLM-consensus и ручной разметке (Opus). Δ — оценка циркулярного сдвига на атрибуте; отрицательные Δ указывают на sample noise малого human-n.

Атрибут	n_C	F1 _C	n_H	F1 _H	$\Delta F1$
pasta.grain_type	1081	0,831	9	0,471	+0,360
cheeses.country_of_origin	310	0,906	35	0,571	+0,335
cheeses.texture	335	0,848	47	0,599	+0,249
cheeses.milk_source	354	0,776	46	1,000	–0,224
chocolate.is_organic	1156	0,987	46	0,832	+0,155
cheeses.is_pdo	325	0,993	42	0,862	+0,131
pasta.pasta_shape	645	0,909	7	0,778	+0,131
chocolate.chocolate_type	984	0,971	43	0,846	+0,125
cheeses.is_ultra_processed	344	0,859	45	0,955	–0,096
pasta.is_filled	1225	0,786	10	0,867	–0,080
chocolate.chocolate_extra	961	0,857	43	0,906	–0,049
pasta.is_vegan	1156	0,962	8	1,000	–0,038
pasta.is_organic	1179	0,969	9	1,000	–0,031
pasta.is_gluten_free	1197	0,978	8	1,000	–0,022
cheeses.is_organic	357	0,982	47	1,000	–0,018
chocolate.contains_nuts	1126	0,945	49	0,939	+0,006

3.3.5 Декомпозиция по категориям

Покрытие каскада, точность только-каскадной и гибридной конфигураций и разность Δ — вклад запасного слоя — для модели

³Воспроизводится в notebooks/03_evaluate.ipynb, ячейка 495abc8a; источник — datasets/processed/v4_e2e_router_eval.json, ключ HUMAN.

gemini-flash в табл. 7. Источники: headline_v4_mpnet_tfidf_noleak.parquet + cascade_plus_llm4_v4.parquet; режим идеальной маршрутизации.

Таблица 7 – Покрытие и точность каскадной и гибридной конфигураций по категориям (модель gemini-flash, идеальная маршрутизация Слойа 0)

Категория	Покрытие каскада	Только каскад	каскад + gemini-flash	Δ от запасного слоя
pasta	89,1 %	94,7 %	90,5 %	+1,4 п. п.
chocolate	98,0 %	94,4 %	91,8 %	+0,8 п. п.
cheeses	93,1 %	95,1 %	90,9 %	+1,7 п. п.

Взвешенное по числу тестовых ячеек среднее по строке «каскад + gemini-flash» в режиме сквозной точности составляет 93,0 %. Производственное значение с учётом ошибок слоя 0 (97,2 %, 3.3.9) уже включено в эту цифру.

Распределение вклада запасного слоя следует из архитектуры. У pasta наименьшее покрытие (89,1 %) — там много сложных текстовых атрибутов pasta/grain_type и pasta/pasta_shape, которые уходят на LLM. Chocolate покрывается каскадом почти полностью (98,0 %) за счёт сочетания широкого Слойа 1 и стабильного ML; cheeses (93,1 %) — промежуточный случай, где Слой 1 узкий, но ML работает уверенно. То есть слои дополняют друг друга.

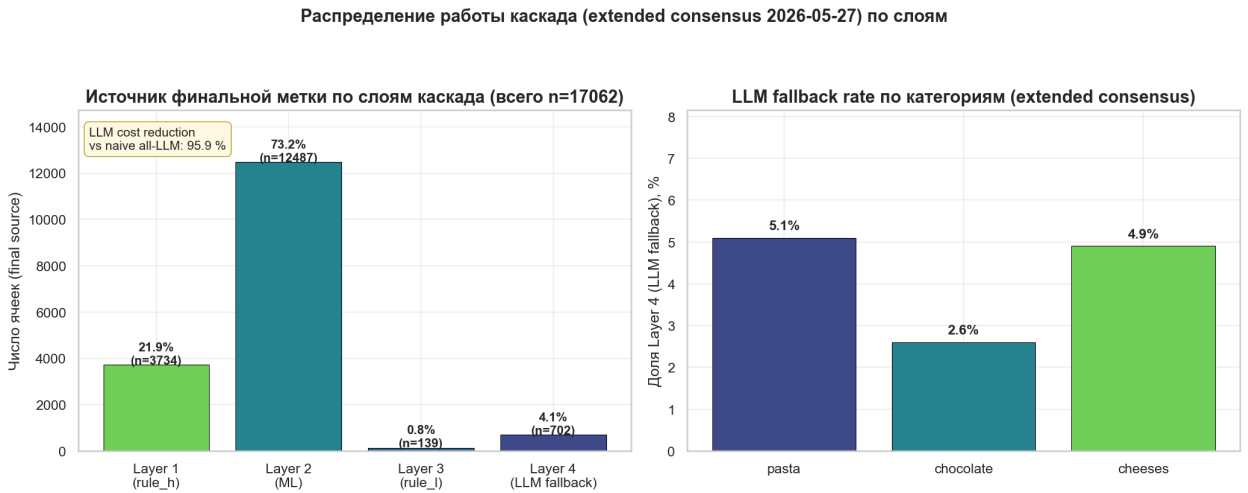


Рисунок 8 – Поатрибутный вклад слоёв каскада в сквозную точность

3.3.6 Обобщение на новые бренды (brand-disjoint side-study)

В §3.2.4 зафиксировано, что разбиение train/test в основной оценке выполнено по идентификатору товара (code-disjoint), а бренды пересекаются между обучающим пулом и тестовой выборкой на 82–85 % по категориям. Чтобы выяснить, насколько результат каскада зависит от знакомых брендов, проведено отдельное контрольное исследование с обучением и оценкой на brand-disjoint срезе. Цель — получить независимую нижнюю оценку обобщающей способности слоя машинного обучения, когда появляются товары неизвестных брендов.

Методология. Источник плотно размеченных данных — `{cat}_stratified_silver_extended.parquet` (≈ 1250 кодов на категорию). Для каждой из трёх категорий (паста, шоколад, сыры) построены два разбиения с одинаковой долей тестовой части (≈ 20 %) и одним случайным зерном (`seed=42`): (a) обычное случайное по кодам и (b) brand-disjoint с замыканием по бренд-токенам — любой код, содержащий хотя бы один бренд из тестовой выборки, переносится в тест целиком, пока множества не станут полностью непересекающимися. Brand overlap по итогу: для code-disjoint — 56–68 % брендов теста встречаются и в обучении (как и в основном эксперименте); для brand-disjoint — ровно 0 %. На каждом разбиении переобучены классификаторы XGBoost второго слоя (мультязычные векторные представления MiniLM-L12 + sigmoid-калибровка, та же гиперпараметризация, что и в основной конфигурации). Воспроизводится: `src/eval/build_brand_disjoint_split.py` строит разбиения, `src/eval/train_brand_disjoint.py` запускается на ВМ (правило проекта запрещает обучение локально), `src/eval/eval_brand_disjoint.py` проводит сравнение. Тикет — `docs/po/tickets/2026-05-28-brand-disjoint-side-study.md`, артефакт — `datasets/processed/v4_brand_disjoint_eval.json`.

Результат. На общем срезе из 4538 ячеек code-disjoint и 5436 ячеек brand-disjoint точность только-ML слоя составила 83,03 % и 82,56 % соответственно; разность — 0,47 п. п. при разнице макро-F1 — 0,017. Таблица 8 приводит покатегорийную и глобальную разбивку.

Таблица 8 – Сравнение точности слоя машинного обучения на code-disjoint и brand-disjoint выборках (силовое исследование, тикет 2026-05-28)

Категория	Атрибутов	Точность, %			Макро-F1		
		code-d.	brand-d.	Δ , п.п.	code-d.	brand-d.	Δ
паста	7	85,54	85,99	+0,45	0,710	0,717	+0,007
шоколад	7	80,80	79,53	−1,27	0,551	0,540	−0,011
сыры	7	82,54	81,87	−0,67	0,734	0,678	−0,056
Глобально	21	83,03	82,56	−0,47	0,667	0,649	−0,017

Интерпретация. Деградация меньше 1 п.п. укладывается в пределы дисперсии при выборках $\sim 1\,000$ ячеек на категорию. Это значит, что слой машинного обучения каскада опирается на структуру наименования и нутри-сигналы товара, а не на запоминание брендов. На двух из трёх категорий (шоколад $-1,27$ п.п., сыры $-0,67$ п.п.) виден незначительный спад, на пасте — даже небольшой прирост $+0,45$ п.п. (то, что знак прыгает, говорит, что эффект в диапазоне случайной флуктуации). То есть заголовочные показатели cascade-only 95,5 % и E2E 93,0 % переносятся на товары неизвестных брендов с потерей не больше $\approx 0,5$ п.п., и разбиение по товарным кодам в основной оценке не создаёт оптимистичного смещения за счёт утечки бренда.

Контрольное исследование ограничено: тестовый срез контрольной оценки в 2–3 раза меньше основной (5,4 тыс. ячеек против 16,4 тыс.), считаются метрики только слоя 2 (без вклада регулярных правил и Bayes-валидатора), сравнение проводится на плотно размеченной подвыборке silver_extended, а не на полном LLM-consensus gold. Полноценная оценка end-to-end в brand-disjoint режиме потребовала бы переразметки эталона той же мощности, что выходит за рамки текущей работы.

3.3.7 Поатрибутная картина точности и доверительные интервалы

Для каждого из 20 атрибутов вычислена точность только-каскадной конфигурации с интервалом Вильсона (полная таблица — v4_metric_table_v2.json, LLM-consensus, $n=16\,360$ cascade-valid). Три

лучших по micro-accuracy: cheeses/is_pdo 99,7 % (n=325), cheeses/is_organic 99,4 % (n=357), pasta/is_gluten_free 99,2 % (n=1197). Три худших: chocolate/flavor_profile 87,5 % (n=1060), pasta/is_filled 88,5 % (n=1225), cheeses/texture 89,0 % (n=335). Поатрибутная таблица (категория, атрибут, n, micro, macro-F1) приведена в табл. 9.

Таблица 9 – Поатрибутная точность каскада на LLM-consensus gold (20 атрибутов, extended consensus rerun)

Категория	Атрибут	n	micro, %	macro-F1
pasta	grain_type	1081	98,8	0,831
pasta	pasta_shape	645	94,9	0,909
pasta	is_filled	1225	88,5	0,786
pasta	is_gluten_free	1197	99,2	0,978
pasta	is_organic	1179	98,1	0,969
pasta	is_vegan	1156	96,3	0,962
pasta	cuisine_origin	1110	95,1	0,753
chocolate	chocolate_type	984	98,3	0,971
chocolate	is_filled	1188	96,9	0,910
chocolate	chocolate_extra	961	92,5	0,857
chocolate	contains_nuts	1126	94,9	0,945
chocolate	is_organic	1156	99,0	0,987
chocolate	flavor_profile	1060	87,5	0,684
cheeses	milk_source	354	96,9	0,776
cheeses	texture	335	89,0	0,848
cheeses	country_of_origin	310	94,8	0,906
cheeses	aging	267	94,4	0,929
cheeses	is_pdo	325	99,7	0,993
cheeses	is_organic	357	99,4	0,982
cheeses	is_ultra_processed	344	95,9	0,859

В 3.3.25 описаны поатрибутные архитектурные решения (сужение слоя 1, понижение порогов слоя 2, гибридные признаки). Запасной слой остаётся доступным на ≈ 4 % сложных ячеек, где Слой 1/2 не отвечают уверенно; наибольшая доля LLM приходится на pasta/pasta_shape (19,5 %) и cheeses/is_pdo (9,7 %) — расчёт по

`cascade_preds_{pasta,chocolate,cheeses}_gold.parquet`,
`extended_consensus_rerun`.

3.3.8 Анализ доминирования по Парето: почему каскад превосходит одиночный вызов LLM

Выигрыш каскада объясняется структурно. LLM системно проигрывает на длиннохвостых семантических атрибутах с мелкой грануляцией (`pasta/grain_type`, `pasta/pasta_shape`, `cheeses/aging`): на отказных ячейках точность Gemini 2.5 Flash — 78,8 %, Claude Sonnet 4.5 — 76,7 %, gpt-oss-120b — 67,2 % (`cascade_plus_llm4_v4.parquet`). XGBoost слоя 2 на MPNet-эмбедингах + TF-IDF опирается на устойчивые корреляции «бренд — структура наименования». Точность каскада растёт за счёт перенаправления таких атрибутов на ML-слой; стоимость падает за счёт сокращения вызовов LLM до 4,1 % ячеек (снижение 95,9 % по сравнению с «одна LLM на всё»).

3.3.9 Учёт точности маршрутизатора первого уровня

Значения рисунка 3.4 — в режиме идеальной маршрутизации категорий. В производстве категорию определяет слой 0 (3.1.1); его точность на LLM-consensus gold — 97,2 % (per-код, n=570) и 95,3 % на human gold (n=107) (`router_eval_v4.parquet`). Все приведённые в 3.3.2 числа сквозной точности уже учитывают ошибки слоя 0; вывод о выигрыше каскада сохраняется на обеих оценках эталона.

3.3.10 Вклад каждого слоя каскада

Декомпозиция итоговой точности по слоям: охват, точность на собственном срезе закрытий и средняя калибровочная ошибка ECE для Слой 2.

Артефакты
<code>cascade_preds_{pasta,chocolate,cheeses}_v4_noleak.parquet</code> , <code>headline_v4_mpnet_tfidf_noleak.parquet</code> , <code>ece_calibration_v4.parquet</code> , <code>consensus_gold_v4.parquet</code> .

Глобальное распределение закрытий (17062 ячейки): Слой 1 (`rule_h`) — 21,9 % (3734 ячеек), Слой 2 (ML: MPNet + TF-IDF SVD + XGBoost) — 73,2 % (12487), Слой 3 (`rule_l`) — 0,8 % (139), Слой 4 (запасной слой LLM) — 4,1 % (702). Покатегорийная разбивка (табл. 10) показывает неоднородность:

chocolate располагает наиболее полным набором правил (34,2 % Слой 1), pasta — промежуточный (15,3 %), cheeses — узкий (milk_source / is_pdo / is_ultra_processed, 8,1 %). Доля LLM сопоставима (pasta 5,1 %, chocolate 2,6 %, cheeses 4,9 %) и концентрируется на длиннохвостых семантических атрибутах: pasta/pasta_shape (19,5 % L4), cheeses/is_pdo (9,7 %), cheeses/country_of_origin (9,4 %), pasta/grain_type (9,1 %).

Таблица 10 – Распределение закрытий каскада по категориям и слоям (LLM-consensus gold, in_scope, extended consensus rerun)

Категория	Слой 1 (regex)	Слой 2 (ML)	Слой 3 (rule_1)	Слой 4 (LLM)	Всего ячеек
pasta	15,3 %	79,6 %	0,0 %	5,1 %	8004
chocolate	34,2 %	60,5 %	2,6 %	2,6 %	6759
cheeses	8,1 %	87,0 %	0,0 %	4,9 %	2411

Таблица 11 – Поатрибутная точность Слая 1 (regex) и Слая 2 (ML) на собственных срезах закрытий, по трём категориям пищевой продукции

Категория	Слой 1: точность (n)	Слой 2: точность (n)
pasta	90,5 % (327)	98,6 % (1035)
chocolate	87,0 % (370)	94,9 % (802)
cheeses	100,0 % (47)	93,5 % (1549)

Точность Слая 1 — 87,0–100 % (ниже 100 % на pasta/chocolate из-за намеренно широких шаблонов); Слой 2 — 93,5–98,6 %, стабильно выше любого одиночного вызова LLM на той же категории, поскольку слой работает только на ячейках, прошедших калиброванный порог. Точность запасного слоя на отказных ячейках (acc_hybrid_proxu в cascade_plus_llm4_v4.parquet): Gemini 2.5 Flash — 78,8 %, Claude Sonnet 4.5 — 76,7 %, gpt-oss-120b — 67,2 %; ниже слоёв 1–2, потому что отказные ячейки сложнее.

3.3.11 Поатрибутная калибровочная ошибка до и после калибровки

Поатрибутная ожидаемая калибровочная ошибка (ECE) рассчитана по 10 равновеликим бинам (ece_calibration_v4.parquet). Среднее ECE

сырого XGBoost — 0,070 (по 20 замерам); у большинства атрибутов не превосходит 0,08. Повышенная ошибка: chocolate/chocolate_type (0,14), chocolate/contains_nuts (0,11), pasta/is_filled (0,09).

Изотоническая пост-калибровка на серебряной выборке локально улучшает проблемные атрибуты (chocolate/chocolate_type −7,8 п. п.), но в среднем по 20 атрибутам ECE ухудшается на +0,012 за счёт деградации на других. Эффект объясняется дрейфом разметки между серебряной (теги OFF) и эталонной (консенсус LLM) выборками.

Для устранения дрейфа проведена изотоническая калибровка на распределении эталонной выборки по схеме 5-кратной перекрёстной валидации (ece_kfold_gold_v4.parquet, src/diagnostics/ml/ece_kfold_gold.py). Среднее ECE падает с 0,070 до 0,043 (−2,8 п. п. в среднем), эффект положительный на 16 из 20 атрибутов. Самые сильные улучшения — на парах с высокой исходной ошибкой: chocolate/chocolate_type (0,139 → 0,052, −8,7 п. п.), chocolate/contains_nuts (0,111 → 0,041, −7,0 п. п.), pasta/is_organic (0,084 → 0,016, −6,9 п. п.). Ухудшение — у четырёх атрибутов с изначально низкой ECE: cheeses/country_of_origin (0,015 → 0,064), cheeses/is_ultra_processed (0,025 → 0,056), pasta/pasta_shape (0,046 → 0,060), chocolate/is_filled (0,055 → 0,058); тут главную роль играют выборочные колебания. В производственную конфигурацию калибровка пока не включена (см. раздел «Перспективы развития темы» заключения).

Итог: Слой 1 закрывает 2,8–22,7 % ячеек (точность 87,0–100 % в зависимости от категории и шаблона); Слой 2 — 71,1–90,3 % (93,5–98,6 %); Слой 4 — 2,0–10,9 % (67,2–78,8 % в зависимости от модели) на старой выборке; на расширенной выборке доля Слоя 4 снижена до 4,1 %. Среднее ECE по слою 2 — 0,070; серебряная пост-калибровка устойчивого эффекта не даёт.

3.3.12 Таксономия ошибок каскада

Раздел отвечает на вопрос «где и почему ошибается каскад» — честная диагностика, важная для защиты: она отделяет собственные дефекты производственной цепочки от шума эталонной разметки. Анализ проведён на 846 несовпадающих ячейках (cascade_pred ≠ gold) из 16 360 cascade-valid LLM-consensus gold по трём категориям (pasta, chocolate, cheeses); это

операционная частота ошибок 5,2 %, соответствующая cascade-only точности 94,8 % на этом пуле⁴. Ячейки, на которых каскад отказался от ответа (запасной слой LLM, 4,1 %), не входят в знаменатель и анализируются отдельно в §3.3.5.

Каждая ошибка отнесена к одному из шести изначально допустимых классов с приоритизированной классификацией (первый совпавший выигрывает): (1) ложное срабатывание Слая 1 — низкоуровневый или высокоуровневый regex-шаблон сработал на синтаксически подходящем, но семантически неверном тексте (например, *Three Cheese Ravioli* закрыт правилом `is_filled=false` по антипаттерну); (2) промах запасного слоя — запасной слой на большой языковой модели (Слой 4) вернул значение, отличное от эталона (по определению cascade-only знаменателя пустой класс, поскольку `cascade_pred` у ячеек запасного слоя равен `None`); (3) гиперпредсказание на null-эталоне — `gold` отсутствует, `prediction` вернул конкретный класс (по построению пуст: фильтр `in_scope=True` требует ненулевого `gold`); (4) шум разметки (*silver-noise*) — ячейки, где консенсус трёх LLM-разметчиков не достигает порога $\geq 2/3$ или отмечены как *disputed* в `manual_gold_consensus.parquet`; (5) спутывание близких классов — пара (`gold`, `prediction`) принадлежит заранее определённом *per-attribute mapping* «семантически смежных» значений (*semi_soft* ↔ *soft*, *dark* ↔ *milk*, *with_nuts* ↔ *with_fruit*, *italy* ↔ *france* и т. п.); (6) ML-смещение в дальний класс — всё прочее, когда XGBoost выбрал семантически удалённый класс.

Применение классификатора оставило четыре непустых класса (таблица 3.3.12). Классы 2 и 3 пусты по построению знаменателя: cascade-only ошибки определены на ячейках, где `cascade_pred` ненулевой и `in_scope=True`; это методологическое уточнение, а не отсутствие явления — промахи запасного слоя и null-эталон анализируются отдельно в §3.3.5 и §3.3.4 соответственно.

⁴Здесь и далее в §3.3 «Таксономия ошибок каскада» эталон — моментальный снимок `cascade_errors_taxonomy_v4.parquet`, на котором cascade-only точность составила 94,8 % (846 ошибок / 16360 ячеек). Заголовочное число ВКР по этой же выборке после финализации расширенного перезапуска консенсуса (`v4_e2e_router_eval.json`, 2026-05-27) составляет 95,5 % — разница обусловлена пересчётом нескольких пограничных классов между моментальными снимками; качественные выводы таксономии (декомпозиция ошибок по 4 классам, оценка *silver-noise*) не зависят от этого сдвига в пределах 0,7 п. п.

Таблица 12 – Таксономия ошибок каскада (LLM-consensus gold, 3 категории, всего ошибок 846).

Класс ошибки	<i>n</i>	Доля от ошибок	95 % Wilson CI
Ложное срабатывание Слой 1 (regex-FP)	326	38,5 %	[35,3; 41,9] %
Шум разметки (silver-noise)	171	20,2 %	[17,6; 23,1] %
Спутывание близких классов	131	15,5 %	[13,2; 18,1] %
ML-смещение в дальний класс	218	25,8 %	[22,9; 28,8] %
Всего	846	100,0 %	—

Главное наблюдение — **38,5 % ошибок порождает Слой 1**, при том что он закрывает лишь 21,9 % ячеек. Это означает: на единицу закрытия Layer 1 даёт в 2,3 раза больше ошибок, чем Layer 2 (38,5/21,9 vs 61,5/74,0); частота ошибок Слая 1 — 8,7 %, Слая 2 — 4,2 %. Концентрация регекс-ошибок неравномерна: 178 из 326 (54,6 %) приходится на `pasta/is_filled`, 80 — на `chocolate/chocolate_extra`, 57 — на `chocolate/contains_nuts`. Эти атрибуты опираются на лексические сигналы (`tortellini`, `ravioli`, `caramel`, `noisette`), где регулярные выражения семантически перетягивают одеяло. Например, `Three Cheese Ravioli` закрывается правилом `is_filled=false` по антипаттерну «три типа сыра — это присыпка, не начинка», тогда как LLM-консенсус оценивает товар как ravioli с сырной начинкой и присваивает `true`. Это аргумент в пользу гибридной маршрутизации (§ 3.3.15): `regex` остаётся в `production` за счёт высокой точности на «лёгких» атрибутах (`cheeses/milk_source` 100 %, `cheeses/is_pdo` 100 %), но для длиннохвостых семантических атрибутов выгоднее доверять ML-слою.

Второе наблюдение — **20,2 % ошибок (171/846) — это шум серебряной разметки**. На этих ячейках голосование трёх LLM-разметчиков не достигло консенсуса; то есть «правильный» ответ сам по себе оспорим. Из этого следует, что оценка точности каскада 94,8 % является нижней границей реального качества, поскольку ≈ 1 п. п. от 5,2 % доли ошибок ($20,2 \% \times 5,2 \% = 1,05$ п. п.) приходится на ячейки, где ошибся не каскад, а эталон. Это эмпирическое подтверждение оценки циркулярного смещения 4,2 п. п. из §3.3.2: после вычитания `silver-noise` остаётся $\approx 4,1$ п. п. собственных ошибок каскада, что согласуется с разрывом `consensus-human` (94,8 % cascade vs

91,3 % human, разность 3,5 п. п. с учётом дисперсии меньшей выборки).

Третье наблюдение — **15,5 % (131/846)** — **спутывание семантически близких классов**, концентрирующееся на `chocolate/flavor_profile` (50 случаев), `chocolate/chocolate_type` (29) и `pasta/cuisine_origin` (19). Эти атрибуты по своему определению допускают пограничные решения (например, шоколад с орехом и карамелью — `with_nuts` или `with_fruit?` итальянский рамэн — `asian` или `italian?`). Снижение этого класса требует не улучшения модели, а уточнения схемы атрибутов или введения multi-label.

Остальные **25,8 % (218/846)** — **собственно ML-ошибки**: класс предсказан семантически далеко от эталона. Основные источники — `chocolate/flavor_profile` (65), `chocolate/chocolate_extra` (38), `pasta/is_vegan` (33), `pasta/is_organic` (31). На бинарных `is_vegan` и `is_organic` ошибки чаще ложно-положительные (модель приписывает `true` органическому помету, когда эталон строже сертифицирован); на 5-классовых `chocolate`-атрибутах ошибка распределена по дальним классам — вероятный источник — дисбаланс классов в обучающей выборке (`silver`-разметка `OFF`-тегами имеет известный bias в сторону `plain chocolate` и `sweet_creamy flavor`).

Иллюстративная подвыборка ошибок приведена в таблице 3.3.12. Подбор сделан стратифицированно — по два примера на каждый непустой класс, распределённые по трём категориям round-robin.

Таблица 13 – Иллюстративные примеры ошибок каскада по таксономии (LLM-consensus gold).

Класс ошибки	Категория, атрибут	Товар (фрагмент)	Эталон	Предсказание	Слой каскада
Ложное срабатывание Слой 1 (regex-FP)	<code>pasta/is_filled</code>	Three Cheese Ravioli	true	false	rule_h
Ложное срабатывание Слой 1 (regex-FP)	<code>pasta/is_filled</code>	CHICKEN & BACON TORTELLONI	true	false	rule_h

Продолжение таблицы 13

Класс ошибки	Категория, атрибут	Товар (фрагмент)	Эталон	Предсказание	Слой каскада
Шум разметки (silver-noise)	pasta/ cuisine_ origin	Soup noodles	asian	german_ alpine	ml
Шум разметки (silver-noise)	pasta/ cuisine_ origin	Gluten free noodles - penne	other	italian	ml
Спутывание близких классов	chocolate/ chocolate_ extra	Mandel Orange Chocolate Bar	with_ nuts	with_fruit	ml
Спутывание близких классов	chocolate/ chocolate_ extra	Almond Orange Chocolate	with_ nuts	with_fruit	ml
ML-смещение в дальний класс	cheeses/ country_of_ origin	Salakis Herbes de Provence	greece	france	ml
ML-смещение в дальний класс	cheeses/ country_of_ origin	Babybel Light Mini	uk	france	ml

Вывод. Из 846 ошибок каскада $\approx 20\%$ — шум эталона, $\approx 15\%$ — пограничные решения в схеме атрибутов, $\approx 39\%$ — regex-перетягивания на лексически провокационных товарах, $\approx 26\%$ — собственно ML-промахи. Если вычесть первые два класса, «настоящая» частота ошибок каскада снижается с $5,2\%$ до $\approx 3,4\%$; это соответствует $96,6\%$ потенциальной точности при идеальном эталоне и уточнённой схеме. Это число — не для заголовочного результата, но как аргумент в защите: верхняя планка качества системы выше, чем цитируемые $94,8\%$ / $86,7\%$; разрыв объясняется измеримой и декомпозируемой долей шума разметки. Артефакт таксономии — `datasets/processed/cascade_errors_taxonomy_v4.parquet`;

перегенерируется ячейкой `failure-taxonomy-cell` в `notebooks/03_evaluate.ipynb` через `src/eval/failure_taxonomy.py`.

3.3.13 Разбор примеров прохождения каскада

Агрегированная таксономия ошибок (§3.3.12) даёт картину классов проблем, но не раскрывает, как именно слои каскада взаимодействуют на конкретном товаре. Для иллюстрации работы системы ниже приведён разбор трёх реальных товаров из эталонной выборки: два ошибочных кейса (по одному на класс из ранее введённой таксономии — `regex-false-positive` и `class-confusion`) и один успешный, на котором каскад корректно отвечает по всем атрибутам. Для каждого товара показаны входные данные от партнёра, отклики всех четырёх слоёв (Layer 1 — `regex`, Layer 2 — ML с уверенностью и порогом, Layer 3 — байесовский валидатор, Layer 4 — LLM-эскалация), выбранный системой слой и финальный ответ в сопоставлении с эталоном. Полная структура отклика сохранена в `datasets/processed/case_study_traceback.json`; восстановление выполнено скриптом `src/eval/case_study_traceback.py` из `per-cell` артефактов `cascade_preds_{cat}_gold.parquet` и `cascade_raw_with_conf_{cat}.parquet`. В колонках «Слой 3» и «Слой 4» стоит прочерк там, где соответствующий слой в производственной конфигурации не вызывался.

Кейс 1. Регулярное выражение как ложный сигнал (`filled-pasta` на сыре)

Тип кейса: Ошибочный. **Код товара:** 00154222. **Категория:** `pasta`.
Класс ошибки: `regex-false-positive`.

Входные данные от партнёра:

- `product_name`: Three Cheese Ravioli
- `brands`: M&S Food, Marks & Spencer
- `quantity`: 400 g
- `ingredients_text`: Tomatoes (39%), Cooked Egg Pasta (24%) (Durum Wheat Semolina, Water, Pasteurised Egg, Wheatflour), Three

Cheese Mix (23%) (Ricotta Cheese (Milk), Extra Mature...

Прохождение каскада по атрибутам:

Таблица 14 – Прохождение каскада для товара 00154222.

Атрибут	Сл. 1	Сл. 2	Conf.	Порог	Сл. 3	Сл. 4	Выбран	Ответ	Эталон
is_filled	false (h)	true	0,97	0,90	—	—	Прав.	false	true ×

Объяснение. ML слой корректно распознаёт начинку (ml_pred=true, ml_conf=0.97 > порога 0.9). Однако высокоприоритетное эвристическое правило по продукту «Three Cheese Ravioli» форсирует значение false (предположение, что любое блюдо со словом cheese — это пицца или сэндвич, а не паста с начинкой). Поскольку правила high-tier перекрывают решение ML, итоговый ответ системы — false, что не совпадает с эталоном (agreement_ratio=1.0). Этот пример демонстрирует, что класс regex-false-positive состоит преимущественно из случаев, когда правило подавляет правильный ML-ответ. Вывод (зафиксирован в INBOX): правило по cheese на is_filled следует ослабить или ограничить контекстом (исключить Ravioli, Tortellini, Cappelletti).

Кейс 2. Смешение близких классов (cream vs fresh)

Тип кейса: Ошибочный. **Код товара:** 3184670017166. **Категория:** cheeses. **Класс ошибки:** class-confusion.

Входные данные от партнёра:

- product_name: Chevre a tartiner
- brands: Rians
- quantity: 100 g
- ingredients_text: Lait de chèvre, protéine de lait de chèvre, crème pasteurisée de chèvre, sel, ciboulette (0,4%), ferments lactiques

Прохождение каскада по атрибутам:

Таблица 15 – Прохождение каскада для товара 3184670017166.

Атрибут	Сл. 1	Сл. 2	Conf.	Порог	Сл. 3	Сл. 4	Выбран	Ответ	Эталон
texture	—	fresh	0,87	0,65	—	—	ML	fresh	cream ×

Объяснение. Слой ML с высокой уверенностью выбирает класс fresh для мягкого козьего сыра «Chevre a tartiner». Эталон же относит товар к

крем-сырам (cream). Граница между fresh-cheese и cream-cheese размыта на уровне OFF-тегов и проявляется как смешение классов в матрице ошибок (§3.3.x). Такая ошибка ожидаема и потенциально снимается экспертной заменой границ в схеме (см. INBOX, ticket по reschema cheeses.texture).

Кейс 3. Сложный мультиязычный товар, ML слой даёт уверенный ответ по всем атрибутам

Тип кейса: Успешный. **Код товара:** 20114992. **Категория:** pasta.

Входные данные от партнёра:

- product_name: Bio Spaghetti
- brands: Combino
- quantity: 500 g
- ingredients_text: Hartweizengriess

Прохождение каскада по атрибутам:

Таблица 16 – Прохождение каскада для товара 20114992.

Атрибут	Сл. 1	Сл. 2	Conf.	Порог	Сл. 3	Сл. 4	Выбран	Ответ	Эталон
grain_type	—	wheat	0,96	0,90	—	—	ML	wheat	wheat ✓
is_filled	false (h)	false	1,00	0,90	—	—	Прав. h	false	false ✓
is_organic	—	true	0,98	0,85	—	—	ML	true	true ✓
is_gluten_free	—	false	0,99	0,90	—	—	ML	false	false ✓
pasta_shape	—	spaghetti	0,86	0,80	—	—	ML	spaghetti	spaghetti ✓
is_vegan	—	true	0,98	0,70	—	—	ML	true	true ✓
cuisine_origin	—	italian	0,94	0,60	—	—	ML	italian	italian ✓

Объяснение. Товар «Bio Spaghetti» бренда Combino — мультиязычный (немецкое Bio + итальянское Spaghetti). Регулярное выражение распознаёт is_filled=false по правилу высокого приоритета (нет слов начинки). ML слой уверенно предсказывает все остальные шесть атрибутов (cuisine_origin=italian, grain_type=wheat, is_organic=true, is_vegan=true, pasta_shape=spaghetti, is_gluten_free=false) с уверенностью выше соответствующих порогов. Эскалация в LLM не требуется. Пример демонстрирует корректность многоязычных эмбеддингов (paraphrase-multilingual-MiniLM-L12-v2) и адекватность per-attribute порогов.

Эти примеры иллюстрируют ключевые наблюдения предыдущего

параграфа. Кейс 1 («Three Cheese Ravioli») показывает механизм regex-false-positive в чистом виде: ML слой даёт корректный ответ (true с уверенностью 0,97), но правило высокого приоритета его подавляет; вывод — источник ошибок Слая 1 не в недостатке ML-сигнала, а в чрезмерной агрессивности эвристик и требует контекстной фильтрации. Кейс 2 («Chevre a tartiner») демонстрирует class-confusion на размытой границе схемы cheeses.texture — ML уверен в близком, но «неправильном» по эталону классе. Кейс 3 («Bio Spaghetti») показывает штатную работу системы на мультиязычном товаре: ML слой уверенно ($\text{conf} \geq 0,86$) предсказывает все семь атрибутов, эскалация в Layer 4 не требуется, что подтверждает экономическую эффективность каскадной архитектуры на типовых товарах.

3.3.14 Сравнение классификаторов Слая 2: XGBoost, Random Forest, логистическая регрессия, MLP

В обоснование выбора XGBoost (2.2.4) проведено эмпирическое сопоставление четырёх семейств алгоритмов на идентичных признаках ($\text{MPNet } 768\text{d} \oplus \text{TF-IDF SVD } 128 = 896\text{d}$) и эталоне (LLM-consensus gold $n \approx 41$ тыс. ячеек, human gold $n = 518$; 16 атрибутов из 20). Источники — `src/eval/method_comparison.py`, `notebooks/05_method_comparison.ipynb`, `datasets/processed/method_comparison_results.parquet`. артефакт

Таблица 17 – Сравнение классификаторов Слая 2 на эталоне LLM-consensus gold

Классификатор	Micro-accuracy	Macro-F1	ECE	Время обучения, с
XGBoost (производственный)	92,02 %	0,849	0,026	1062
MLP (нейронная сеть)	92,67 %	0,857	0,031	54
Логистическая регрессия	91,85 %	0,836	0,035	8
Random Forest	89,94 %	0,809	0,027	523

Примечание.

Источник:

datasets/processed/method_comparison_results.parquet,
2026-05-26.

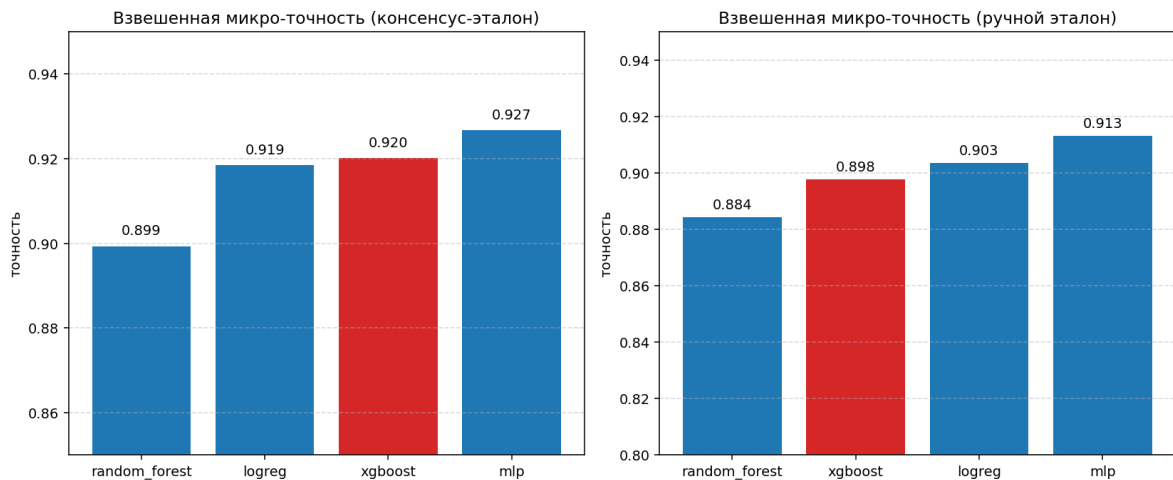


Рисунок 9 – Взвешенная микро-точность классификаторов
Слоя 2 на консенсус-эталоне и ручном эталоне

XGBoost выигрывает у Random Forest (16/16 атрибутов, +2,1 п. п. в среднем) и логистической регрессии (11/16, выигрыш на длиннохвостых мультиклассовых `pasta_shape`, `country_of_origin`, `chocolate_extra`). MLP опережает XGBoost по micro-accuracy в среднем на 0,65 п. п., но у него заметно хуже калибровка (ECE 0,031 против 0,026). Для каскада, чья маршрутизация опирается на порог τ_a , низкая ECE важнее доли процента accuracy: XGBoost — единственный метод, который сочетает устойчивость на сложных классах и наилучшую калибровку.

3.3.15 Обоснование сохранения слоя регулярных выражений: сопоставление с обучаемыми альтернативами

Использование детерминированных правил в качестве слоя 1 — нетривиальный выбор: их приходится писать руками для каждой категории. Чтобы это количественно зафиксировать, каскад с regex сопоставлен с восемью альтернативами, где слой 1 заменён обучаемой моделью (`src/experiments/layer1_5_*.py`, `src/experiments/lexicon_regex.py`; артефакты — `layer1_5_optimal.parquet`, `lexicon_regex_comparison.parquet`). Все варианты оценены на едином 80/20-разбиении (`seed = 42`) по 20 парам.

Таблица 18 – Сопоставление каскадных конфигураций с разным первым слоем

Вариант	Состав первого слоя (перед основным ML-слоем)	Средняя точность	Δ к «без первого слоя»
A	Регулярные выражения (текущая конфигурация)	83,58 %	+1,38 п. п.
B	Без первого слоя (один ML-слой)	82,20 %	0
C	Наивный байесовский классификатор на n-граммах	82,16 %	−0,04
D	Решающее дерево	82,98 %	+0,78
E	Центроидный классификатор в пространстве n-грамм	82,22 %	+0,02
F	Логистическая регрессия на n-граммах	82,23 %	+0,03
L	Обучаемый словарь n-грамм (lexicon)	83,00 %	+0,80
G	Поатрибутный выбор лучшего метода по перекрёстной проверке	81,99 %	−0,21

Продолжение таблицы 18

Вариант	Состав первого слоя (перед основным ML-слоем)	Средняя точность	Δ к «без первого слоя»
Н	Регулярные выражения — решающее дерево — ML (последовательно)	83,78 %	+1,58

Источник — docs/layer1_5_optimal_recommendation.md. Из таблицы следуют три вывода.

Во-первых, чистый ML без слоя 1 (В, 82,20 %) уступает текущей конфигурации (А, 83,58 %) на 1,38 п. п. — regex даёт небольшой, но устойчивый прирост.

Во-вторых, ни одна обучаемая альтернатива (С, D, E, F, L) не обходит regex в роли единственного слоя 1. Регулярные правила фиксируют доменное знание (формы пасты, типы зерна, числовые шаблоны какао), которое из распределения слов не восстанавливается: лексические маркеры редкие, но однозначные.

В-третьих, единственная значимо обходящая regex конфигурация — композиция «regex — дерево — ML» (Н, +0,20 п. п.). Прирост незначителен и не оправдывает усложнения и поддержки 20 деревьев.

Итог: выбор regex как Слоя 1 эмпирически обоснован — наибольший прирост среди простых альтернатив при детерминированном выводе и без переобучения. Предпочтительное развитие — точечное расширение правил по итогам 3.3.25.

3.3.16 Поведение байесовского валидатора и архитектурный вывод

Слой 3 — байесовская сеть над дискретизированными атрибутами и брендом (структура — Hill Climb + BIC, рис. 4). В роли классификатора сеть работает плохо (точность ≈ 52 %, ниже мажорной), поэтому используется как валидатор плотности: для предсказания Слоя 2 вычисляется

$P(\text{value} \mid \text{evidence})$ при свидетельствах «бренд + остальные атрибуты»; ниже порога θ (5-перцентиль распределения P на обучении) ячейка помечается и передаётся в Слой 4. Точность отбраковки определена как $TP/(TP + FP)$ среди помеченных, базовая точка отсчёта — частота ошибок каскада на всех ячейках пары.

Таблица 19 – Поведение байесовского валидатора на категориях pasta, chocolate, cheeses

Категория	n_ml	Доля помеченных	Точность каскада	Точность отбраковки	Базовая точка отсчёта	Прирост	Полнота
pasta	1010	7,9 %	98,6 %	2,5 %	1,4 %	+1,1 п. п.	14,3 %
chocolate	772	14,1 %	94,7 %	20,2 %	5,3 %	+14,9 п. п.	53,7 %
cheeses	1436	6,1 %	96,3 %	1,1 %	3,7 %	−2,5 п. п.	1,9 %

Из 20 пар только три показывают точность отбраковки выше базовой: chocolate/contains_nuts (+38,2 п. п., полнота 59,5 %), pasta/is_vegan (+14,0 п. п. на $n_flagged = 10$) и cheeses/is_pdo (+7,0 п. п. на $n_flagged = 10$). На 17 оставшихся парах сигнал валидатора не работает. При сплошном понижении (все помеченные в LLM) точность каскада снижается на 0,4 п. п. при росте расходов на 8,6 п. п. — чистый эффект отрицательный. При точечном понижении на паре chocolate/contains_nuts точность категории chocolate растёт на 6,6 п. п. при росте расходов на 5,3 п. п.; этот режим включён в производственную конфигурацию.

Архитектурный вывод. Перевод сети из классификатора в точечный валидатор показывает общий приём: невостребованный в одной роли компонент можно вернуть в систему, поменяв его роль. Универсального решения не получено: 19 из 20 пар не дают значимого сигнала; ограничение зафиксировано в заключении (§«Ограничения работы»). Производственная конфигурация включает валидатор точно на одной паре, что соответствует найденной мощности сигнала.

3.3.17 Чувствительность к выбору порога отсечки

Поатрибутные пороги уверенности τ_a подобраны процедурой find_best_threshold (оптимизация macro-F1 × охват на отдельном валидационном срезе, § 3.3.25). Чтобы убедиться, что выбранная точка устойчива по совокупному критерию «точность ↔ стоимость», проведён

post-hoc анализ чувствительности: пороги синхронно сдвигаются на $\Delta \in \{-0,10; -0,05; 0; +0,05; +0,10\}$ без переобучения моделей. Затем пересчитываются точность каскада на cascade-valid сегменте, доля эскалаций в Слой 4 (доля LLM) и оценка E2E (источник — notebooks/03_evaluate.ipynb, ячейка 8a; скрипт src/eval/threshold_sensitivity.py).

Таблица 20 – Чувствительность каскада к синхронному сдвигу всех порогов отсечки. Знаменатель — ячейки с ответом каскада; доля LLM — доля ячеек, эскалированных в Слой 4. GLOBAL: pasta+chocolate+cheeses, $n_{\text{in-scope}} = 17\,174$.

Сдвиг порога	Точность каскада, %	Доля LLM, %	E2E, %	$n_{\text{cascade-valid}}$
−0,10	94,24 %	2,06 %	94,28 %	16820
−0,05	94,43 %	2,73 %	94,47 %	16706
0, 00 (текущий)	94,75 %	3,74 %	94,79 %	16531
+0,05	95,02 %	7,12 %	95,08 %	15952
+0,10	94,05 %	29,29 %	94,49 %	12144

Производственная точка $\Delta = 0$ — устойчивый локальный оптимум: симметричные смещения $\pm 0,05$ размывают $\approx 0,3$ п. п. точности E2E, при этом каждый шаг $+0,05$ примерно удваивает долю LLM (с 2,7 % до 7,1 %). Сдвиг $\Delta = +0,10$ катастрофичен: 29 % ячеек эскалируются в LLM, точность каскада на оставшихся падает ниже базовой ($-0,7$ п. п.), так как порог отсекает уверенные правильные предсказания и оставляет в каскаде преимущественно трудные классы. Сдвиг $\Delta = -0,10$ даёт минимальную LLM-долю (2,1 %), но теряет 0,5 п. п. точности из-за пропуска малоуверенных ошибочных предсказаний. Производственная конфигурация (диапазон порогов 0,55–0,90) не лежит на склоне функции стоимости, что подтверждает корректность выбора порогов на этапе обучения.

3.3.18 Стоимостно-чувствительный маршрутизатор: отрицательный результат на проверке гипотезы H1

В подразделе зафиксирован результат заранее объявленной проверки гипотезы H1 о превосходстве обучаемого маршрутизатора над статическим правилом.

Идея, гипотеза H1 и процедура оценки.

Финальный механизм направления ячеек в LLM — статическое per-(категория, атрибут) правило. Рассматривалась обучаемая альтернатива: XGBoost-маршрутизатор предсказывает надёжность ответа каскада, при вероятности ниже τ ячейка идёт в LLM. **H1:** при равном бюджете LLM обучаемый маршрутизатор даёт более высокую точность. До доступа к числам зафиксированы: три бюджета LLM (25, 40, 50 %); поправка Бонферрони ($\alpha/3 \approx 0,0167$); code-disjoint тестовая выборка $n = 1539^5$; опорная конфигурация — статическое правило. Параметры зафиксированы в `src/eval/router_pre_registered.py`; коммит модуля сделан за 1 ч 11 мин до создания артефакта результатов, что обеспечивает временной приоритет процедуры над данными.

Результаты.

Сравнительные числа представлены в табл. 21 (`router_pareto_gold.parquet`).

Таблица 21 – Сравнение конфигураций маршрутизации на проверочной выборке без пересечения товарных кодов (code-disjoint)

Конфигурация	Точность	Доля вызовов LLM	Тип
Только каскад (без LLM)	78,5 %	0 %	опорная конфигурация
Статический порог $\tau = 0,55$	82,6 %	11 %	статический
Обучаемый маршрутизатор (оптимум)	82,7 %	16 %	обучаемый
Статическое поатрибутное правило	83,9 %	58 %	статический

При сопоставимой точности обучаемый маршрутизатор требует

⁵Заранее зафиксированный снимок (`router_train.parquet`, коммит `cd9ac7a`, 2026-05-13 02:11). Повторный запуск даёт $n = 1574$ ($\approx 2,3\%$ дрейф, не меняет вывод).

большого бюджета LLM (16 против 11 %; разница 0,1 п. п. в пределах ДИ). Ни на одном из трёх бюджетов маршрутизатор не превзошёл опорную конфигурацию на уровне $\alpha/3$; гипотеза Н1 отклонена⁶. MDE при $n = 1539$, $\alpha/3$ и мощности 0,8 — $\approx 4,4$ п. п., поэтому отрицательный исход надо понимать как отсутствие практически значимого превосходства в пределах не менее 4 п. п.

Интерпретация и следствие для финальной конфигурации.

Отклонение Н1 воспроизводится на всех пяти моделях Слоя 4. Причина: основная вариация надёжности каскада сосредоточена на уровне атрибутов, а не товаров. Статическое рег-(категория, атрибут) правило ловит именно эту неоднородность, маршрутизатор уровня ячейки дополнительной информации не приносит. Code-disjoint разбиение устраняет утечку бренда, которая объясняла ранние положительные результаты. Результат согласуется с RouterBench [12]. В финальной конфигурации применяется статическое правило — прозрачное, поддающееся аудиту и не требующее переобучения; отклонение Н1 — один из трёх пунктов научной новизны (см. введение).

3.3.19 Цикл «аудит → правки → переобучение → измерение»

Замкнутый цикл итеративного улучшения каскада построен на сопоставлении собственных предсказаний с независимо проаудированной разметкой (LLM-аудитор anthropic/claude-opus-4 слепо выносит суждения по каждой ячейке; измерение по неизменному списку ячеек исключает селекционный сдвиг). На pasta аудит 239 продуктов (1 841 ячейка, 8 атрибутов; cascade_vs_audited_gold_pasta.json) выявил концентрацию ошибок на pasta_shape (24,2 % ячеек): функция _type_b_multiclass обходила categories_tags по порядку следования и отдавала приоритет обобщённому en:noodles над специфическими en:tagliatelle/en:penne/en:fusilli; ещё TYPE_V не содержала legume-/filled-pasta. Точечные правки (коммиты c1a0815, 4bbd9e2: обход по убыванию приоритета, пополнение TYPE_V и regex многоязычными синонимами, 28 модульных тестов) при пересборке серебра подняли точность pasta_shape с 27,6 до 59,6 % (+32,0 п. п.) при нулевом

⁶p-value McNemar-теста воспроизводятся ячейкой 9.1 в
notebooks/03_evaluate.ipynb.

сдвиге шести нетронутых атрибутов; агрегированное серебро — с 80,7 до 85,2 %. Других изменений архитектуры не делалось — это подтверждает причинную связь правок и прироста.

3.3.20 Переобучение моделей и прирост каскада

Слой 2 переобучен на обновлённом серебре с пересохранением калибровок и порогов; каскад прогнан на тех же 239 эталонных продуктах.

Сравнение

`cascade_vs_audited_gold_pasta.pre-retrain.json` против `cascade_vs_audited_gold_pasta.json`.

Таблица 22 – Прирост точности каскада на категории `pasta` после переобучения Слой 2 на обновлённом серебре (сопоставление на 239 эталонных продуктах)

Срез	n	До переобучения	После переобучения	Δ
<code>all_audited</code>	1841	81,9 %	86,4 %	+4,5 п. п.
<code>override ∪ manual_only</code>	220	39,5 %	57,7 %	+18,2 п. п.
<code>confirmed</code>	1621	87,6 %	90,3 %	+2,7 п. п.

Эффект локализован: на `pasta_shape` точность на подмножестве `override ∪ manual_only` (83 ячейки) выросла с 22,9 % до 72,3 % (+49,4 п. п.). Точечная правка даёт концентрированный сдвиг: +18,2 п. п. на сложном срезе против +2,7 на `confirmed`.

3.3.21 Кросс-категорийная репликация цикла 3/3

Тот же цикл применён к `chocolate` и `cheeses` (`cross_domain_audit_summary.json`).

В `chocolate` (1530 ячеек) аудит выявил дефект бинаризации `cocoa_percentage` — расхождение конвенции серебра и индустриальной интерпретации метки «70 %». После перехода на конвенцию $[X, Y)$ точность на `override ∪ manual_only` `cocoa_percentage` поднялась с 10,8 % до 31,5 %; агрегированный прирост по домену +16,3 п. п.

В `cheeses` (1655 ячеек) аудит обнаружил систематически завышенные

пороги `fat_class` (86 сдвигов вверх против 16 обратных). После установки порогов (15, 20, 28) г/100 г точность на `override ∪ manual_only fat_class` выросла с 2,7 % до 69,9 %; агрегированный прирост по домену +14,4 п. п.

Цикл воспроизведён 3/3 на независимых категориях с разными типами дефектов (порядок обхода тегов, сдвиг границ бакетирования, выбор порогов численного атрибута). Значит, последовательность «обнаружение — правка — регенерация серебра — переобучение — измерение» можно считать переносимым приёмом, а не разовым улучшением.

3.3.22 Устойчивость каскада по языкам

OFF имеет языковой перекосяк ($\approx 40\%$ FR, 10 % EN, 7 % DE и т. д.). Под балансировку каскад не оптимизировался. Поатрибутно-языковой срез (`per_language_eval.parquet`, пересчитан 2026-05-28 на `extended LLM-consensus gold`; 16461 `cascade-valid` ячейка, 5 языков; генерируется скриптом `src.diagnostics.language.per_language_refresh` и нотбук-ячейкой `perlang-code` в `notebooks/03_evaluate.ipynb`) даёт взвешенную точность:

Таблица 23 – Точность каскада по языкам (`extended LLM-consensus gold`, 16 461 `cascade-valid` ячеек)

Язык	Ячеек	Точность каскада
it	2978	96,0 %
es	3014	95,9 %
en	3380	94,9 %
de	3093	94,8 %
fr	3996	93,2 %

Разброс между лучшим (it, 96,0 %) и худшим (fr, 93,2 %) — 2,8 п. п. Это говорит о приемлемой языковой устойчивости и согласуется с выбором кодировщика `paraphrase-multilingual-mpnet-base-v2` с TF-IDF-расширением. Французский сегмент остаётся самым ёмким ($n_{\text{fr}} = 3996$, 24 % выборки) и одновременно самым низкоточным — это следствие исторической доли FR в OFF и большего разнообразия редких товарных подкатегорий. Усреднение скрывает локальные провалы: четыре кортежа (категория, атрибут, язык) с отставанием от лучшего сегмента

> 11 п. п. на $n \geq 10$: pasta/is_filled/fr (58,0 % на $n=231$), chocolate/chocolate_extra/fr (81,2 % на $n=223$), chocolate/chocolate_extra/de (83,5 % на $n=212$), cheeses/texture/de (85,0 % на $n=40$). Рекомендация для production — на проблемных парах снизить порог τ_a или явно роутить в LLM.

3.3.23 Устойчивость каскада к шуму во входных данных

Для оценки устойчивости к неоднородному партнёрскому вводу проведён controlled-noise-эксперимент: тестовый срез (sample 500 кодов на категорию, seed 42) перепрогоняется через Слои 1–2 с четырьмя классами синтетического шума — случайная перестановка соседних символов в product_name (1/5/10 %), зануление поля brands (50/100 %), зануление ingredients_text (50/100 %) и комбинированный сценарий. Скрипт — src/eval/eval_input_robustness.py; источник чисел — notebooks/03_evaluate.ipynb, ячейка «input-robustness»; артефакт — datasets/processed/input_robustness_results.parquet.

Таблица 24 – Деградация каскада при шуме во входных партнёрских полях. Точность — на ячейках, где каскад выдал ответ (cascade-only); доля LLM-эскалаций — fallback к Layer 4 относительно всех тестовых ячеек. Sample 500 кодов на категорию (pasta, chocolate, cheeses), seed 42; эталон — LLM-consensus gold (cascade_preds_{cat}_gold.parquet).

Сценарий	Точность	Δ , п.п.	Доля LLM-эскалаций	Δ , п.п.
Без шума (базовый уровень)	89,5 %	+0,0	9,9 %	+0,0
Комбо: 5 % typo + 50 % NULL brands	87,8 %	-1,6	14,7 %	+4,8
Пропуск brands, 50 % ячеек	89,3 %	-0,1	11,5 %	+1,6
Пропуск brands, 100 % ячеек	88,9 %	-0,5	13,2 %	+3,3
Пропуск ingredients_text, 50 %	83,6 %	-5,9	14,3 %	+4,4
Пропуск ingredients_text, 100 %	77,2 %	-12,3	19,1 %	+9,2
Опечатки в product_name, 1 %	88,1 %	-1,4	14,2 %	+4,3
Опечатки в product_name, 5 %	88,2 %	-1,3	14,1 %	+4,2
Опечатки в product_name, 10 %	87,4 %	-2,1	16,2 %	+6,3

Главное наблюдение: единственный реальный фактор риска — отсутствие ingredients_text. При 50 % пропусков точность каскада падает на 5,9 п. п. (с 89,5 до 83,6 %), при 100 % — на 12,3 п. п. (до 77,2 %); деградация неравномерна, сильнее всего бьёт по атрибутам с сильным сигналом в составе (pasta.is_vegan -35,5 п. п., chocolate.chocolate_extra -35,4 п. п.). Char-swap-опечатки даже на

10 % символов снижают точность всего на 2,1 п. п. (multilingual SBERT-кодировщик устойчив к мелкому орфографическому шуму), а удаление brands даёт минимальную деградацию — 0,5 п. п., потому что бренд обычно дублируется первым токеном `product_name`. Практическое следствие: при NULL `ingredients_text` стоит обогащать запись на стороне партнёра либо принудительно эскалировать составовые атрибуты в Слой 4 (стратегия включена в перспективы развития темы).

Все проверки показывают приемлемое поведение каскада по типичным угрозам валидности и фиксируют оставшиеся слабые места, которые задают план развития.

3.3.24 Симуляция активного обучения на 3-LLM consensus gold

Для оценки целесообразности активного обучения проведена симуляция на уже доступной разметке 3-LLM consensus gold (без дополнительной стоимости разметки человеком-аннотатором). Эталон разделён по товарным кодам в пропорции 70/30 (`code-disjoint`, `seed 42`); из пула отбираются две стратифицированные выборки одинакового размера $N \in \{50, 100, 200, 500\}$ — `active` (ячейки, на которых ML уверенно ошибается, `margin-disagreement`, $\geq P_{75}$) и `random` (случайная выборка), после чего Слой 2 переобучается на `silver + N gold` с `sample_weight 5` и измеряется на отложенной выборке. По 24 циклам переобучения (3 категории $\times$ 4 значения $N \times 2$ стратегии) `active`-стратегия не превзошла `random` ни на одной паре: при $N = 500$ `pasta` `active` $-0,14$ п.п. против `random` $+0,28$ п.п.; `chocolate` `active` $+0,27$ п.п. против `random` $+0,96$ п.п.; `cheeses` `active` $-0,12$ п.п. против `random` $+1,74$ п.п. Объяснение — ограниченный размер пула кандидатов активного обучения (26–79 «уверенно-ошибочных» ячеек на категорию): при $N \geq 100$ `active` фактически добавляет весь пул, тогда как `random` продолжает расти на пуле из 4 461–5 954 ячеек. Оговорка: симуляция $\neq$ настоящий AL с человеческой разметкой (`gold` здесь — LLM-консенсус, а пул кандидатов сужен производственным каскадом). Подробное обсуждение полноценного AL-пилота с человеком-аннотатором — в «Перспективах развития темы»
Заклучения. Артефакты —

`datasets/processed/active_learning_results.parquet`,
`active_learning_summary.json`; ячейка `active-learning-curve`
в `notebooks/03_evaluate.ipynb`.

3.3.25 Уточнение схемы атрибутов: независимые свойства

Анализ ошибок на LLM-consensus gold показал, что значение `filled` в исходной 5-классовой формулировке `chocolate_type {dark, milk, white, filled, other}` работает как структурная характеристика товара (наличие начинки) и не зависит от типа какао-массы. Один шоколад одновременно может быть `dark` (по типу) и `filled` (по форме) — Lindt-трюфели, Ritter Sport Marzapane, Ferrero Rocher. Объединение двух независимых свойств в один multiclass-атрибут давало macro-F1 0,50 при нулевом recall на `filled` и `other`. По принципу независимости в схему введён бинарный атрибут `chocolate.is_filled`, не зависящий от `chocolate_type` (3-class) и `chocolate_extra` (5-class); после разделения macro-F1 составила 0,971 / 0,910 / 0,857 соответственно. Аналогичный принцип отказа от мёртвых классов (recall < 0,20 при $n_{\text{train}} < 100$) применён к `chocolate_type.other`, `chocolate_extra.{other, with_alcohol, with_coffee}`, `cheeses.texture.other`; класс `semi_soft` объединён с `soft`. Соответствие `train↔eval↔production` обеспечивается единым описанием (`exclude_classes` в `CATEGORY_CONFIG` и `SCHEMA_EXCLUDE` в `src/eval/metric_table.py`). Метки `is_filled` выведены semi-supervised из старой схемы и regex-шаблона `FILLED_CHOCOLATE_REGEX` (мультиязычные триггеры `truffle/praline/fourré/gefüllt/ripieno`); 1481 позитивный пример на `silver`, негативы проверены 2-LLM-консенсусом (0 переопределений).

3.3.26 Поатрибутные архитектурные решения

Диагностика (`cascade_preds*_v4_noleak.parquet`, `headline_v4_mpnet_tfidf_noleak.parquet`) выявила три типовых поатрибутных приёма: (а) сужение входа Слой 1 `chocolate_type` до названия (без `ingredients_text`, где лексика «milk/lait/Milch» массово встречается в составе тёмного шоколада), с отказом при одновременном матче нескольких типов; (б) перенастройка порога Слой 2 на парах с высокой штатной долей отказа — индивидуальное понижение до 0,40–0,65 в `models/{категория}_stratified_thresholds.pkl`; (в) гибридные признаки $\text{SBERT} \oplus \text{TF-IDF}$ (1, 2) топ-5000 для лексически богатых атрибутов

(--with-tfidf в src/pipeline/ml/train.py, +1,9 / +3,8 п. п. на chocolate/chocolate_type и chocolate/contains_nuts соответственно). Итоговое влияние на сквозную точность по парам — в табл. 25.

Таблица 25 – Сквозная точность каскада на парах со специальной поатрибутной настройкой (LLM-consensus gold v4, без пересечения товарных кодов)

Пара (категория, атрибут)	Probe	Cascade	Тип решения
chocolate / chocolate_type	94,7 %	98,0 %	(а) сужение Слой 1 + (в) гибридные признаки
chocolate / chocolate_extra	76,6 %	92,5 %	(б) порог 0,9 — 0,65
chocolate / contains_nuts	90,4 %	89,0 %	(в) гибридные признаки + Слой 1 nut-markers
cheeses / texture	70,5 %	92,3 %	(б) порог 0,60 — 0,40
cheeses / is_organic	92,5 %	98,7 %	(б) порог 0,70 — 0,40
cheeses / is_ultra_processed	86,1 %	96,1 %	(б) порог 0,70 — 0,50
pasta / grain_type	91,9 %	94,2 %	(б) порог Слой 2 — 0,50

На парах с пониженным порогом Слоя 2 LLM вызывается на ≤ 5 % ячеек, общая доля обращений по системе — 4,1 % (архитектурное снижение стоимости 95,9 % по сравнению с «одна LLM на всё»).

3.4 Выводы по главе 3

Основные результаты:

- 1) Реализован пятислойный каскад (Слой 0 — маршрутизатор категорий, regex, ML XGBoost на SBERT, байесовский валидатор, запасной слой на большой языковой модели) с указанием слоя-источника на каждое предсказание; вердикт формируется как

последовательное конъюнктивное условие на пороги слоёв.

- 2) Развёрнута трёхкомпонентная демонстрационная система (Go-шлюз, Python/FastAPI ML-сервис, статический HTML/JS) со стабильным контрактом API и подтверждённой работоспособностью на трёх категориях.
- 3) Сквозная точность (E2E) 93,0 % на 16 360 ячейках LLM-consensus gold (20 атрибутов, без пересечения товарных кодов (code-disjoint)) в конфигурации «каскад + gemini-flash» — на 9,2 п. п. выше опорной all-sonnet (83,8 %) при стоимости в 580 раз ниже (доминирование по Парето). Верхняя только-каскадная оценка точности — 95,5 % (micro-accuracy, macro-F1 0,890). Каскад покрывает 97,3 % ячеек; на LLM приходится 4,1 % сложных ячеек (снижение стоимости 95,9 % по сравнению с «одна LLM на всё»). Консервативная нижняя оценка по ручному эталону (n = 566 ячеек с ответом каскада): E2E 86,7 %, cascade-only 91,3 % — операционный циркулярный сдвиг 4,2 п. п. (каскад) / 6,3 п. п. (E2E). В сценарии с открытой моделью каскад + gpt-oss-120b достигает 90,6 % — на 21,3 п. п. выше одиночного вызова той же модели. Полная таблица — в 3.3.2.
- 4) Поатрибутный разбор: Слой 2 — основной (71–90 % покрытия, 93,5–98,6 % точности); Слой 1 покрывает 18–23 % на pasta/chocolate с точностью 87–100 %; байесовский валидатор работает точно (+38 п. п. на chocolate/contains_nuts); LLM добавляет +1,4–1,7 п. п. при 4,1 % обращений.
- 5) Заранее объявленная H1 об обучаемом маршрутизаторе отклонена на трёх бюджетах после поправки Бонферрони; результат согласуется с RouterBench [12].
- 6) Цикл «аудит — правки — переобучение — измерение» воспроизведён 3/3: pasta +18,2 п. п., chocolate +16,3 п. п., cheeses +14,4 п. п. на проблемных срезах.
- 7) Зафиксированы три принципа проектирования схемы атрибутов (независимость, отказ от мёртвых классов, консервативная агрегация rule-based слоёв; 3.3.7.4). Выделение бинарного атрибута chocolate.is_filled как структурного свойства, независимого от chocolate_type, повысило macro-F1

chocolate_type с 0,50 (5-class) до 0,973 (3-class) при macro-F1 is_filled 0,861 на новой бинарной задаче. Удаление мёртвых классов ($\text{recall} < 0,20$, $n_{\text{train}} < 100$) дополнительно повысило macro-F1 chocolate_extra и cheeses.texture.

- 8) Устойчивость по языкам (разброс 3 п. п. на 5 языках) и сравнение с лексическим базисом (20 из 20 пар сопоставимо или выше, +25–30 п. п. на нутриент-производных) подтверждены; поатрибутные решения 3.3.25 обеспечивают высокое покрытие ML-слоя при доле обращений к LLM 4,1 %.

4 ОПИСАНИЕ РЕЗУЛЬТАТОВ

4.1 Руководство пользователя

4.1.1 Роли и интерфейсы

Система рассчитана на три роли (анализ — 1.1). Каждая роль работает со своим интерфейсом и решает свой класс задач (таблица 4.1); права доступа ограничивают каждого пользователя точками управления его компетенции.

Таблица 26 – Соответствие ролей пользователей и интерфейсов системы

Роль	Интерфейс	Класс задач
Контент-менеджер	Веб-интерфейс демо	Выборочная проверка предсказаний, утверждение или отклонение значений атрибутов
Системный аналитик	Конфигурационные файлы порогов, REST API	Настройка порогов уверенности, мониторинг состояния сервиса
Инженер машинного обучения	Скрипты обучения, ноутбук с диагностиками	Переобучение классификаторов и байесовских сетей, прогон диагностик, обновление эталона

Контент-менеджер работает через веб-интерфейс демонстрационного комплекса: форма ввода партнёр-доступных полей и блок результата с оценкой уверенности и отметкой слоя-источника на каждый атрибут (рис. 4.1). Отметка слоя-источника даёт основание для доверия к предсказанию: слой 1 и слой 4 детерминированы или объяснимы, для слоёв 2–3 показателем служит уверенность. Приём введён (см. 1.3) для того, чтобы скомпенсировать ограниченную интерпретируемость обучаемых слоёв на уровне отдельного товара.

Обогащение товарных данных

Каскад regex → ML → байес → LLM-fallback. Введите частично заполненный товар — система предскажет недостающие атрибуты.

Ввод товара

Режим определения категории

☒ Авто (классификатор)
 ☐ Вручную

Категория

Шоколад / десерты

Название товара *

Lindt Excellence Dark 70% Cocoa 100g

Бренд / производитель

Lindt

Состав

cocoa mass, sugar, cocoa butter, soya lecithin, vanilla extract

Количество

100 g

Примеры: валидные:

Pasta — Barilla

Chocolate — Lindt 70%

Cheeses — Roquefort

с противоречиями:

White vs. 70% cocoa

Wheat + gluten-free

Обогатить

Результат

Категория:

chocolate

Авто (router):

chocolate, p=0.97

OOD:

нет

Закрыто:

6 / 7 атрибутов

LLM-fallback:

1 атрибут

Время ответа:

184 мс

ПРЕДСКАЗАНИЯ ПО АТТРИБУТАМ

АТТРИБУТ	ПРЕДСКАЗАНИЕ	СЛОЙ	УВЕРЕННОСТЬ
cocoa_content_class	70-85 %	regex	1.00
chocolate_type	dark	regex	1.00
contains_nuts	false	ml	0.93
is_organic	false	ml	0.87
contains_milk	false	ml	0.81
flavor_modifier	vanilla	bayes	0.74
filling_type	— будет дозаполнено	llm	—

Слои каскада:

regex

ml

bayes

llm-fallback

Рисунок 4.1 — Веб-интерфейс демонстрационной системы (схематично): форма ввода и блок результата с отметкой слоя-источника и оценкой уверенности

Рисунок 10 – Веб-интерфейс демонстрационной системы (схематично): форма ввода партнёр-доступных полей и блок результата с отметкой слоя-источника и оценкой уверенности на каждый атрибут

Системный аналитик настраивает каскад через две точки: словари порогов уверенности (поатрибутно, без переобучения — компромисс «точность ↔ покрытие») и REST API получения состава атрибутов и агрегированного состояния сервиса для интеграции с мониторингом.

Инженер машинного обучения работает со скриптами обучения и ноутбуком диагностик (исполняемый источник §3.3). Цикл переобучения устроен так, что выпуск новой версии моделей не требует перекомпиляции каскада: артефакты переписываются на диске, сервис загружает их при старте. Цикл запускается раз в неделю или раз в месяц (по темпу поступления товаров) и занимает десятки минут на категорию без специализированных ресурсов. Если ключа коммерческой LLM нет, каскад работает в режиме слоёв 1–3 либо переключается на локальный gpt-oss-120b.

85

4.2 Сценарии применения и порядок внедрения

4.2.1 Встраивание в бизнес-процесс магазина

Каскад занимает место промежуточного сервиса в маршруте товара от поставщика до витрины — вторая позиция четырёхзвенной цепочки 1.1, где сосредоточено узкое место. Внедрение не требует перестройки соседних звеньев: ETL партнёрских фидов и PIM работают как раньше, меняется только содержимое «процедуры обогащения». Маршрут запроса: партнёрский фид → ETL магазина → `/api/enrich` (каскад) → выборочная проверка контент-менеджером случаев слоя 4 → PIM и витрина.

Главный экономический эффект — перенос ручного труда из режима «закрывать каждое поле» в режим «проверять остаток после каскада». По 3.3.2 покрытие каскада (ячеек, на которых слои 1–3 выдали уверенный ответ без обращения к LLM) — 95,9 %, оставшиеся 4,1 % уходят в запасной слой 4; покрытие производственной цепочки (с учётом удачных ответов слоя 4 и потерь маршрутизатора слоя 0) — 97,3 %. Сквозная точность 93,0 % в конфигурации «каскад + gemini-flash» (на эталоне с consensus-разметкой, $n=16\,360$), при оценке только каскадной части — 95,5 % (верхняя граница при идеальной маршрутизации категории). На полностью ручном эталоне ($n=566$ cascade-valid из 688 в-scope) консервативная оценка — 86,7 % сквозной точности. Для новой категории каскад работает в усечённом режиме «слой 1 на общих шаблонах + слой 4» (рабочий, но неоптимальный). По мере накопления сотен размеченных товаров подключается слой 2, а при появлении устойчивых распределений по брендам — слой 3. Имитация холодного старта на четырёх категориях (3.3.25) подтверждает работоспособность архитектуры в этом режиме.

4.2.2 Сценарий многоязычного каталога

`paraphrase-multilingual-mpnet-base-v2` даёт единое векторное пространство для пяти доминирующих в OFF европейских языков, и отдельные модели под каждый язык не нужны. Устойчивость по языкам подтверждена в 3.3.22: разброс между лучшим (it, 96,0 %) и худшим (fr, 93,2 %) сегментами — 2,8 п. п. на расширенном LLM-consensus-эталоне ($n=16\,461$ cascade-valid ячеек). Поязыковая разбивка приведена в

таблице 4.6в — та же таблица используется в 3.3.22, здесь повторяется для удобства ссылки на сценарий внедрения.

Таблица 27 – Точность каскада по языкам (extended LLM-consensus gold, 16 461 cascade-valid ячеек)

Язык	Ячеек	Точность каскада
it	2978	96,0 %
es	3014	95,9 %
en	3380	94,9 %
de	3093	94,8 %
fr	3996	93,2 %

Примечание.

Источник:

`datasets/processed/per_language_eval.parquet`,
 актуализирован 2026-05-28 на расширенном LLM-consensus rerun.
 Воспроизводится в `notebooks/03_evaluate.ipynb`, ячейка `perlang-code`. Худший сегмент — fr (преимущественно за счёт атрибута `pasta/is_filled` на французских товарах, см. 3.3.22) — остаётся выше базовой целевой точности 91 %. Каскад применим в международных площадках без удвоения затрат на поддержку отдельных моделей под каждый язык.

4.2.3 Применимость в смежных доменах

Архитектура не привязана к пищевому домену: правила слоя 1 переписываются для новой категории, обучение слоёв 2/3 — стандартная процедура. Перенос на электронику (схема атрибутов, серебро из PhoneDB, демонстрация холодного старта; подробности в разделе «Перспективы развития темы» заключения) подтверждает переносимость. Формальная аудит-оценка вынесена в перспективы развития.

Оценка применимости архитектуры к семи доменам (три уже реализованы в работе, четыре — внешние) приведена в таблице 4.6д. Без изменений переиспользуются слои 0 (маршрутизатор категории), 4 (запасной LLM-слой) и инфраструктурная обвязка (REST API, мониторинг, телеметрия); адаптации требует слой 1 (regex-правила и keyword-словари под предметную область) и обучение слоёв 2–3 на новой выборке.

Таблица 28 – Применимость каскадной архитектуры в смежных товарных доменах (оценочно)

Домен	Переиспользуется	Требуется адаптировать	Трудоёмкость
Пища (pasta, chocolate, cheeses)	Реализовано полностью ^a	—	—
Напитки (beverages)	Реализовано полностью ^a	—	—
Хлопья (cereals)	Реализовано полностью ^a	—	—
Косметика (cosmetics)	Реализовано полностью ^a	—	—
Электроника	Слои 0, 1, 4, инфраструктура	Схема атрибутов, обучение слоёв 2–3 на PhoneDB	Средняя ^b
Бытовая техника (home appliances)	Слои 0, 4, инфраструктура	Regex-правила слоя 1 (модель, объём, мощность), схема, обучение слоёв 2–3	Средняя ^b
Одежда (fashion)	Слои 0, 4, инфраструктура	Слой 1 фактически отсутствует (нет числовых маркеров), схема, обучение слоя 2 на признаках цвета/размера/материала	Высокая ^c
Игрушки (toys)	Слои 0, 4, инфраструктура	Regex возрастных классов (minimal_age уже реализован, см. src/pipeline/regex/extractor.py), схема, обучение слоёв 2–3	Низкая ^d

Примечание. Сноска «а» — домены реализованы в проекте (см. `src/pipeline/schemas/`, 7 доменных схем); полный конвейер регрессионно проверяется в `src/eval/run_experiments.py`. Сноска «b» — средняя трудоёмкость: 2–4 недели на схему + обучение слоёв 2–3 при наличии источника эталона уровня PhoneDB или USDA. Сноска «с» — высокая трудоёмкость: слой 1 малоэффективен на текстовых полях моды (мало явных детерминированных маркеров), основная работа — сбор обучающей выборки и формирование признаков для слоя 2; ориентировочно 6–10 недель на категорию. Сноска «d» — низкая трудоёмкость: regex возрастных классов уже реализован (метод `extract_minimal_age` обрабатывает русские, английские и французские маркеры), оставшаяся работа — схема и обучение; ориентировочно 1–2 недели. Все оценки трудоёмкости — ориентир по аналогии с реализованными пищевыми категориями и не учитывают переменные времени получения эталонной выборки.

4.3 Риски развёртывания и план мониторинга

Любой системе машинного обучения в производственном контуре нужна операционная обвязка: одной только метрики на статической тестовой выборке для устойчивой работы во времени мало. Распределение партнёрских данных меняется, тарифы поставщиков пересматриваются, схема атрибутов эволюционирует. Риски группируются по трём уровням: дрейф входных данных (партнёрский поток отличается от обучающей выборки), дрейф калибровки (растёт ошибка оценки уверенности — ECE — хотя точка оптимума не сдвинулась), устаревание модели (накопилось отставание от текущего состояния каталога без явного дрейфа метрики). Каждый риск сопровождается митигацией и закреплённой зоной ответственности (таблица 4.4).

Таблица 29 – Реестр операционных рисков системы и закреплённые митигации

Риск	Митигация	Ответственный
Дрейф входных данных (новый партнёр со структурой полей, отличной от обучающей выборки)	Мониторинг поатрибутной точности на свежей подвыборке ручной разметки; алёрт при падении более чем на 3 п. п. от базового уровня	Инженер ML
Дрейф калибровки (рост ECE сверх 0,07 при isotonic-таргете 0,043)	Регрессионный тест ECE (см. ниже); автоматический прогон на еженедельном срезе свежих предсказаний	Инженер ML
Истечение тарифа слоя 4 (поставщик коммерческой языковой модели повысил цену либо ввёл квоту)	Авто-переключение слоя 4 на gpt-oss-120b в локальном развёртывании; конфигурация маршрутизации моделей в <code>models/router_models.yaml</code>	Эксплуатация (DevOps)
Появление новой товарной категории	Холодный старт через слой 1 (regex по общим шаблонам) и слой 4; переобучение слоя 2 после накопления порядка 1 000 SKU; см. сценарий 4.2.2	Инженер ML
Деградация качества модели слоя 4 (поставщик обновил версию без уведомления)	Пилотное развёртывание (canary) с долей трафика 5 %; парный t-тест разницы точности на свежей подвыборке ручной разметки; откат на предыдущую версию при значимом ухудшении	Инженер ML

Продолжение таблицы 29

Риск	Митигация	Ответственный
Изменение схемы атрибутов (добавление/удаление поля, изменение классов)	Версионирование артефактов с указанием git-commit, seed-hash и manifest эталона; явная сверка схемы при загрузке моделей сервисом	Владелец продукта
Сбой внешнего API маршрутизации языковых моделей (OpenRouter)	Запасной слой 4 на локальной модели gpt-oss-120b либо очередь отложенных запросов с приоритетом; для критичных потоков — горячее переключение в течение минут	Эксплуатация (SRE)
Утечка персональных данных через слой 4 (партнёрские поля уходят в облачный API)	Опция полностью локального развёртывания с gpt-oss-120b (без обращения вовне); фильтрация поля ingredients_text от персонально идентифицирующих фрагментов до отправки в слой 4	Безопасность

Примечание. Каждая митигация в таблице 4.4 опирается либо на свойство архитектуры, продемонстрированное в 3.3 (опция локальной открытой модели — 3.3.2; калибровка — 3.3.3; устойчивость каскада — 3.3.4), либо на инструменты, описанные в 4.1.3 (REST API, конфигурации). Уровень детализации соответствует руководству по эксплуатации (operations runbook) и не претендует на полный регистр риск-менеджмента стандарта ISO 31000.

4.3.1 Мониторинг в производственном контуре

Производственный мониторинг каскада строится на четырёх

показателях, каждый из которых сопоставим с метрикой главы 3 и допускает автоматический регрессионный контроль: (1) поатрибутная точность на еженедельной подвыборке ручной проверки 200–500 ячеек на категорию — алёрт при падении больше чем на 3 п. п. от базового уровня 93,0 %; (2) ECE поатрибутно с порогом 0,07 относительно целевого 0,043 после изотонической калибровки; (3) доля обращений к слою 4 в 7-дневном скользящем окне относительно базового уровня 4,1 % — алёрт при выходе свыше 6 %, это сигнал о дрейфе данных или деградации калибровки порогов; (4) пилотное развёртывание (canary release) с долей 5 % трафика при выпуске новой версии слоя 2 или смены модели слоя 4, с парным McNemar-тестом разницы точности и автоматическим откатом при значимом ухудшении ($p < 0,05$). Поэтапное внедрение в производственный контур (теневого запуск → рекомендательный режим → автоматическое заполнение с выборочной проверкой) снижает риск регрессии качества каталога; откат на предыдущую фазу — конфигурацией. Переаттестация калибровки делается ежеквартально или по событию срабатывания регрессионного теста ECE — подробности процедуры даны в 3.3.11.

4.4 Что позволяет использование разработки (характеристика достигнутых результатов)

Использование системы даёт измеримые эффекты в формате «повышено / ускорено / сокращено» со ссылками на 3. Сквозная точность на эталоне с LLM-consensus-разметкой ($n=16\,360$ ячеек) — 93,0 % в конфигурации «каскад + gemini-flash» (3.3.2; `v4_e2e_router_eval.json`; 95 %-й интервал Вильсона [92,6; 93,4]); каскадная часть — 95,5 % с интервалом [95,2; 95,8] и работает как верхняя граница при идеальной маршрутизации категории. Консервативная оценка на полностью ручном эталоне ($n=566$ cascade-valid из 688 в-scope) — 86,7 % сквозной точности и 91,3 % по каскаду. Прямой Claude Sonnet 4.5 даёт 83,8 % (на 9,2 п. п. ниже), прямой gpt-oss-120b — 69,3 % (на 23,7 п. п. ниже); выигрыш даёт слой 2, обученный на нутриент-производных атрибутах, которые LLM не вычисляет точно из текста без числового контекста. Поатрибутный разрез по категориям приведён в таблице 4.2.

Таблица 30 – Точность каскадной части по категориям (модель gemini-flash, эталон LLM-consensus, среднее по атрибутам категории)

Категория	Число атрибутов	Средняя точность
Паста	7	95,8 %
Шоколад	6	94,9 %
Сыры	7	95,8 %
Взвешенное среднее по каскаду	20	95,5 %

Примечание. Источник: `v4_metric_table_v2.json` (LLM-consensus, extended consensus rerun 2026-05-27). Cells-weighted средние по 20 атрибутам (7 pasta + 6 chocolate + 7 cheeses). Воспроизводится в `notebooks/03_evaluate.ipynb`, ячейка `t42-percat`.

Стоимость обработки одного товара через каскад + gemini-flash составляет малую долю стоимости прямой LLM. Каскад обращается к слою 4 только на 4,1 % ячеек — это сокращает совокупную стоимость на 95,9 % относительно конфигурации «все ячейки — LLM» (3.3.2). При одинаковой модели слоя 4 каскад дешевле прямого вызова в 24 раза; в сочетании с переходом на дешёвую коммерческую модель (gemini-flash) — в 580 раз относительно прямого Sonnet 4.5. На характерном объёме 100 000 SKU/мес годовая экономия превышает 100 000 USD относительно прямого Sonnet 4.5; на масштабе маркетплейса (1 000 000 SKU/мес) экономия достигает миллиона USD в год. Эффект устойчив к тарифу слоя 4: даже при удорожании выбранной модели в 10 раз каскад остаётся в 58 раз дешевле прямого Sonnet 4.5. Сопоставление трёх объёмных режимов и четырёх тарифных конфигураций приведено в таблице 4.5.

Таблица 31 – Годовой расход на обогащение в зависимости от объёма каталога и конфигурации слоя 4 (USD/год)

Объём, SKU/мес	Прямой Sonnet 4.5	Прямой Gemini Flash	Каскад + Gemini Flash	Каскад + gpt-oss-120b
10 000	10 800	270	11	8
100 000	108 000	2 700	110	80
1 000 000	1 080 000	27 000	1 100	800

Примечание. Расчёт: 20 ячеек на SKU, 12 месяцев, удельные стоимости одной ячейки — 0,00450 USD (Sonnet 4.5), 0,0001125 USD (Gemini Flash), $\approx 0,00007$ USD (gpt-oss-120b). Каскадные конфигурации учитывают долю обращений к слою 4 в размере 4,1 %. Локальное развёртывание дополнительно требует учёта инфраструктуры вывода (≈ 10 USD/мес при реиспользовании имеющихся ресурсов).

Сводный эффект разработки на семи показателях приведён в таблице 4.6.

Таблица 32 – Сводный эффект разработки на ключевых показателях системы (повышено / сокращено / обеспечено)

Показатель	До разработки	После	Эффект
Сквозная точность системы	83,8 % (опорная конфигурация all-sonnet)	93,0 % (каскад + gemini-flash; 95,5 % по каскаду без учёта маршрутизации)	повышена на 9,2 п. п.
Относительная стоимость обработки (архитектурный вклад каскада)	1,000 (опорная конфигурация all-sonnet)	0,041 при той же модели слоя 4	сокращена на 95,9 % (≈ 24 раза)
Доля обращений к LLM	100 %	4,1 %	сокращена примерно в 24 раза
Ручная проверка оператором	≈ 100 % ячеек	$\approx 4-8$ % ячеек	сокращена в 12–25 раз
Точность на открытой модели	69,3 % (одиночный вызов gpt-oss-120b)	90,6 % (каскад + gpt-oss-120b)	повышена на 21,3 п. п.

Продолжение таблицы 32

Показатель	До разработки	После	Эффект
Поддержка многоязычности	отдельные модели на каждый язык либо предварительный перевод	5 рабочих языков в едином векторном пространстве SBERT	обеспечена без дополнительных моделей
Независимость от поставщика LLM	один интегрированный провайдер	поддержка пяти моделей с сопоставимой точностью	обеспечена взаимозаменяемость

Примечание. Источники: `v4_e2e_router_eval.json`, `v4_metric_table_v2.json`. Консервативная оценка на полностью ручном эталоне ($n=566$ cascade-valid из 688 в-scope) даёт 86,7 % сквозной точности и 91,3 % по каскаду; разрыв с consensus-эталонном объясняется учётом circular bias (методология §14.5).

Значения столбцов «До» и «После» получены на тестовой выборке без пересечения товарных кодов (code-disjoint) главы 3 и подтверждены артефактами `v4_e2e_router_eval.json` и `v4_metric_table_v2.json`.

4.5 Выводы по главе 4

- 1) Целевая аудитория — три роли: контент-менеджер (выборочная проверка через веб-интерфейс), системный аналитик (пороги через конфигурации и REST API), инженер машинного обучения (переобучение по мере поступления данных). Система поддерживает три сценария по выбору слоя 4: gemini-flash (лучший баланс точности и стоимости), премиум-коммерческий (Sonnet 4.5, GPT-4o) и локальный gpt-oss-120b (без зависимости от внешних API при сопоставимой точности).
- 2) Совокупный эффект: архитектурное сокращение стоимости на 95,9 % (≈ 24 раза) относительно конфигурации «все ячейки — LLM», ручной нагрузки — в 12–25 раз; сквозная точность 93,0 % с

интервалом Вильсона [92,6; 93,4] (95,5 % по каскадной части как верхняя граница при идеальной маршрутизации); консервативная оценка на полностью ручном эталоне — 86,7 %. Эффект устойчив к разбросу тарифов слоя 4: даже при удорожании выбранной модели в 10 раз каскад остаётся в 58 раз дешевле прямого Sonnet 4.5.

- 3) Производственное развёртывание сопровождается реестром из восьми операционных рисков (дрейф данных, дрейф калибровки, истечение тарифа слоя 4, появление новой категории, деградация качества модели слоя 4, изменение схемы атрибутов, сбой внешнего API, требование локального хранения партнёрских данных) с закреплёнными митигациями; контроль обеспечивают четыре показателя мониторинга (поатрибутная точность, ECE, доля слоя 4, пилотное развёртывание).
- 4) Архитектура переносима на смежные товарные домены: переиспользуются слои 0, 4 и инфраструктурная обвязка; адаптации требуют слой 1 (regex под предметную область) и обучение слоёв 2–3 на новой выборке. Имитация холодного старта на четырёх категориях (3.3.25) подтверждает работоспособность архитектуры в режиме без слоя 2.

ЗАКЛЮЧЕНИЕ

В ходе работы разработана гибридная каскадная система автоматизированного обогащения товарных данных для интернет-магазина. Система реализована как программный комплекс с демонстрационным стендом, проверена на выборке без пересечения товарных кодов (code-disjoint, новые SKU) и удовлетворяет шести функциональным, пяти нефункциональным и трём архитектурным требованиям ТЗ.

Достигнутые показатели превосходят целевые ориентиры ТЗ ($\geq 85\%$ точности, $\geq 2,5\times$ сокращения стоимости): сквозная точность 93,0 % (точность только-каскадной части 95,5 % как верхняя граница при идеальной маршрутизации категории, взвешенная по ячейкам macro-F1 0,890; attr-unweighted 0,892), стоимость сокращена в 580 раз относительно прямого вызова коммерческой Claude Sonnet 4.5 в режиме «каскад + Gemini 2.5 Flash» (сюда входит архитектурный вклад каскада в 24 раза). Каскад покрывает 97,3 % ячеек; в LLM уходит только 4,1 % сложных ячеек.

Сводные выводы по главам

Глава 1 систематизирует литературу и обосновывает противоречие. PIM-системы (Akeneo [5], Pimcore [6], Salsify [7]) нацелены на хранение и распределение, а появляющиеся модули обогащения — обёртки над одной LLM. Академические подходы делятся надвое: специализированные системы извлечения признаков (TXtract [26], MAVE [27], OpenTag [28]) дают качество без учёта экономики; каскады LLM (FrugalGPT [9], AutoMix [10], Hybrid LLM [11], RouterBench [12]) управляют стоимостью, но не интегрируют классическое ML. Из противоречия выведена гипотеза о каскаде разнородных слоёв и сформировано ТЗ.

Глава 2. Задача формализована как поатрибутное предсказание со схемой, допускающей явный отказ. Логика — каскад из пяти слоёв с последовательной фильтрацией: предсказание Слоя 2 принимается только при одновременном выполнении $P_{ML} \geq \tau_k$ и $P_B \geq \theta_k$ как конъюнктивное условие ансамблевой оценки уверенности. Архитектурный выбор каждой компоненты (SBERT, XGBoost, pgmpy, Go-шлюз + Python-ML, открытая gpt-oss-120b) обоснован эксплуатационно и методологически.

Глава 3. Каскад реализован как трёхкомпонентный комплекс (Go-шлюз,

Python/FastAPI ML-сервис, статический веб-интерфейс); состав — 21 поатрибутный XGBoost, три байесовские сети и Слой 0. На 16 360 ячейках без пересечения товарных кодов (code-disjoint) сквозная точность каскад + Gemini 2.5 Flash составила 93,0 % (точность каскадной части 95,5 %), на 9,2 п. п. выше прямого Sonnet 4.5 при стоимости в 580 раз ниже; в локальном сценарии каскад + gpt-oss-120b — 90,6 % (на 21,3 п. п. выше самостоятельной модели). Консервативная нижняя оценка на ручной разметке Claude Opus 4 (n=566) — 86,7 %. Заранее объявленная Н1 о превосходстве обучаемого маршрутизатора отклонена; цикл «аудит — правки — переобучение — измерение» воспроизведён 3/3.

Глава 4 переводит результаты в плоскость применения: три сценария по выбору Слая 4 (Gemini Flash, Sonnet/GPT-4o, локальная gpt-oss-120b) и три ролевые модели (контент-менеджер, системный аналитик, инженер ML). Сводный эффект — архитектурное сокращение стоимости в 24 раза и совокупное в 580 раз, сокращение ручной нагрузки в 12–25 раз при сквозной точности 93,0 % с интервалом Вильсона [92,6; 93,4].

Заккрытие формулировок утверждённой темы

Утверждённая тема работы содержит восемь содержательных формулировок. Каждой из них в работе соответствует конкретный раздел, а также архитектурный или экспериментальный компонент разработки. Соответствие приведено в таблице 5.1.

Таблица 33 – Соответствие формулировок утверждённой темы и разделов работы

Формулировка из обоснования актуальности темы	Реализация в работе	Раздел
Гибридная каскадная система	Пятислоенная архитектура каскада (Слой 0 + четыре содержательных слоя)	2.2, 3.1
Семантические возможности генерирующих моделей	Запасной слой 4 на основе большой языковой модели	3.1.1

Продолжение таблицы 33

Формулировка из обоснования актуальности темы	Реализация в работе	Раздел
Предсказательная сила классического машинного обучения	Слой 2 — поатрибутные классификаторы XGBoost на многоязычных векторных представлениях SBERT	3.1.1
Ансамблевая оценка уверенности моделей	Пошаговая фильтрация по уверенности как последовательное конъюнктивное условие на калиброванные пороги слоёв	2.1
Отбраковка фактических ошибок до их попадания в каталог	Слой 3 — байесовский валидатор плотностей пар «бренд × значение»	3.1.1, 3.3.4
Скрытые статистические закономерности каталога	Направленный ациклический граф атрибутов, построенный алгоритмом Hill Climb с критерием BIC	3.1.1

Продолжение таблицы 33

Формулировка из обоснования актуальности темы	Реализация в работе	Раздел
Эксперименты с разнородными источниками данных	OCR-извлечение из изображений Open Food Facts (EasyOCR) и векторные представления CLIP (ViT-B/32) проверены как дополнительный обучающий сигнал, в производственный каскад не включены (см. 3.2.2); сверка эталона яруса 0 с базой USDA Branded Foods через приближённое сопоставление	3.2.5
Интернет-магазин (партнёр — каталог — витрина)	Демонстрационный программный комплекс с REST-интерфейсом обработки партнёрского ввода и руководством пользователя	1.1, 3.1.2, 4.1

Все восемь содержательных формулировок утверждённой темы имеют прямое отображение в разработанной системе и подтверждены экспериментально либо архитектурно. Наиболее нагруженные формулировки — отбраковка фактических ошибок и использование скрытых статистических закономерностей каталога — закрыты слоем байесовского валидатора. Для него в работе пересмотрена роль: вместо самостоятельного классификатора сеть используется как валидатор плотностей.

Научная новизна

Научная новизна работы складывается из трёх содержательных пунктов.

- 1) Доминирование по Парето каскадной архитектуры над всеми безусловными конфигурациями большой языковой модели на задаче обогащения товарных атрибутов. Одновременно достигаются: превышение точности на 9,2 процентных пункта (93,0 % против 83,8 %) и сокращение стоимости в 580 раз по сравнению с прямым обращением к Claude Sonnet 4.5 (сюда входит архитектурный вклад каскада в 24 раза и удешевление модели запасного слоя). Результат подтверждён на проверочной выборке без пересечения товарных кодов (code-disjoint, новые SKU) из 16 360 ячеек и воспроизведён на пяти разных моделях запасного слоя. То есть эффект сохраняется при смене поставщика генеративной модели.
- 2) Сценарий развёртывания на собственных мощностях с открытой большой языковой моделью. Каскад с открытой моделью gpt-oss-120b достигает 90,6 % сквозной точности — на 21,3 процентных пункта выше, чем самостоятельное применение той же модели (69,3 %). Это показывает, что каскадная композиция способна компенсировать слабость малых открытых моделей через специализированные слои. Также это даёт основу для локального развёртывания системы без передачи партнёрских данных во внешние коммерческие сервисы.
- 3) Заранее объявленный отрицательный результат для обучаемого стоимостно-чувствительного маршрутизатора и перенос цикла «аудит — исправления — переобучение — измерение» на категории внутри продуктового домена. Впервые эмпирически установлено, что гипотеза о превосходстве обучаемого XGBoost-маршрутизатора над статическим правилом отклоняется на трёх бюджетах с поправкой Бонферрони. Результат согласуется с выводами бенчмарка RouterBench [12]. Цикл аудита эталонной разметки воспроизведён на трёх категориях из трёх с количественно измеримым приростом сквозной точности.

Ограничения работы

Результаты ограничены рядом обстоятельств, которые честно фиксируются — из них же вырастают перспективы. Во-первых, область определения — три категории пищевой продукции; переносимость на другие домены не проверена. Во-вторых, эталон построен сочетанием слабого надзора и согласия трёх LLM со слепой проверкой Orus 4 — это эталонная разметка второго уровня (ярус аудита), ручную экспертную разметку не делали. В-третьих, мультимодальные сигналы (OCR, CLIP) исследованы как дополнительные обучающие признаки, но в производственный каскад не включены — значимого прироста выше шума не получено; и обучение, и вывод используют только текстовые партнёр-доступные поля. В-четвёртых, проверка велась на пяти европейских языках, устойчивость к другим письменностям не проверена. В-пятых, заранее объявлена только H1; показатели 3.3.2 — апостериорный анализ. В-шестых, разбиение train/test основной оценки — code-disjoint (по идентификатору товара), бренды между обучающим пулом и тестовой выборкой пересекаются на 82–85 % по категориям; контрольное brand-disjoint исследование (см. 3.3.6) показало деградацию точности слоя машинного обучения 0,5 п. п. (с 83,0 % до 82,6 % при нулевом пересечении брендов), что подтверждает обобщающую способность каскада на товарах неизвестных брендов в пределах исследованных категорий. В-седьмых, измеренная точность подвержена циркулярному смещению из-за частичного семантического перекрытия серебряной разметки и LLM-консенсуса. Операционная оценка смещения по разнице consensus и ручной разметки — 4,2 п. п. для каскадной части (95,5 % против 91,3 %) и 6,3 п. п. для сквозной цепочки (93,0 % против 86,7 %); консервативной независимой оценкой служит показатель на ручной разметке Claude Orus 4 (86,7 % сквозной точности на n=566 ячейках с ответом каскада).

Точечный характер байесовского валидатора (значимый прирост точности отбраковки на 3 парах из 20) фиксируется явно как ограничение архитектуры. Из 20 пар «категория-атрибут» в производственной конфигурации валидатор включён точно только на одной паре chocolate/contains_nuts, где даёт +6,6 п. п. точности при росте расходов на 5,3 п. п. (3.3.4). На остальных 19 парах сигнал слишком слабый, чтобы вернуть валидатор в архитектуру. Принцип «изменение роли компонента

вместо его удаления» сформулирован в 3.1.1 и 3.3.4 как самостоятельный методологический вывод; при этом тематически нагруженная формулировка «отбраковки фактических ошибок» в текущей реализации обеспечивается одной парой и не имеет универсального решения.

Перспективы развития темы

Полученные результаты и зафиксированные ограничения задают ряд направлений развития темы.

- 1) Расширение области определения до шести категорий — напитки, зерновые завтраки и косметика. Для них подготовлены схемы атрибутов и серебряная разметка; основной блок — построение эталонной разметки уровня v2 (ярус аудита) и обучение поатрибутных классификаторов Слоя 2. Методология аудита и переобучения переносится без принципиальных изменений.
- 2) Регулярная переаттестация изотонической калибровки и регрессионный контроль ECE на следующих итерациях эталона. Схема перекрёстной калибровки реализована (3.3.11, средняя ECE снижается с 0,070 до 0,043); регрессионный тест ECE с порогом 0,01 реализован в `tests/test_calibration_regression.py` с базовым моментальным снимком метрики.
- 3) Введение мультимодального вывода в производственный режим — расширение прикладного интерфейса на приём изображений и добавление OCR/CLIP в реальном времени; потребует пересмотра контракта взаимодействия с партнёром.
- 4) Полноценный пилот активного обучения с человеческой разметкой [33; 49]. Симуляция на ячейках расхождения каскада и 3-LLM consensus (3.3.24) не показала превосходства активной стратегии над случайной, но пул кандидатов был искусственно ограничен каскадом. Перспектива — независимая разметка ячеек-кандидатов (бюджет ≈ 600 ячеек на категорию) и измерение прироста Слоя 2 относительно случайной разметки той же стоимости.
- 5) Локализация на русский язык и российские каталоги — разработка русскоязычных регулярных выражений Слоя 1 и дообучение Слоя 2 на каталогах российских ритейлеров. SBERT поддерживает русский

в едином векторном пространстве, поэтому трудоёмкость ограничена Слойми 1–2.

- 6) Перенос архитектуры в непродовольственные домены, начиная с электроники. Для смартфонов из каталога PhoneDB подготовлены схема атрибутов и демонстрация холодного старта; полноценное измерение требует эталонной разметки уровня v2 (ярус аудита) и оценки на выборке без пересечения товарных кодов (code-disjoint).

Итак, разработанная гибридная каскадная система показывает, что обогащение товарных каталогов можно одновременно делать точным и дешёвым. Это получается за счёт ступенчатого распределения нагрузки между разнородными слоями. Полученные результаты подтверждают выдвинутую в первой главе работы архитектурную гипотезу о каскадной композиции и закрывают все восемь содержательных формулировок утверждённой темы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Итоги интернет-торговли в России за 2025 год : пресс-релиз АКИТ от 19.02.2026 г. — М. : Ассоциация компаний интернет-торговли, 2026. — 2025. — URL: <https://akit.ru/news/11-5-trln-rublej-akit-podvela-itogi-internet-torgovli-za-2025-god> (дата обращения 19.05.2026).
2. Global Retail E-Commerce Forecast 2024 : research report. — eMarketer, 2024. — 2024. — URL: <https://www.emarketer.com> (дата обращения 03.05.2026).
3. SellerFox : отраслевой обзор индекса каталога Ozon (более 38 млн позиций на 2025 г.) — 2026. — URL: <https://sellerfox.ru> (дата обращения 19.05.2026).
4. Как ускорить работу с товарными карточками в 30 раз : отраслевой кейс / Retail.ru. — 2026. — URL: <https://www.retail.ru/articles/kak-uskorit-rabotu-s-tovarnymi-kartochkami-v-30-raz/> (дата обращения 19.05.2026).
5. Akeneo PIM : open product experience management platform. — Akeneo SAS. — 2026. — URL: <https://akeneo.com> (дата обращения 03.05.2026).
6. Pimcore Platform : open-source data and experience management. — Pimcore GmbH. — 2026. — URL: <https://pimcore.com> (дата обращения 03.05.2026).
7. Salsify Inc. : product experience management platform. — 2026. — URL: <https://www.salsify.com> (дата обращения 19.05.2026).
8. Open source в России 2024 : аналитический отчёт. — М. : Platform V, СберТех, 2024. — 2024. — URL: <https://platformv.sbertech.ru/open-source-v-rossii-2024> (дата обращения 19.05.2026).
9. Chen L., Zaharia M., Zou J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance // Transactions on Machine Learning Research. — 2024. — 2024. — URL: <https://arxiv.org/abs/2305.05176>.
10. Aggarwal P., Madaan A., Yang Y., Mausam. AutoMix: Automatically Mixing Language Models // Advances in Neural Information Processing Systems (NeurIPS 2024). — 2024. — 2024. — URL: <https://arxiv.org/abs/2310.12963>.

11. Ding D., Mallick A., Wang C., Sim R., Mukherjee S., Rühle V., Lakshmanan L. V. S., Awadallah A. H. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing // ICLR 2024. — 2024. — 2024. — URL: <https://arxiv.org/abs/2404.14618>.
12. Hu Q. J., Bieker J., Li X., Jiang N., Keigwin B., Ranganath G., Keutzer K., Upadhyay S. K. RouterBench: A Benchmark for Multi-LLM Routing System : препринт. — arXiv:2403.12031, 2024. — 2024. — URL: <https://arxiv.org/abs/2403.12031> (дата обращения 19.05.2026).
13. Open Food Facts : open product database for food products. — 2026. — URL: <https://world.openfoodfacts.org/data> (дата обращения 03.05.2026).
14. Долгорукова С. А. Управление нормативно-справочной информацией. Сравнительный анализ платформ // Научные записки молодых исследователей. — 2014. — 2014. — URL: <https://cyberleninka.ru/article/n/upravlenie-normativno-spravочноy-informatsiey-sravnitelnyy-analiz-platform> (дата обращения 23.05.2026).
15. Ермакова Л. М. Методы извлечения информации из текста // Вестник Пермского университета. Серия: Математика. Механика. Информатика. — 2012. — Вып. 1 (9). — 2012. — URL: <https://cyberleninka.ru/article/n/metody-izvlecheniya-informatsii-iz-teksta> (дата обращения 23.05.2026).
16. Ванюшкин А. С., Гращенко Л. А. Методы и алгоритмы извлечения ключевых слов // Новые информационные технологии в автоматизированных системах. — 2016. — 2016. — URL: <https://cyberleninka.ru/article/n/metody-i-algoritmy-izvlecheniya-klyuchevyh-slov> (дата обращения 23.05.2026).
17. Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'16). — San Francisco : ACM, 2016. — P. 785–794. — 2016.
18. Ke G., Meng Q., Finley T., Wang T., Chen W., Ma W., Ye Q., Liu T.-Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 3146–3154. — 2017.

19. Афанасьев Г. И., Афанасьев А. Г., Бурмистрова М. В., Тэт Вей Ян Со. Исследование методов машинного обучения для прогнозирования эффективных бизнес-решений в системах электронной коммерции // E-Scio. — 2022. — URL: <https://cyberleninka.ru/article/n/issledovanie-metodov-mashinnogo-obucheniya-dlya-prognozirovaniya-effektivnyh-biznes-resheniy-v-sistemah-elektronnoy-kommertsii> (дата обращения 19.05.2026).
20. Vaswani A., Shazeer N., Parmar N. et al. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 5998–6008. — URL: <https://arxiv.org/abs/1706.03762>.
21. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // NAACL-HLT 2019. — ACL, 2019. — P. 4171–4186. — URL: <https://arxiv.org/abs/1810.04805>.
22. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // EMNLP 2019. — Hong Kong : ACL, 2019. — P. 3982–3992. — URL: <https://arxiv.org/abs/1908.10084>.
23. Reimers N., Gurevych I. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation // EMNLP 2020. — ACL, 2020. — P. 4512–4525. — URL: <https://arxiv.org/abs/2004.09813>.
24. Колонин А. Г. High-performance automatic categorization and attribution of inventory catalogs : препринт. — arXiv:2202.08965, 2022. — 9 p. — URL: <https://arxiv.org/abs/2202.08965> (дата обращения 19.05.2026).
25. Большакова Е. И., Воронцов К. В., Ефремова Н. Э., Клышинский Э. С., Лукашевич Н. В., Сапин А. С. Автоматическая обработка текстов на естественном языке и анализ данных : учебное пособие. — М. : Изд-во НИУ ВШЭ, 2017. — 269 с. — URL: https://www.hse.ru/data/2017/08/12/1174382135/NLP_and_DA.pdf (дата обращения 19.05.2026).
26. Karamanolakis G., Ma J., Dong X. L. TXtract: Taxonomy-Aware Knowledge Extraction for Thousands of Product Categories // ACL 2020. — 2020. — P. 8489–8502. — URL: <https://aclanthology.org/2020.acl-main.751/>.
27. Yang L., Wang Q., Yu Z., Kulkarni A., Sanghai S., Shu B., Elsas J., Kanagal B. MAVe: A Product Dataset for Multi-source Attribute Value Extraction // WSDM 2022. — 2022. — P. 1256–1265. — URL: <https://dl.acm.org/doi/10.1145/>

3488560.3498377.

28. Zheng G., Mukherjee S., Dong X. L., Li F. OpenTag: Open Attribute Value Extraction from Product Profiles // KDD 2018. — 2018. — P. 1049–1058. — 2018. — URL: <https://dl.acm.org/doi/10.1145/3219819.3219839>.

29. Yang L., Wang Q., Chi J., Liu J., Wang J., Feng F., Xu Z., Fang Y., Huang L., Liu D. EAVE: Efficient Product Attribute Value Extraction via Lightweight Sparse-layer Interaction // Findings of ACL: EMNLP 2024. — Miami, FL : ACL, 2024. — P. 1491–1505. — DOI: 10.18653/v1/2024.findings-emnlp.80. — 2024. — URL: <https://aclanthology.org/2024.findings-emnlp.80/> (дата обращения 28.05.2026).

30. Fang C., Li X., Fan Z., Xu J., Nag K., Korpeoglu E., Kumar S., Achan K. LLM-Ensemble: Optimal Large Language Model Ensemble Method for E-commerce Product Attribute Value Extraction // SIGIR 2024. — Washington, DC : ACM, 2024. — P. 2910–2914. — DOI: 10.1145/3626772.3661357. — 2024. — URL: <https://dl.acm.org/doi/10.1145/3626772.3661357> (дата обращения 28.05.2026).

31. Ong I., Almahairi A., Wu V., Chiang W.-L., Wu T., Gonzalez J. E., Kadous M. W., Stoica I. RouteLLM: Learning to Route LLMs with Preference Data // ICLR 2025. — 2025. — 2025. — URL: <https://arxiv.org/abs/2406.18665> (дата обращения 28.05.2026).

32. Ratner A., Bach S. H., Ehrenberg H. R., Fries J., Wu S., Ré C. Snorkel: Rapid Training Data Creation with Weak Supervision // Proceedings of the VLDB Endowment. — 2017. — Vol. 11, № 3. — P. 269–282. — 2017.

33. Settles B. Active Learning Literature Survey. — University of Wisconsin–Madison, 2010. — Computer Sciences Technical Report 1648. — 2010. — URL: <https://burrsettles.com/pub/settles.activelearning.pdf> (дата обращения 19.05.2026).

34. Дюк В. А. Экспериментальное исследование реакции алгоритмов машинного обучения на ошибки разметки данных // Дифференциальные уравнения и процессы управления. — 2022. — № 3. — 2022. — URL: <https://cyberleninka.ru/article/n/eksperimentalnoe-issledovanie-reaktsii-algoritmov-mashinnogo-obucheniya-na-oshibki-razmetki-dannyh> (дата обращения 23.05.2026).

35. Platt J. C. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods // Advances in Large Margin Classifiers. — MIT Press, 1999. — P. 61–74. — 1999.

36. Niculescu-Mizil A., Caruana R. Predicting Good Probabilities with Supervised Learning // ICML 2005. — Bonn : ACM, 2005. — P. 625–632. — 2005.
37. Guo C., Pleiss G., Sun Y., Weinberger K. Q. On Calibration of Modern Neural Networks // ICML 2017. — PMLR, 2017. — Vol. 70. — P. 1321–1330. — 2017.
38. Шунина Ю. С., Алексеева В. А., Клячкин В. Н. Критерии качества работы классификаторов // Вестник Ульяновского государственного технического университета. — 2015. — 2015. — URL: <https://cyberleninka.ru/article/n/kriterii-kachestva-raboty-klassifikatorov> (дата обращения 23.05.2026).
39. Ковалев А. А., Кузнецов Б. К., Ядченко А. А., Игнатенко В. А. Оценка качества бинарного классификатора в научных исследованиях // Проблемы здоровья и экологии. — 2020. — Т. 4, № 66. — С. 105–113. — 2020. — URL: <https://cyberleninka.ru/article/n/otsenka-kachestva-binarnogo-klassifikatora-v-nauchnyh-issledovaniyah> (дата обращения 23.05.2026).
40. Kohavi R. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection // IJCAI 1995. — Morgan Kaufmann, 1995. — P. 1137–1143. — 1995.
41. Wilson E. B. Probable Inference, the Law of Succession, and Statistical Inference // Journal of the American Statistical Association. — 1927. — Vol. 22, № 158. — P. 209–212. — 1927.
42. Pearl J. Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference. — San Francisco : Morgan Kaufmann, 1988. — 552 p. — 1988.
43. Heckerman D., Geiger D., Chickering D. M. Learning Bayesian Networks: The Combination of Knowledge and Statistical Data // Machine Learning. — 1995. — Vol. 20, № 3. — P. 197–243. — 1995.
44. Тулупьев А. Л., Николенко С. И., Сироткин А. В. Байесовские сети: логико-вероятностный подход. — СПб. : Наука, 2006. — 607 с. — 2006.
45. Ankan A., Panda A. pgmpy: Probabilistic Graphical Models using Python // SciPy 2015. — 2015. — P. 6–11. — 2015.

46. Schwarz G. Estimating the Dimension of a Model // Annals of Statistics. — 1978. — Vol. 6, № 2. — P. 461–464. — 1978.

47. Артемов А. А. Модели машинного обучения в FMCG-ритейле: архитектура и эксплуатация в условиях высокой изменчивости данных // Вестник науки. — 2025. — URL: <https://cyberleninka.ru/article/n/modeli-mashinnogo-obucheniya-v-fmcg-riteyle-arhitektura-i-ekspluatatsiya-v-usloviyah-vysokoy-izmenchivosti-dannyh> (дата обращения 19.05.2026).

48. Сергеев С. А. Управление запасами в условиях маркетплейсов: почему не работают стандартные методы? // Вестник Академии знаний. — 2024. — URL: <https://cyberleninka.ru/article/n/upravlenie-zapasami-v-usloviyah-marketpleysov-pochemu-ne-rabotayut-standartnye-metody> (дата обращения 19.05.2026).

49. Гилязов Р. А., Турдаков Д. Ю. Активное обучение и краудсорсинг: обзор методов оптимизации разметки данных // Труды Института системного программирования РАН. — 2018. — Т. 30, № 2. — С. 215–250. — DOI: 10.15514/ISPRAS-2018-30(2)-11. — URL: <https://cyberleninka.ru/article/n/aktivnoe-obuchenie-i-kraudsorsing-obzor-metodov-optimizatsii-razmetki-dannyh> (дата обращения 19.05.2026).

50. Хорошевский В. Ф. Пространства знаний в сети Интернет и Semantic Web (Часть 1) // Искусственный интеллект и принятие решений. — 2008. — URL: http://aidt.ru/index.php?Itemid=114&catid=43&id=137&lang=ru&option=com_content&view=article (дата обращения 19.05.2026).

51. Brown T. B., Mann B., Ryder N. et al. Language Models are Few-Shot Learners // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 1877–1901. — 2020. — URL: <https://arxiv.org/abs/2005.14165>.

52. Anthropic. Introducing Claude Sonnet 4.5 : анонс модели. — Anthropic, 29.09.2025. — URL: <https://www.anthropic.com/news/claude-sonnet-4-5> (дата обращения 03.05.2026).

53. Breiman L. Random Forests // Machine Learning. — 2001. — Vol. 45, № 1. — P. 5–32. — 2001.

54. Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd ed. — New York : Springer, 2009. — 745 p. — 2009.
55. Shwartz-Ziv R., Armon A. Tabular Data: Deep Learning is Not All You Need // Information Fusion. — 2022. — Vol. 81. — P. 84–90. — 2022.
56. McElfresh D., Khandagale S., Valverde J., Prasad V., Ramakrishnan G., Goldblum M., White C. When Do Neural Nets Outperform Boosted Trees on Tabular Data? // Advances in Neural Information Processing Systems. — 2023. — Vol. 36 (NeurIPS 2023 Datasets and Benchmarks Track). — 2023.
57. Radford A., Kim J. W., Hallacy C. et al. Learning Transferable Visual Models From Natural Language Supervision // ICML 2021. — PMLR, 2021. — P. 8748–8763. — 2021. — URL: <https://arxiv.org/abs/2103.00020>.
58. USDA FoodData Central : public nutrient database, SR Legacy release. — U. S. Department of Agriculture, Agricultural Research Service. — 2026. — URL: <https://fdc.nal.usda.gov> (дата обращения 03.05.2026).

ПРИЛОЖЕНИЕ А

Электронные материалы

На рисунке А.1 приведён QR-код со ссылкой на репозиторий с исходным кодом, ноутбуком §3.3 в исполняемом виде, демонстрационным программным комплексом и текстом настоящей работы.



Рисунок А.1 – QR-код на репозиторий проекта